

MQ for z/OS Performance Report (v1.0)

IBM MQ Advanced for z/OS 10.0

and

IBM MQ for z/OS 10.0

September 2026

IBM MQ Performance

IBM UK Laboratories

Hursley Park

Winchester

Hampshire

SO21 2JN

Take Note!

Before using this report, please be sure to read the paragraphs on “disclaimers”, “warranty and liability exclusion”, “errors and omissions” and other general information paragraphs in the “Notices” section below.

First edition, September 2026. This edition applies to IBM MQ for z/OS 10.0 and IBM MQ Advanced for z/OS 10.0 (and to all subsequent releases and modifications until otherwise indicated in new editions).

© Copyright International Business Machines Corporation 2026.

All rights reserved.

Note to U.S. Government Users Documentation related to restricted rights. Use, duplication or disclosure is subject to restrictions set forth in GSA ADP Schedule contract with IBM Corp.

Notices

DISCLAIMERS

The performance data contained in this report were measured in a controlled environment. Results obtained in other environments may vary significantly.

You should not assume that the information contained in this report has been submitted to any formal testing by IBM.

Any use of this information and implementation of any of the techniques are the responsibility of the licensed user. Much depends on the ability of the licensed user to evaluate the data and to project the results into their own operational environment.

WARRANTY AND LIABILITY EXCLUSION

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law:

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE.

Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore this statement may not apply to you.

In Germany and Austria, notwithstanding the above exclusions, IBM’s warranty and liability are governed only by the respective terms applicable for Germany and Austria in the corresponding IBM program license agreement(s).

ERRORS AND OMISSIONS

The information set forth in this report could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; any such change will be incorporated in new editions of the information. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this information at any time and without notice.

INTENDED AUDIENCE

This report is intended for Architects, Systems Programmers, Analysts and Programmers wanting to understand the performance characteristics of **IBM MQ for z/OS version 10.0**. The information is not intended as the specification of any programming interfaces that are provided by IBM MQ. Full descriptions of the IBM MQ facilities are available in the product publications. It is assumed that the reader is familiar with the concepts and operation of IBM MQ.

LOCAL AVAILABILITY

References in this report to IBM products or programs do not imply that IBM intends to make these available in all countries in which IBM operates. Consult your local IBM representative for information on the products and services currently available in your area.

ALTERNATIVE PRODUCTS AND SERVICES

Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program,

or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

USE OF INFORMATION PROVIDED BY YOU

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

TRADEMARKS and SERVICE MARKS

The following terms, used in this publication, are trademarks or registered trademarks of the IBM Corporation in the United States or other countries or both:

- IBM®
- z/OS®
- IBM MQ®
- CICS®
- Db2 for z/OS®
- IMS™
- MVS™
- z15™
- z16™
- z17™
- FICON®
- WebSphere®

Other company, product and service names may be trademarks or service marks of others.

EXPORT REGULATIONS

You agree to comply with all applicable export and import laws and regulations.

Summary of Amendments

Date	Changes
2026-09	Version 1.0 - Initial Version.

Table of contents

1	Performance highlights	1
2	Existing function	2
	General Statement Of Regression	2
	Storage Usage	3
	CSA Usage	3
	Initial CSA usage	3
	CSA usage per connection	3
	Object Sizes	4
	PAGESET(0) Usage	4
	Virtual Storage Usage	5
	Capacity of the queue manager and channel initiator	6
	How much storage does a connection use?	6
	How many clients can I connect to my queue manager?	7
	31-bit storage usage with SVRCONN channels	7
	64-bit storage usage with SVRCONN channels	8
	How many SVRCONN channels can I run?	10
	How many MCA channels <i>can</i> I run to or from my queue manager?	11
3	New function for 10.0	12
	New function for version 10.0.0	13
4	Long CONNAME support	14
	Impact to Page Set Zero usage	15
	Minimum Objects per 4KB page	16
	Maximum Objects per 4KB page	16
	CLNTCONN channels	16
	Impact of multiple host/port names in the CONNAME attribute on channel start	17
	Using CSQUTIL to define MQ channels with long CONNAME values	19
	Conclusions	20
5	Log Output Buffer Size (OUTBUFF)	21
	Configuration	21
	What does OUTBUFF affect?	21
	Monitoring	22
6	OpenTelemetry support	23
	Configuration	24
	Span emission	24
	Trace propagation	25
	Where are spans emitted?	26
	What does OTel cost?	26

How Do the Different Propagation Styles Compare?	27
Scalability	30
Conclusions	31
Appendix A Regression	32
Private Queue	33
Non-persistent out-of-syncpoint workload	33
Maximum throughput on a single pair of request/reply queues	33
Scalability of request/reply model across multiple queues	34
Non-persistent server in-syncpoint workload	36
Maximum throughput on a single pair of request/reply queues	36
Scalability of request/reply model across multiple queues	37
Persistent server in-syncpoint workload	41
Maximum throughput on a single pair of request/reply queues	41
Upper bounds of persistent logging rate	42
CICS Workload	43
Shared Queue	44
Non-persistent out-of-syncpoint workload	44
Maximum throughput on a single pair of request/reply queues	44
Non-persistent server in-syncpoint workload	48
Maximum throughput on a single pair of request/reply queues	48
Data sharing non-persistent server in-syncpoint workload	52
Moving messages across channels	55
Channel compression	55
Streaming messages across channels	55
Non-persistent in-syncpoint - 1 to 5 sender-receiver channels	57
Channel compression using ZLIBFAST	59
Channel compression using ZLIBHIGH	60
Channel compression using ZLIBFAST on SSL channels	61
Channel compression using ZLIBHIGH on SSL channels	62
Streaming workload between 2 z/OS queue managers	63
Moving messages across cluster channels	64
Bind-on-open	65
Bind-not-fixed	67
Moving messages across SVRCONN channels	69
Client pass through tests using SHARECNV(0)	70
Client pass through tests using SHARECNV(1)	71
IMS Bridge	72
Commit mode 0 (commit-then-send)	73
Commit mode 1 (send-then-commit)	74
Trace	75
Queue manager global trace	75
Channel initiator trace	77
Advanced Message Security	78
Background	78
AMS regression test configuration	79
Impact of AMS policy type on 2KB request/reply workload	80
Impact of AMS policy type on 64KB request/reply workload	81
Impact of AMS policy type on 4MB request/reply workload	82
Appendix B System configuration	83

Chapter 1

Performance highlights

This report focuses on performance changes since previous versions (9.3 and 9.4) and on the performance of new function in this release.

SupportPac [MP16](#) “Capacity Planning and Tuning Guide” will continue to be the repository for ongoing advice and guidance learned as systems increase in power and experience is gained.

In IBM MQ for z/OS 10.0, there are a number of features that can benefit the performance of the MQ queue manager:

As introduced in [IBM MQ for z/OS 9.3](#), all new queue managers have, by default, 64-bit log RBA and are capable of having up to 310 active logs. All measurements run in this report use 64-bit RBA queue managers.

Chapter 2

Existing function

General statement of regression

CPU costs and throughput are not significantly different in version 10.0 for typical messaging workloads when compared with versions 9.3 and 9.4.

The reader can see how that statement has been determined by reviewing details of the regression test cases in the [Regression](#) appendix.

Storage usage

Virtual storage constraint relief has not been a primary focus of this release.

CSA usage

Common Service Area (CSA) storage usage is important as the amount available is restricted by the amount of 31-bit storage available and this is limited to an absolute limit of 2GB.

The CSA is allocated in all address spaces in an LPAR, so its use reduces the available private storage for all address spaces.

In real terms, the queue manager does not have 2GB of storage to use - as there is some amount used by MVS for system tasks and it is possible for individual customer sites to set the limit even lower.

From the storage remaining of the 2GB of 31-bit storage, a large (but configurable) amount of storage may be used by the queue manager for buffer pools. This storage usage may be reduced with the use of 64-bit buffer pools.

The storage remaining is available for actually connecting to the queue manager in a variety of ways and using IBM MQ to put and get messages.

Initial CSA usage

CSA usage for 10.0 is similar to both 9.3 and 9.4 when similarly configured queue managers are started. On our systems this is approximately 6MB per queue manager.

CSA usage per connection

CSA usage has seen little change in the following releases: 9.3, 9.4 and 10.0.

- For local connections, MCA channels and SVRCONN channels with SHARECNV(0), CSA usage is 2.47KB per connection.
- For SVRCONN channels with SHARECNV(1), CSA usage is approximately 4.9KB per connection.
- For SVRCONN channels with SHARECNV(5), CSA usage is approximately 3KB per connection, based on 5 clients sharing the channel instance.
- For SVRCONN channels with SHARECNV(10), CSA usage is approximately 2.7KB per connection, based on 10 clients sharing the channel instance.

Object Sizes

When defining objects, the queue manager may store information about that object in pageset 0 and may also require storage taken from the queue manager's extended private storage allocation.

IBM MQ for z/OS 10.0 increases the maximum length of the CONNAME attribute from 48 to 264 characters, bringing the maximum length in line with other MQ platforms.

This change affects the following channel types:

SDR, SVR, RQSTR, CLUSSDR and CLUSRCVR.

In addition to these changes, IBM MQ for z/OS 10.0 has optimised how certain channel attributes are stored on page set zero and this change affects all channel types except CLNTCONN channels.

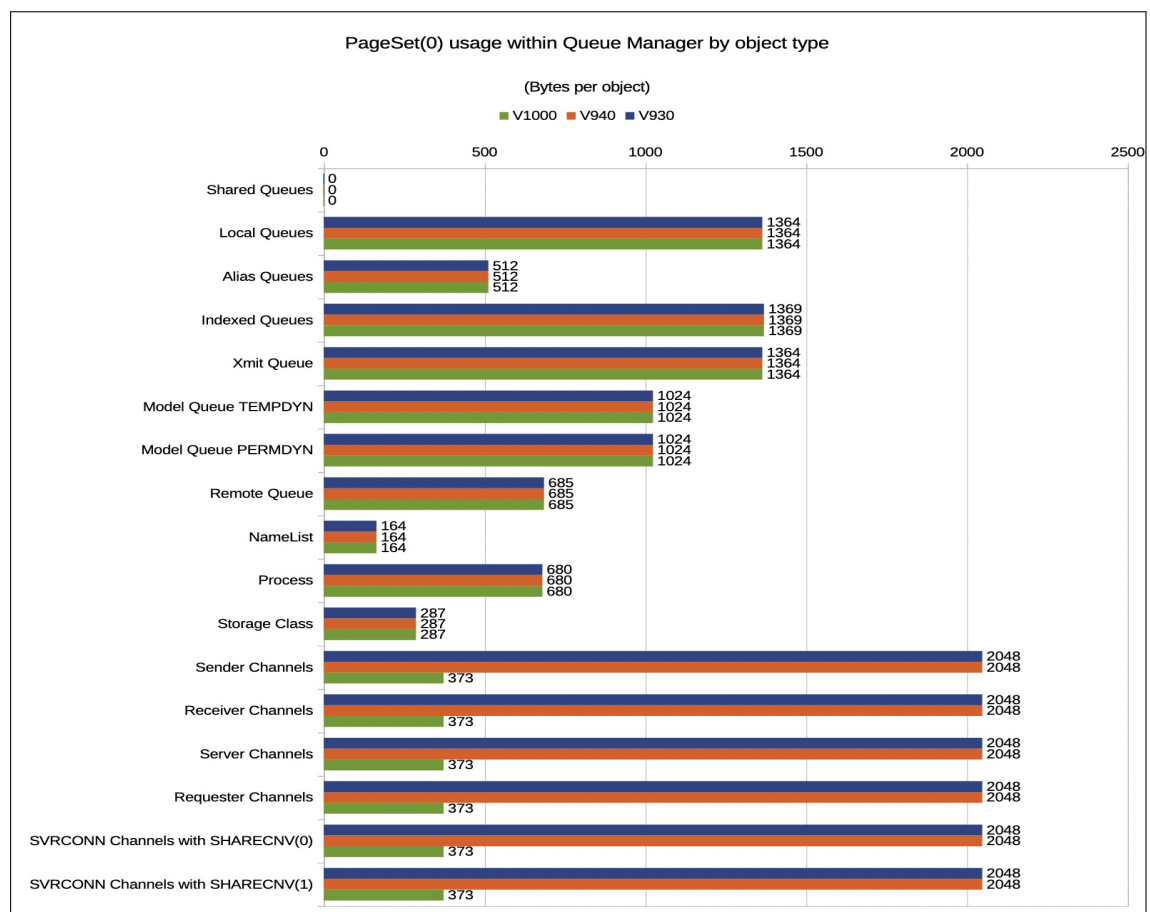
This change is detailed further in the [Long CONNAME support](#) section.

The data shown on the following 2 charts only includes the storage used when defining the objects.

For clarity, these calculations were based on defining 1000 of each type of MQ object, specifying the minimum attributes necessary and determining the increase in page set zero usage.

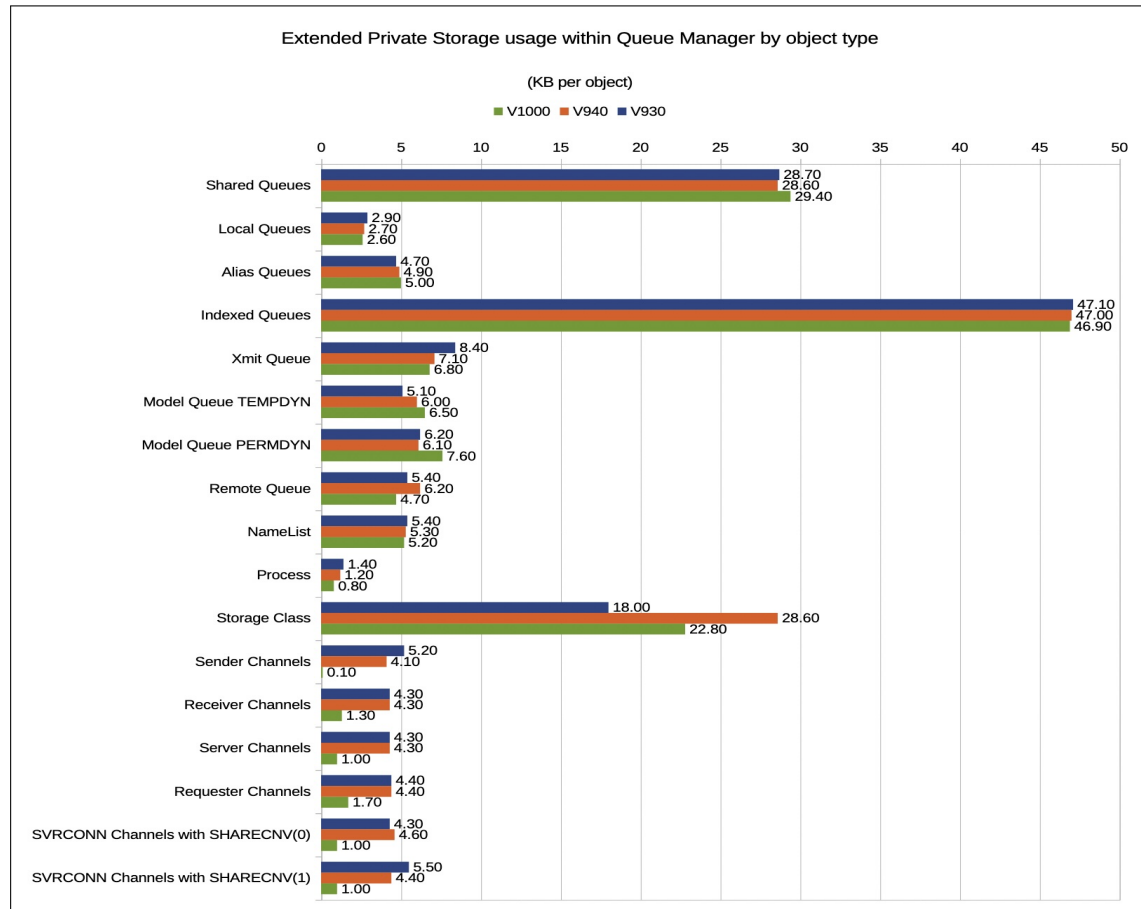
PAGESET(0) Usage

Chart: Pageset usage by object type



Virtual Storage Usage

Chart: Virtual Storage usage by object type



As can be seen in both the “PAGESET(0) Usage” and the “Virtual Storage Usage” charts, channel object types see a significant decrease in usage in IBM MQ for z/OS 10.0.

This is explained in the [Long CONNAME support](#) section, but is essentially the outcome of defining the objects with the minimum attributes required, and benefitting from the storage of certain string-type attributes in variable length fields.

Capacity of the queue manager and channel initiator

How much storage does a connection use?

When an application connects to a queue manager, an amount of storage is allocated from the queue manager's available storage.

Some of this storage is held above 2GB, such as security data, and other data is stored on storage taken from that available below the 2GB bar. In the following examples, only allocations from below the 2GB bar are reported.

From version 9.0 onwards, the typical storage usage is approximately 17KB per connection, however there is additional usage in the following (non-exhaustive) cases:

- Where connection is over a SHARECNV(1) channel, the usage increases to 27.4KB.
- Where connection is over a SHARECNV(10) channel and has a CURSHCNV of 10, the usage is 18.8KB per connection (188KB per channel).
- Where connection is over a SHARECNV(1) channel to shared queues - either CFLEVEL(4) or CFLEVEL(5) backed by SMDS, the storage is 34KB, giving an additional shared queue overhead of 7KB.

These numbers are based upon the connecting applications accessing a small number of queues. If your application has more than 32 objects open, the amount of storage will be increased.

If the number of messages held in a unit of work is large and the size of the messages is large then additional lock storage may be required.

How many clients can I connect to my queue manager?

The maximum number of clients you can connect to a queue manager depends on a number of factors, for example:

- Storage available in the queue manager's address space.
- Storage available in the channel initiator's address space.
- How large the messages being put or got are.
- Whether the channel initiator is already running its maximum number of connected clients (either 9,999 or the value specified by channel attributes like MAXCHL).

Since [IBM MQ for z/OS 9.4](#), some of the storage required by client channels has been changed to use 64-bit storage.

The 64-bit storage usage is discussed further in the “Increased channel initiator capacity for SVRCONN channel” section of the [IBM MQ for z/OS 9.4](#) performance report.

31-bit storage usage with SVRCONN channels

The table below shows the typical **31-bit** footprint when connecting a client application to a z/OS queue manager via the channel initiator.

The value used in the SHARECNV channel attribute can affect the footprint and consideration as to the setting should be taken. Guidance on the SHARECNV attribute can be found in SupportPac [MP16](#) “Capacity Planning and Tuning Guide”.

		31-bit Channel initiator footprint (KB / SVRCONN channel)			
		Message size (KB)			
IBM MQ release	SHARECNV	1	10	32	64
9.3	0	87	100	161	194
9.4	0	87	90	94	96
10.0	0	87	89	93	95
9.3	1	168	191	245	278
9.4	1	164	174	203	203
10.0	1	165	174	203	205
9.3	10	222	243	309	340
9.4	10	219	236	263	265
10.0	10	228	240	275	272

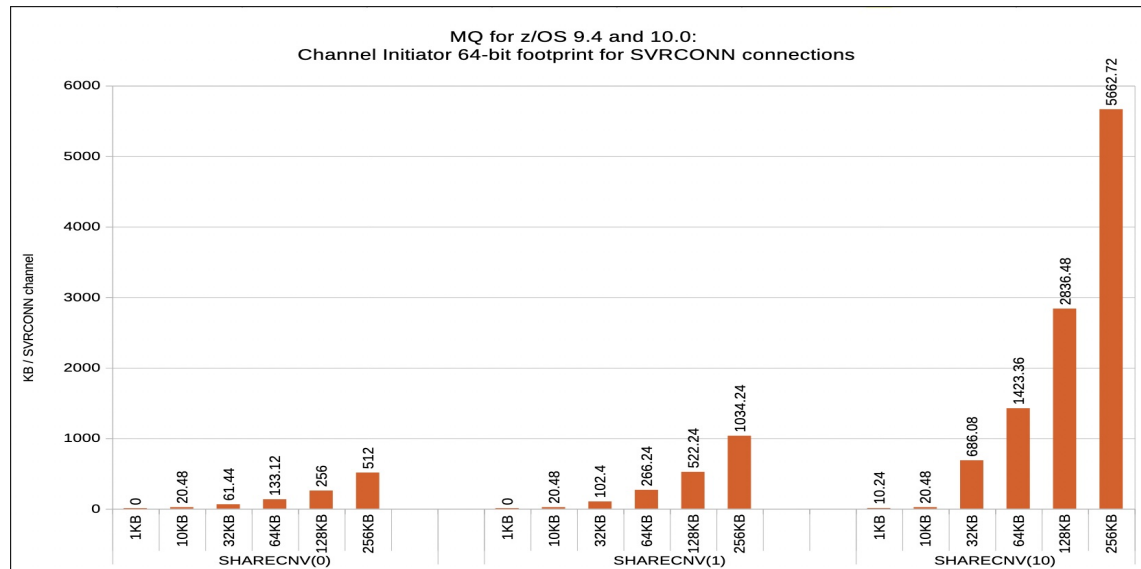
Note: For the SHARECNV(10) channel measurements, the channels are running with 10 conversations per channel instance, so the cost per conversation would be the value in the table divided by 10.

The table shows for SVRCONN channels with SHARECNV(10) configured, there is a 9KB increase per SVRCONN channel for 1KB messages and a 12.1KB increase per SVRCONN channel with 32KB messages on IBM MQ for z/OS 10.0.

64-bit storage usage with SVRCONN channels

The chart below shows the typical **64-bit** footprint when connecting a client application to a z/OS queue manager via the channel initiator.

Chart: 64-bit usage for MQ 9.4 and MQ 10.0 SVRCONN channels



Notes on chart:

- The 64-bit private storage is assigned in a range of pool sizes from 4KB doubling in size up to 1MB. For messages larger than 1MB, storage is allocated on a per MB requirement.
- In the above chart, we see relatively high 64-bit storage requirements. For example
 - SHARECNV(0): Approximately 2 times the message size.
 - SHARECNV(1): Approximately 4 times the message size.
 - SHARECNV(10): Approximately 22 times the message size per channel, or 2.2 times the message size per conversation.
- It is worth noting that the message sizes used in the chart are not optimised for the 64-bit pool sizes used in the channel initiator, such that for each of the messages used of 32KB or more, storage was used from the next sized-buffer pool, i.e. a 32KB message required a 64KB buffer. This is simply because the storage requirements include message headers, which takes the message size over the lower pool-size threshold.
 - With a SHARECNV(0) channel using a 31KB message, we would use a single 32KB buffer, reducing the 64-bit private storage usage by half.
- Additional storage is required for channels with non-zero SHARECNV values, and the storage requirements for channels with SHARECNV(1) are relatively high compared to SVRCONN channels with SHARECNV of greater than 1 and matching CURSHCNV.

Rule of Thumb

For all SVRCONN channel conversations, allow space for the message header(s), approximately 500 bytes, and round up to the nearest buffer pool size (using a power of 2) e.g. 4KB, 8KB, 16KB, etc.

- For SVRCONN channels with SHARECNV(0), this gives an approximate 64-bit storage requirement per channel.
- For SVRCONN channels with a non-zero SHARECNV value but only a single conversation i.e. CURSHCNV(1), double the calculated space to determine an approximate 64-bit storage requirement.
- For SVRCONN channels where multiple conversations are sharing a single SVRCONN channel, i.e. SHARECNV of greater than 1, and CURSHCNV of greater than 1, multiply the size by the number of conversations and then again by 1.1 to determine an approximate 64-bit storage requirement.

How many SVRCONN channels can I run?

Prior to MQ 9.4, the storage used for SVRCONN channels was sourced entirely by 31-bit storage. Since MQ 9.4, storage used by SVRCONN channels is sourced from both 31-bit and 64-bit storage.

This does mean there is a change in how to calculate the number of SVRCONN channels that can be supported.

When the channel initiator is started, it will log a message similar to the following:

```
“CSQX004I @VKW8 CSQXSPRM Channel initiator storage usage: local storage: used 141MB,  
free 1375MB: above bar: used 187MB, free >10GB ”.
```

In our example, this means that the channel initiator had 1516MB of 31-bit storage available and has 1375MB available for channels to be started. To avoid letting the channel initiator run short on storage, it is advisable to aim to keep the usage below 80% of the total available. In the example this means keeping the channel initiator storage usage below 1212MB, which in turn means that there is 1071 MB available for channels.

If the workload is expected to be clients connecting via SHARECNV(1) SVRCONN channels and using 32KB messages, we could predict that the MQ version 10 channel initiator could support a maximum of 5,402 running SVRCONN channels before using 80% of the available 31-bit storage.

Additionally, some 64-bit storage may be required. The IBM documentation suggests the channel initiator JCL is configured with MEMLIMIT of 2GB. Whether 2GB is sufficient will depend on the messages flowing through your SVRCONN channels.

For example, continuing with 5,402 SVRCONN channels with SHARECNV(1) using 32KB messages, we would expect a total of 540MB of 64-bit storage being required.

However for 9,999 SVRCONN channels configured with SHARECNV(0) using 256KB messages, each would require 512KB of 64-bit storage. This would mean that approximately 5GB of 64-bit storage would be required to support all of those channels running concurrently.

How many MCA channels can I run to or from my queue manager?

This depends on the size of the messages flowing through the channel.

A channel will hold onto a certain amount of storage for its lifetime. This footprint depends on the size of the messages.

	Channel initiator footprint per MCA channel			
	Message Size			
IBM MQ release	1KB	32KB	64KB	4MB
9.3	91	100	109	1128
9.4	90	100	109	1128
10.0	106	116	125	1139
Overhead of message size increase on 1KB messages		+9 KB	+19 KB	+1032 KB

The table shows that there is a 16KB increase in a channel footprint on IBM MQ for z/OS 10.0.

Chapter 3

New function for 10.0

New function for version 10.0.0

This release has introduced a number of items that can affect performance and these include:

- Queue Manager:
 - [Long CONNAME](#) support
 - 64-bit [OUTBUFF](#)
 - Support for [OpenTelemetry](#) in MQ for z/OS

Each of these features will be detailed in subsequent chapters.

In IBM MQ for z/OS 10.0, there has been the deprecation of many TLS1.2 ciphers, and as a result MQ performance measurements using TLS protection on MQ channels are now reported using only TLS1.3 ciphers.

Chapter 4

Long CONNAME support

In previous MQ release performance reports, such as [IBM MQ for z/OS 9.4](#) and [MP16](#) “Capacity Planning and Tuning Guide”, there is a section reporting “Pageset(0) usage” for MQ objects, which includes each of the channel types. This data is repeated in this report in the [Object Sizes](#) section.

Prior to IBM MQ for z/OS 10.0, it was reported that each of the channel types used **2048 bytes** of page set zero usage, or that **2 objects could be stored per 4KB page**.

These calculations were based on defining 1000 of each type of MQ object and determining the increase in page set zero usage.

Prior to IBM MQ for z/OS 10.0, it was irrelevant whether any of the MQ channels had the minimum number of attributes specified for that channel type or whether every attribute was configured to the maximum length - in either case, MQ for z/OS could only store 2 channel objects per 4KB page on page set zero.

For example, SDR-type channels where the SENDEXIT attribute is either blank or contains all eight exit names each of eight characters in length, would use the same amount of page set zero storage.

IBM MQ for z/OS 10.0 increases the maximum length of the CONNAME attribute from 48 to 264 characters, bringing the maximum length in-line with other MQ platforms. This change affects the following channel types:

- SDR, SVR, RQSTR, CLUSSDR and CLUSRCVR.

Increasing the length of the CONNAME attribute makes it easier to use a comma separated list of host/port names. Typically, only one hostname is required but you can provide multiple hostnames to configure multiple connections with the same properties. The use of [multiple hostnames](#) may affect the channel start rate, as the connections are usually tried in the order they are specified in the connection list until a connection is successfully established.

Impact to Page Set Zero usage

In addition to supporting the increase in CONNAME length, IBM MQ for z/OS 10.0 channel objects have been changed to store certain string-type attributes as variable length fields in page set zero and this change affects all channel types except CLNTCONN channels (see [CLNTCONN channels](#)).

If not exploiting the additional characters in the CONNAME attribute, the page set usage for channel objects will typically decrease, or at worst, have the same page set zero usage as prior MQ releases.

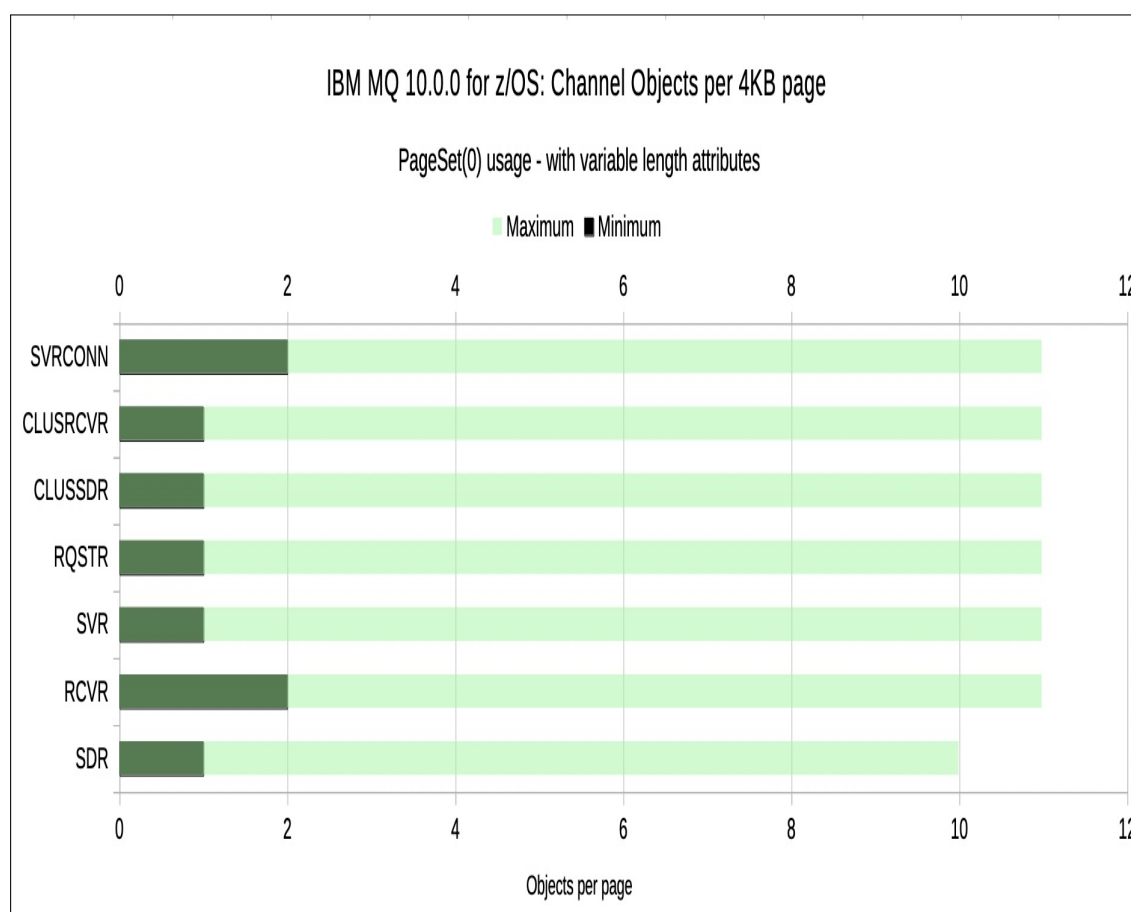
In IBM MQ for z/OS 10.0, the page set usage is reported in up to three configurations for each channel type:

1. Minimum size of an MQ channel, where the minimum attributes are configured, and any mandatory variable length attributes are configured with short values.
2. Maximum size of an MQ channel, using pre-v10.0 CONNAME length.
3. Maximum size of an MQ channel, using v10.0 maximum CONNAME length.

In our measurements, configuration 2 resulted in the MQ channel objects using 2048 bytes or 2 objects per page, matching earlier MQ for z/OS releases.

However, it is unlikely that many channel objects are defined with all string-type characters at maximum length, and as a result we offer the following chart.

Chart: Pageset usage by channel type



The chart shows that for a sender-type (SDR) channel, the maximum number of objects that can be stored in a single 4KB page is 10, and the minimum is 1.

This means that if a sender-type channel is defined with a long CONNAME, for example using the 264 bytes, and all the variable length attributes configured to their maximum lengths, only 1 similarly configured sender channel would be stored per 4KB page.

By contrast a sender-type channel that is configured with the minimum mandatory fields, i.e. example XMITQ and CONNAME only, up to 10 channels could be stored per 4KB page, depending on the length of the values in those attributes.

These channels are based on all channel definitions for a single channel type being defined with the same length attributes and should channel object “A” require more than 2048 bytes and the next object “B” defined requires less than 2048 bytes, then both objects may be stored on a single page.

Minimum Objects per 4KB page

For a receiver or SVRCONN-type channel, neither of which support CONNAME, a minimum of 2 objects will be stored per 4KB page of page set zero.

For all other channel types, which do support CONNAME, it is possible that 1 object may be stored per 4KB page on page set zero.

Maximum Objects per 4KB page

For a sender-type object, a maximum of 10 SDR-type channel objects may be stored on a single 4KB page on page set zero.

For all other channel types, a maximum of 11 channel objects may be stored on a single 4KB page on page set zero.

CLNTCONN channels

CLNTCONN channels on z/OS have not been updated to support a CONNAME of greater than 48 characters in length.

Although CLNTCONN channels can be defined on z/OS, they cannot be exported to a client system because the CSQUTIL MAKECLNT tool used for this purpose was deprecated in IBM MQ 9.1.0 for z/OS.

Impact of multiple host/port names in the CONNAME attribute on channel start

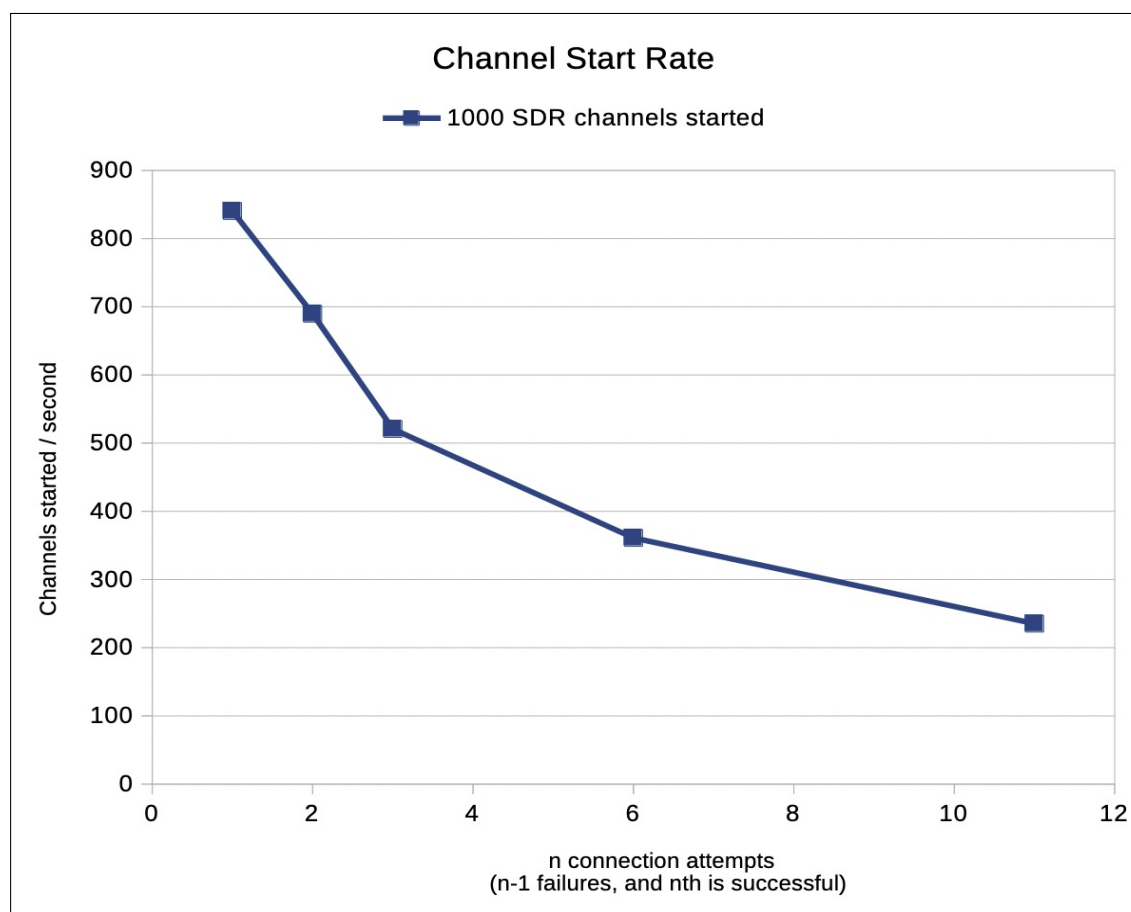
The connections are usually tried in the order they are specified in the connection list until a connection is successfully established.

Should the list contain multiple host/port names that are not available at the head of the list, then there is an impact to the performance of channel start.

In [MP16](#) “Capacity Planning and Tuning Guide” section titled “Channel start / stop rates and cost”, it is documented that to start 1000 channels on IBM z17 (9175-703), a rate of 826 channel starts per second was achieved at a cost of 200 microseconds per channel.

When this measurement is repeated but with increasing numbers of host/port names where the connection attempt is not successfully established before the final connection is established the following chart can be created:

Chart: Impact of increasing unsuccessful connection attempts on channel start rate



In the low-latency test environment where the remote end of the channel was a separate LPAR on the same SYSPLEX, a channel configured with 1 host/port name took 1190 microseconds to start.

When configuring multiple host/port names in the CONNAME attribute, each unsuccessful connection added 330 microseconds to the channel start time. In a higher-latency network (greater distance, generally busier or even less responsive target system), the elapsed time to attempt the connection may be even higher.

In terms of CPU attributed to the MQ channel initiator at channel start, we found that each *unsuccessful attempt cost 90 CPU microseconds* on IBM z17 (9175-703) in addition to the 200 CPU microseconds incurred for the successful connection.

For each unsuccessful connection attempt, the channel initiator will log the CSQX202E message, for example:

```
CSQX202E @VTS1 CSQXRCTL Connection or remote listener unavailable,  
channel VTS1_VTS2_0001  
connection 10.20.137.22  
TRPTYPE=TCP RC=00000468 (ECONNREFUSED) reason=769E0291
```

Each failed host/port name will be attempted once before MQ attempts the next machine name in the CONNAME list.

It is not until all host/port names have been attempted that channel retry processing is attempted.

The DISPLAY CHSTATUS() CONNAME command can be used to verify which connection name was successfully established.

As the use case for multiple host/port names is for failover reasons, the list should be specified in order of likely use.

The performance of “STOP CHANNEL” is not affected by the number of machines in the connection list.

Note:

If using multiple host/port names, ensure the entire string value of CONNAME is enclosed in single quotation marks, for example:

```
CONNAME('10.20.37.22(2202),10.20.37.22(2203),10.20.36.103(2202)')
```

Using CSQUTIL to define MQ channels with long CONNAME values

The IBM documentation for [CSQUTIL](#) provides guidance on how to make object definitions that exceed SYSIN record length limits.

Since the CONNAME may now be up to 264 characters, it might be helpful to highlight specific parts of that guidance:

You specify the functions required by statements in the SYSIN data set according to these rules:

- The data set must have a record length of 80.
- Only columns 1 through 72 are significant. Columns 73 through 80 are ignored.
- Records with an asterisk (*) in column 1 are interpreted as comments and are ignored.
- Blank records are ignored.
- Each statement must start on a new line.
- **A trailing - means continue from column 1 of the next record.**
- **A trailing + means continue from the first non-blank column of the next record.**

For example, defining a sender-type channel with multiple host/port names could be defined thus:

Example: Define sender channel with multiple host/port names

```
DEF CHL(VTS1 VTS2_0001) CHLTYPE(SDR) +
  TRPTYPE(TCP) DISCINT(0) +
  BATCHSZ(50) BATCHINT(1000) +
  NPMSPEED(FAST) +
  XMITQ(VTS2_XMIT_0001) +
  MAXMSGL(2097152) +
  CONNAME(' -
10.20.37.22(2202), -
10.20.37.22(2203), -
10.20.37.22(2204), -
10.20.37.22(2205), -
10.20.37.22(2206), -
10.20.37.22(2207), -
10.20.37.22(2208), -
10.20.37.22(2209), -
10.20.37.22(2210), -
10.20.37.22(2211), -
10.20.36.103(2202)-
') +
  MONCHL(QMGR) +
  COMPMMSG(NONE) +
  REPLACE
```

Conclusions

When using a list of multiple host/port names in the CONNAME attribute, be aware that channel start performance may be impacted by the order that the connection names are specified and whether one or more of the connection attempts fails.

Page set zero usage may decrease when using IBM MQ for z/OS 10.0 compared to prior releases.

Should CONNAMEs be configured with values longer than 48 characters, page set zero usage may increase.

Monitor your page set zero usage when starting your MQ queue manager on IBM MQ for z/OS 10.0 to ensure that there is sufficient capacity for all the MQ objects defined.

Finally, and as a reminder, ensure that for the buffer pool used by page set zero has sufficient buffers available to avoid synchronous page set I/O.

Chapter 5

Log Output Buffer Size (OUTBUFF)

IBM MQ for z/OS 10.0 increases the maximum value of the `OUTBUFF` parameter in `CSQ6LOGP` from 4 000 KB to 40 000 KB — a tenfold increase. This matches the limit that has long been available in Db2 for z/OS. Alongside the increased limit, log output buffers are now permanently page-fixed in above-the-bar storage, providing marginally improved performance and relieving pressure on below-the-bar virtual storage.

`OUTBUFF` controls the total size of the in-storage pool used by IBM MQ to buffer log records before they are written to the active and archive log data sets. Each individual output buffer is 4 KB, so the value is rounded up to a multiple of 4. The valid range is 128 through 40 000 KB.

Configuration

`OUTBUFF` is set in the `CSQ6LOGP` system parameter macro, which is assembled into the `CSQJLOGP` system parameter module and takes effect at queue manager startup. It cannot be changed dynamically using the `SET LOG` command — a queue manager restart is required for a change to take effect.

Suggested settings:

- **Test system:** 400 KB
- **Production system:** 4 000 KB — this is the minimum recommended value for a production system. Consider setting this value higher for log-constrained systems.

To change `OUTBUFF`, edit a copy of the `CSQJLOGP` assembler source module (supplied in `thlqual.SCSQASMS`), update the macro invocation, and reassemble it. For example:

```
CSQ6LOGP OUTBUFF(40000)
```

Link-edit the resulting object module with the existing system parameter module to produce a new load module, then restart the queue manager specifying the new module.

What does `OUTBUFF` affect?

When all available log output buffers are in use — typically because log I/O is not keeping pace with the rate at which log records are being generated — MQ tasks that need to write a log record must wait until a buffer becomes free. This condition is recorded in the log manager statistics as

QJSTWTB (wait for buffers). Any non-zero value of **QJSTWTB** in SMF 115 log manager data records is a sign that **OUTBUFF** is too small, or that there is a problem with log I/O throughput.

A larger **OUTBUFF** provides a deeper cushion between the rate of log record generation and the rate of log I/O completion. This is most beneficial on systems that:

- process high volumes of persistent messages or frequent commits;
- process persistent messages out of syncpoint;
- experience irregular or spiked log I/O, for example due to log data sets placed on high-usage or contended DASD volumes;
- already show non-zero **QJSTWTB** counts in log manager statistics.

In our testing, we saw no measurable benefit or detriment from increasing **OUTBUFF**, however our test system is not log-constrained. On a production system that regularly encounters waits for log buffers, increasing **OUTBUFF** — now up to 40 000 KB — could significantly improve throughput and reduce latency for all persistent and syncpoint-commit workloads.

Monitoring

The log manager statistics record **QJSTWTB** (available in SMF 115 subtype 1 log manager data records) is the primary indicator for log output buffer pressure. Check this value whenever reviewing queue manager performance:

- **QJSTWTB** = 0: no tasks have waited for a log output buffer. No action required.
- **QJSTWTB** > 0: tasks have been suspended waiting for a log output buffer to become free. Consider increasing **OUTBUFF** and investigate log I/O throughput (RMF reports may show heavy I/O to log data set volumes).

Related log manager statistics that provide broader context on log performance include **QJSTLLCP** (checkpoint count) and the **WRTHRSH** parameter, which controls how many output buffers must be filled before they are flushed to the active log data sets. Together, these parameters and statistics provide useful context when assessing log output-buffer pressure and general log performance.

For further configuration details, see [Using CSQ6LOGP](#) in the IBM MQ 10.0 documentation.

Chapter 6

OpenTelemetry support

IBM MQ for z/OS 10.0 introduces support for OpenTelemetry (OTel) tracing, enabling IBM MQ for z/OS to participate in end-to-end distributed traces across applications, middleware, and platforms. OpenTelemetry is an industry-standard observability framework that allows a single transaction to be followed as it traverses multiple components, making it easier to identify performance bottlenecks, failures, and unexpected delays. On z/OS, this capability complements OpenTelemetry support in other key middleware products, including CICS, IMS, and Db2, providing the telemetry required for observability platforms to present a unified view of transaction processing across the platform. IBM MQ for z/OS's OTel support is also compatible with IBM MQ on distributed platforms, meaning that OpenTelemetry traces can flow continuously through queue managers on both z/OS and distributed systems.

An OpenTelemetry trace consists of a collection of spans, where each span represents work performed by an individual component. Spans are linked together using parent-child relationships to form a complete view of a transaction. Every span within a trace shares the same Trace ID, while each span has its own unique Span ID.

When OpenTelemetry tracing is enabled, IBM MQ for z/OS generates spans representing queue manager activity. Producer spans are created when messages are successfully put to queues, and Consumer spans are generated when messages are destructively retrieved. For publish/subscribe messaging, MQ for z/OS can emit a producer span for a publication to an OpenTelemetry-enabled topic, together with producer and consumer spans for each subscriber copy created as part of the fan-out. Additional Client spans can also be generated by components such as the IMS Bridge when messages are forwarded to IMS through OTMA.

IBM MQ takes part in an OpenTelemetry trace when trace context is present in the message, and that trace context can be set manually by the application or automatically by the environment in supported cases. Trace context contains the information required for one component to continue a trace that was started elsewhere. Within IBM MQ for z/OS, trace context is carried in message properties, primarily through the **traceparent** property and the optional **tracestate** property.

This allows observability tools to reconstruct the complete transaction path and determine where time is spent within the application flow. The **traceparent** property uses the W3C Trace Context format, allowing MQ trace data to integrate with industry-standard observability products.

Configuration

Two key configuration areas determine how MQ participates in OpenTelemetry tracing:

- **Span Emission (OTELTRAC)** – Controls whether MQ generates spans for queue and topic activity.
- **Trace Propagation (OTELPCTL)** – Controls how trace context is propagated between applications and middleware, either manually by the application or automatically where supported by the environment.

Span emission

Span emission is controlled by the **OTELTRAC** property. The **OTELTRAC** attribute can be configured on the queue manager, or on local, model or remote queues and on topics. By default all local, model or remote queues are configured with **OTELTRAC(QMGR)** which means that changing the **OTELTRAC** on the queue manager can enable, or disable trace for all queues. Topics are configured by default with **OTELTRAC(ASPARENT)**, which delegates its value to the parent topic's configuration. The **SYSTEM.BASE.TOPIC** uses **OTELTRAC(QMGR)** by default, so if no **OTELTRAC** values have been changed in a topic tree, changing the value on the queue manager enables, or disables trace for all topics.

OTELTRAC on local, model or remote queues can be set to:

- **QMGR**: the default, use the value of **OTELTRAC** from the queue manager.
- **OFF**: do not emit spans, even if messages contain trace context.
- **ON**: emit spans if messages contain trace context.

OTELTRAC on the queue manager can be set to:

- **OFF**: do not emit spans, even if messages contain trace context.
- **ON**: emit spans if messages contain trace context.
- **NONE**: the queue manager never emits spans, even if **OTELTRAC(ON)** is specified on individual queues. Setting **OTELTRAC(ON)** on a queue results in the fast put to waiting getter optimization being disabled.

OTELTRAC on topics can be set to:

- **ASPARENT**: The default for everything other than **SYSTEM.BASE.TOPIC**, delegate to the parent topic's configuration.
- **QMGR**: Use the value of **OTELTRAC** from the queue manager.
- **OFF**: Do not emit spans, even if messages contain trace context.
- **ON**: Emit spans if messages contain trace context.

For topics, **SYSTEM.BASE.TOPIC** uses the default:

- **OTELTRAC(QMGR)**

Trace propagation

Trace propagation is configured using the `OTELPCTL` attribute on the queue manager and local, model and remote queues or on topics.

`OTELPCTL` on local, model and remote queues can be set to:

- **QMGR**: the default, use the value of `OTELPCTL` from the queue manager.
- **MANUAL**: manual propagation is possible.
- **AUTO**: automatic propagation is available if the environment supports it.

`OTELPCTL` on the queue manager can be set to:

- **MANUAL**: manual propagation is possible.
- **AUTO**: automatic propagation is available if the environment supports it.

`OTELPCTL` on topics can be set to:

- **ASPARENT**: The default for everything other than `SYSTEM.BASE.TOPIC`, delegate to the parent topic's configuration.
- **QMGR**: Use the value of `OTELPCTL` from the queue manager.
- **MANUAL**: Manual propagation is possible.
- **AUTO**: Automatic propagation is available if the environment supports it.

For topics, `SYSTEM.BASE.TOPIC` uses the default:

- `OTELPCTL(QMGR)`

Automatic propagation is supported in the following environments:

- IBM CICS Transaction Server for z/OS 6.3 or later (see CICS and OpenTelemetry in the IBM CICS Transaction Server for z/OS documentation).
- IBM IMS 15.5 or later, with enablement PTFs (see OpenTelemetry tracing in IMS in the IBM Information Management System documentation) for the IBM MQ IMS bridge.
- IBM IMS 15.5 or later, with PTFs for APAR PH69252 and PH70962 applied for the IBM MQ IMS adapter.

Where are spans emitted?

IBM MQ emits OpenTelemetry span data as SMF 1158 records. The z/OS OpenTelemetry Emitter can be configured to collect these records and forward them to observability platforms.

An SMF in-memory resource must be used for these records, allowing trace data to be cyclically overwritten and preventing excessive DASD consumption when tracing high transaction volumes. This process is described in the MQ documentation as follows:

1. Determine the name of the in-memory resource.

For example: `IFASMF.MQOTEL`.

2. Ask a security administrator to define a new SAF resource to the **FACILITY** class to protect the IBM MQ resource.

The resource is the name of the in-memory resource prefixed with **IFA**. For example: `IFA.IFASMF.MQOTEL`.

3. Give each IBM MQ MSTR started task ID **READ** access to the SAF resource.
4. Update the `SMFPRMxx` parmlib member to specify the **INMEM** parameter, resource name, and values for the **RESSIZMAX** and **TYPE** sub-parameters for each in-memory resource that you want to define.

For example:

```
SETSMF INMEM(IFASMF.MQOTEL,RESSIZMAX(128M),TYPE(1158))
```

5. Issue the `SET SMF=xx` command to activate the updated `SMFPRMxx` member.
6. Set up and configure the z/OS OpenTelemetry Emitter as described in the z/OS documentation.

What does OTel cost?

The impact to cost per transaction and transaction rate by using OTel with MQ for z/OS is dependent on both the method by which OTel is propagated and the type of workload.

The following configurations are used and referred to throughout this section.

Baseline

```
OTELTRAC(ON) OTELPCTL(MANUAL)
```

This is the baseline case. The queue manager is configured such that tracing could occur if valid trace context were present, but no trace properties are supplied to MQ. As a result, there is no trace-context propagation cost and no spans are emitted for these messages.

Manual propagation

```
OTELTRAC(ON) OTELPCTL(MANUAL)
```

Trace properties are passed from the application to MQ. This uses the same queue manager tracing configuration as the baseline, but the application supplies trace context to MQ using message properties. Because trace context is present, MQ can participate in the trace and emit spans.

Automatic propagation, trace off

`OTELTRAC(OFF) OTELPCTL(AUTO)`

`OTELTRAC` is disabled, so no MQ spans are emitted, but OTel's trace context is still propagated automatically by the environment. This mode isolates the cost of automatic propagation without the additional cost of MQ span emission. It shows the cost of carrying trace context through MQ automatically when MQ itself is not emitting spans.

Automatic propagation, trace on

`OTELTRAC(ON) OTELPCTL(AUTO)`

`OTELTRAC` is enabled and trace context is propagated automatically by the environment. This mode represents full automatic propagation with MQ span emission enabled. The environment propagates trace context, and MQ emits spans when valid trace context is present.

How Do the Different Propagation Styles Compare?

With a small message size and therefore high transaction rate the impacts of OTel and the type of propagation are most apparent. The following test results show the impact of enabling OTel in different configurations with a 2KB message size for persistent and non-persistent messages.

Both workloads use a single pair of request/reply queues running at maximum throughput. Requester tasks each put a message out-of-syncpoint to a common request queue and wait for a specific response on the reply queue; once the reply arrives, the next message is put immediately. Server tasks action `MQGET-with-wait` calls on the request queue, get the message and put a reply in-syncpoint to the known, pre-opened reply queue, then return to wait. Every transaction produces 4 spans, one producer span and one consumer span on each of the request and reply legs of the transaction. A new trace context is propagated for every transaction.

The workloads correspond to those described in the private queue server in-syncpoint persistent and non-persistent sections of this report. A server in-syncpoint setup is used throughout these tests because messages that are eligible to emit OTel spans cannot take advantage of the fast put to waiting getter optimisation. Using a server in-syncpoint workload means the optimisation is already disabled in the baseline, providing a fair before and after comparison.

Chart: Transaction rate for 2KB persistent and non-persistent workload with OTel

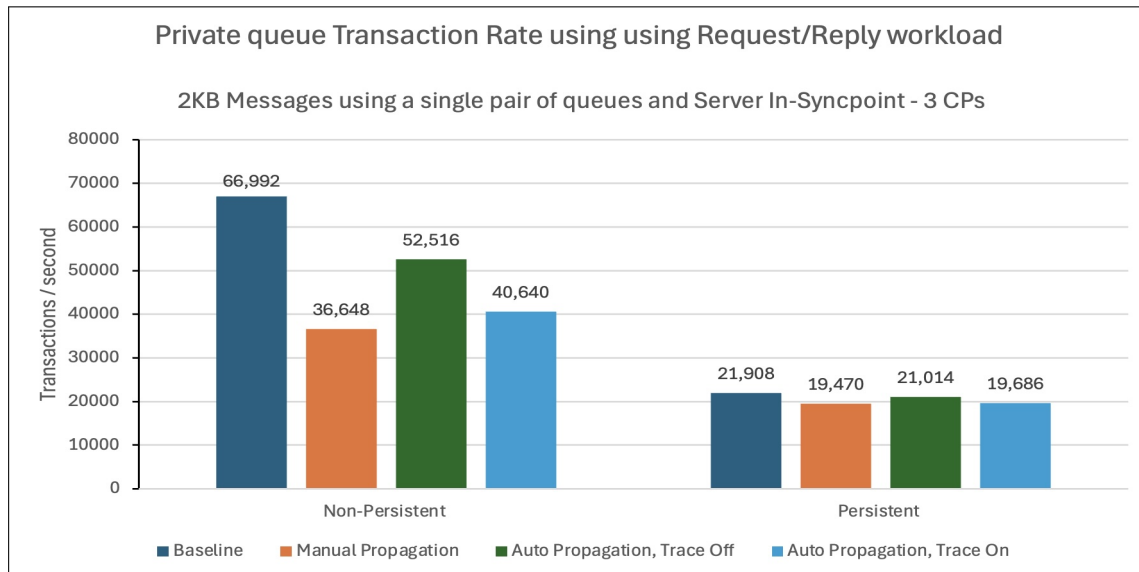
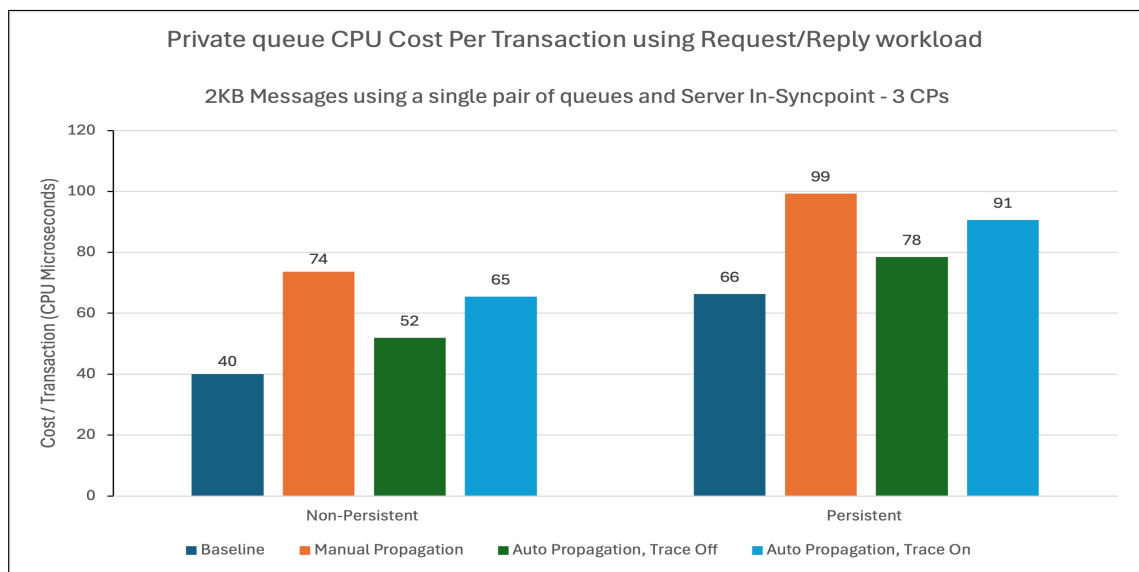


Chart: CPU Cost per Transaction for 2KB persistent and non-persistent workload with OTel



In these scenarios, the impact of OTel is significant. Automatic propagation outperforms manual propagation, and the data shows that the largest impact on transaction rate and CPU cost per transaction for the automatic propagation scenarios comes from the propagation itself rather than span emission.

Another notable observation is the smaller difference seen in the persistent workloads. Persistent messaging has a higher CPU cost per transaction, while the CPU time required for trace propagation and span emission remains relatively constant on a per-transaction basis. As a result, the percentage difference in both transaction rate and cost per transaction is considerably smaller than in the non-persistent workload.

From the above data we can estimate the following additional costs in MQ while using OTel:

- Emission CPU cost for 4 spans is around 13 μ s, or 3.25 μ s per span emission.

For the trace propagation itself, excluding the emission of spans:

- Automatic trace propagation takes around 12 μ s of CPU transaction or 3 μ s of CPU per span.
- Manual trace propagation takes around 20 μ s of CPU time per transaction or 5 μ s of CPU per span..

If we then compare the 2KB results above to a much larger message size of 4MB the comparative impact decreases as message size increases.

Chart: Transaction rate for 4MB persistent and non-persistent workload with OTel

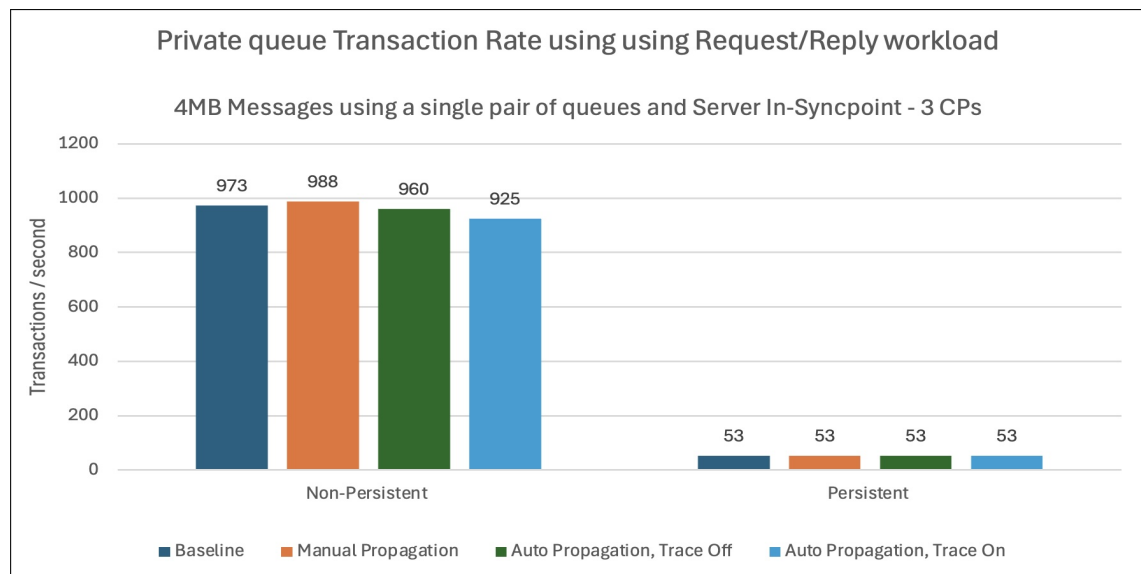
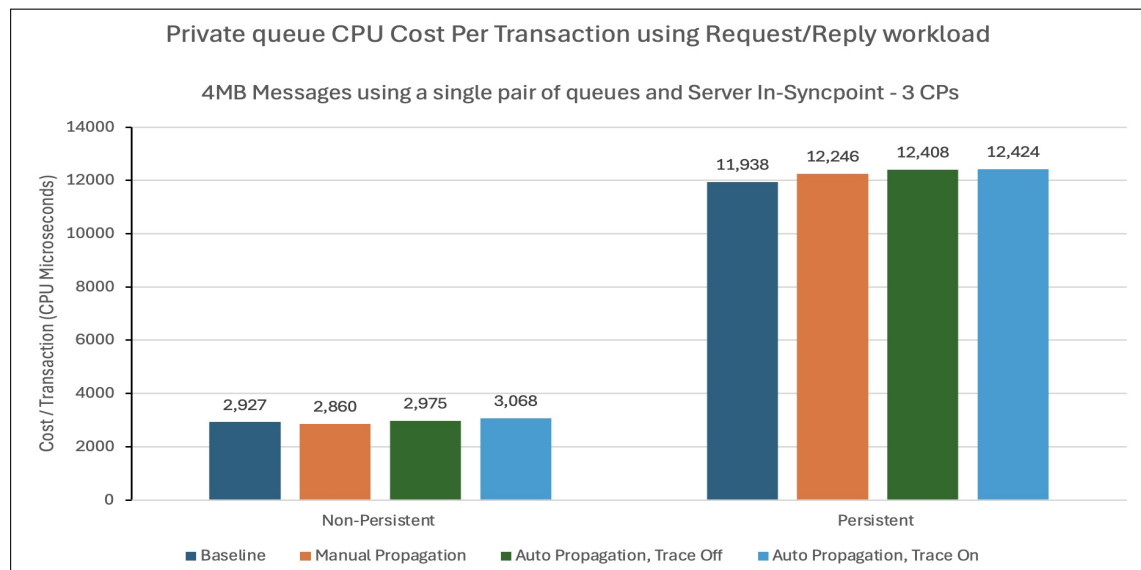


Chart: CPU Cost per Transaction for 4MB persistent and non-persistent workload with OTel



For the baseline tests, increasing the message size increases the CPU cost per transaction and reduces the achievable transaction rate. The CPU μ s cost associated with trace propagation and span emission remains relatively small, resulting in the largest messages seeing a negligible overall impact to transaction rate and CPU usage per transaction.

Scalability

The same effect seen in the baseline tests is also observed at scale. Using the same scalability workload described in the private queue non-persistent server in-syncpoint scalability workload in Appendix A of this report, the queue manager is driven across an increasing number of concurrent requester and server task pairs, up to fifteen in total, running across 16 dedicated processors on an IBM z17 (9175). As the last scenario in the OTel configuration, each transaction generates 4 spans. Results are captured for 2KB, 64KB and 4MB messages.

As with the baseline results, smaller messages at higher throughput see a higher relative impact from OTel enablement, while larger messages at lower throughput see a negligible impact on both transaction rate and CPU cost per transaction. The cost per transaction in CPU μ s remains consistent with the private queue tests.

At throughput rates approaching 100,000 transactions per second this results in approximately 400,000 spans being generated every second. At this point span generation begins to outpace the ability of SMF collection to transfer records to the in-memory storage object used for OpenTelemetry data. As a result, MQ's buffer fills and MQ can no longer keep storing spans to emit. The number of spans lost is small near the threshold where this begins to occur, but increases with throughput beyond that point. This can be seen as a slight reduction in measured CPU cost per transaction at the higher end of the scalability range, as less span work is being performed. You can see the number of spans not emitted due to the inability for SMF to keep up in the *OTEL spans not generated, no storage* field in MP1B output from SMF 115 Queue statistics.

This behaviour is expected. MQ's OpenTelemetry implementation is designed so that an inability to emit spans does not prevent IBM MQ from continuing to process messages. This ensures that observability data does not become a bottleneck for message processing and that the primary function of the queue manager takes priority over trace emission.

Conclusions

OpenTelemetry tracing is a useful tool for understanding message flows and diagnosing latency across distributed systems. For many workloads the cost of enabling it is low. Where MQ is not the bottleneck – for example, with larger messages or lower transaction rates – the overhead is negligible, and OTel is well suited to production use.

For high-throughput, small-message workloads the additional CPU cost is more noticeable. In these cases, scoping tracing to specific queues or topics using **OTELTRAC** can help keep the overhead manageable. Alternatively, sampling a small proportion of messages allows observability data to be collected gradually over time, building a useful picture of application behaviour without tracing every transaction.

When evaluating OTel for production use, it is worth noting that messages eligible to emit OTel spans cannot take advantage of the fast put to waiting getter optimisation. For workloads that rely on this optimisation, this is a factor to consider when deciding whether to enable OTel on a given queue.

OTel is particularly well suited to development and test environments. Full tracing is practical at lower throughputs, and being able to trace transactions across MQ, CICS, IMS, and Db2 alongside MQ on distributed systems makes it much easier to understand application behaviour and diagnose problems.

Appendix A

Regression

When performance monitoring both IBM MQ Advanced for z/OS 10.0 and IBM MQ for z/OS 10.0, a number of different categories of tests are run, which include:

- Private queue
- Shared queue
- Moving messages using MCA channels
 - SSL
 - Channel compression (ZLIBFAST / ZLIBHIGH)
 - Streaming messages
- Moving messages using cluster channels
- Client
- Bridges and adaptors
- Trace
- AMS message protection

These tests are run against version V9.3 (V930) and 9.4 (V940) and 10.0 (V1000) and the comparison of the results is shown in subsequent pages.

The statement of regression is based upon these results.

All measurements were run on a performance sysplex of an IBM z17 (9175) which was configured as described in [“System Configuration”](#).

Given the complexity of the z/OS environment even in our controlled performance environment, a tolerance of +/-6% is regarded as *acceptable* variation between runs.

Private Queue

Non-persistent out-of-syncpoint workload

Maximum throughput on a single pair of request/reply queues

The test uses 5 batch requester tasks that each put a message to a common request queue and wait for a specific response on the reply queue. Once they have gotten the message, they put another message to the request queue. The messages are put and got out of syncpoint.

There are 4 batch server tasks that action MQGET-with-wait calls on the request queue, get the message and put a reply to the known (and pre-opened) reply queue and then the application goes back into the MQGET-with-wait call. The messages are put and got out of syncpoint.

Chart: Transaction rate for non-persistent out-of-syncpoint workload

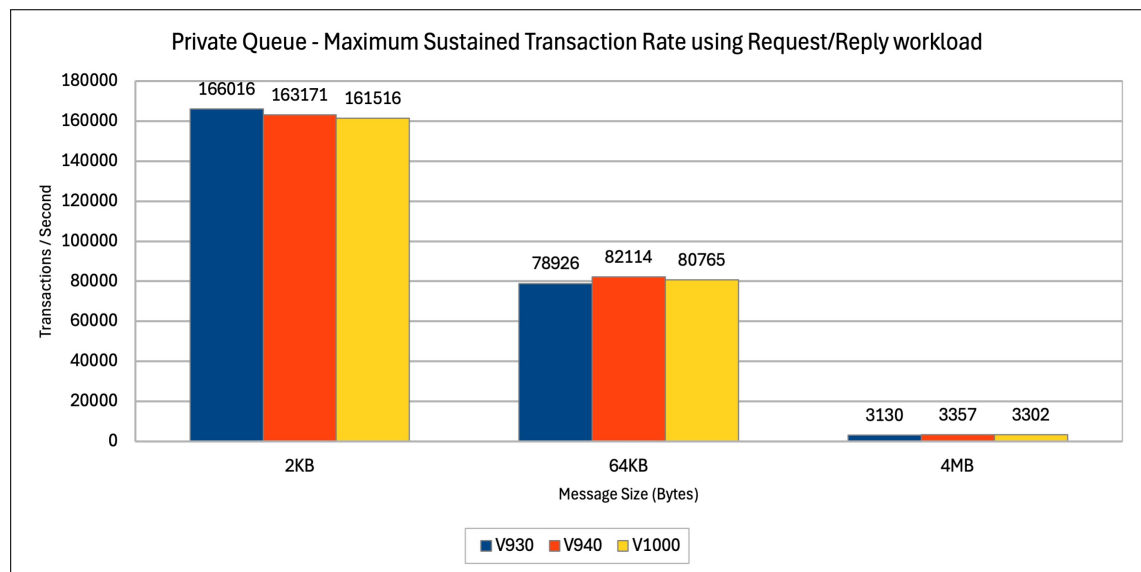
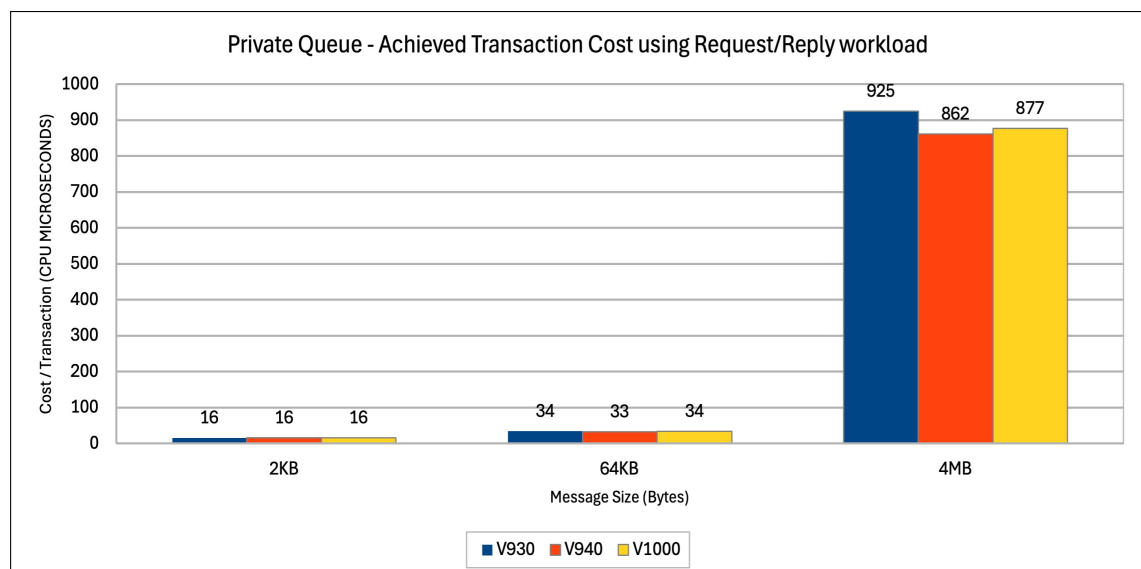


Chart: Transaction cost for non-persistent out-of-syncpoint workload



Scalability of request/reply model across multiple queues

The queue manager is configured with pagesets 0 through 15 and with 1 buffer pool per pageset.

On each of pagesets 1 to 15, a pair of request and reply queues are defined. The test starts up 1 requester and 1 server task accessing the queues on pageset 1 and runs a request/reply workload. At the end, the test starts a second pair of requester and server tasks which access the queues on pageset 2 and so on until there are 15 requester and 15 server tasks using queues on all 15 pagesets with the application queues defined.

The requester and server tasks specify NO_SYNCPOINT for all messages.

The measurements are run on a single LPAR with 16 dedicated processors online on the z17 (9175) used for testing.

Chart: Transaction rate for non-persistent out-of-syncpoint scalability workload

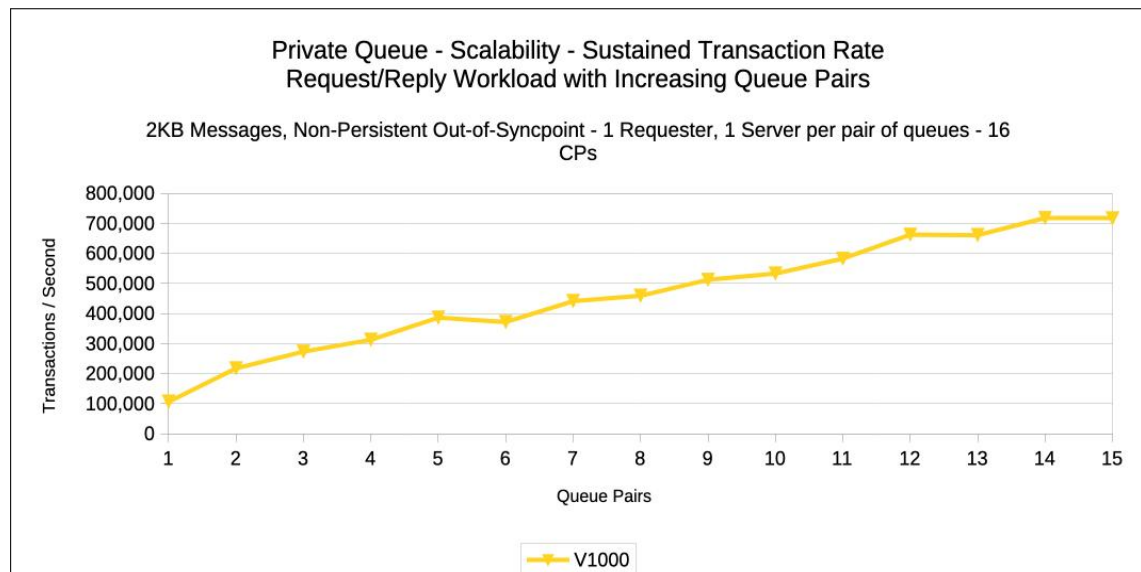
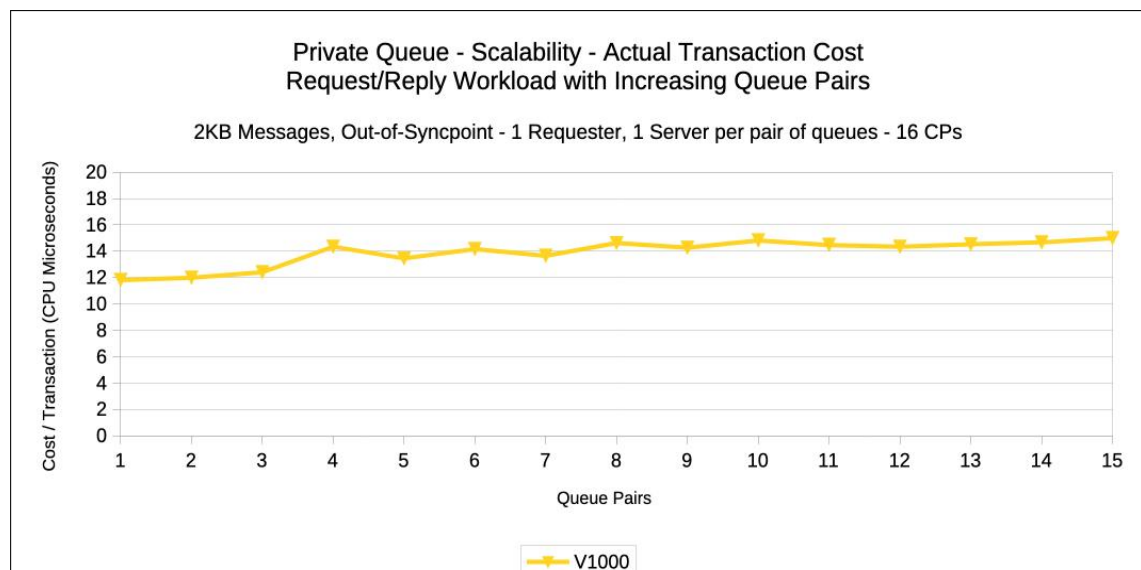


Chart: Transaction cost for non-persistent out-of-syncpoint scalability workload



The preceding 2 charts show that a single queue manager is able to drive in excess of 715,000

transactions per second - or 1,430,000 non-persistent messages per second on a 16-way LPAR.

Note: Since the performance for 9.3, 9.4 and 10.0 is comparable, the charts show only 10.0 data for the purpose of clarity.

Non-persistent server in-syncpoint workload

Maximum throughput on a single pair of request/reply queues

The test uses 5 batch requester tasks that each put a message to a common request queue and wait for a specific response on the reply queue. Once they have gotten the message, they put another message to the request queue. The messages are put and got out of syncpoint.

There are 4 batch server tasks that action MQGET-with-wait calls on the request queue, get the message and put a reply to the known (and pre-opened) reply queue and then the application goes back into the MQGET-with-wait call. The messages are got and put in syncpoint with 1 MQGET and 1 MQPUT per commit.

Chart: Transaction rate for non-persistent in syncpoint workload

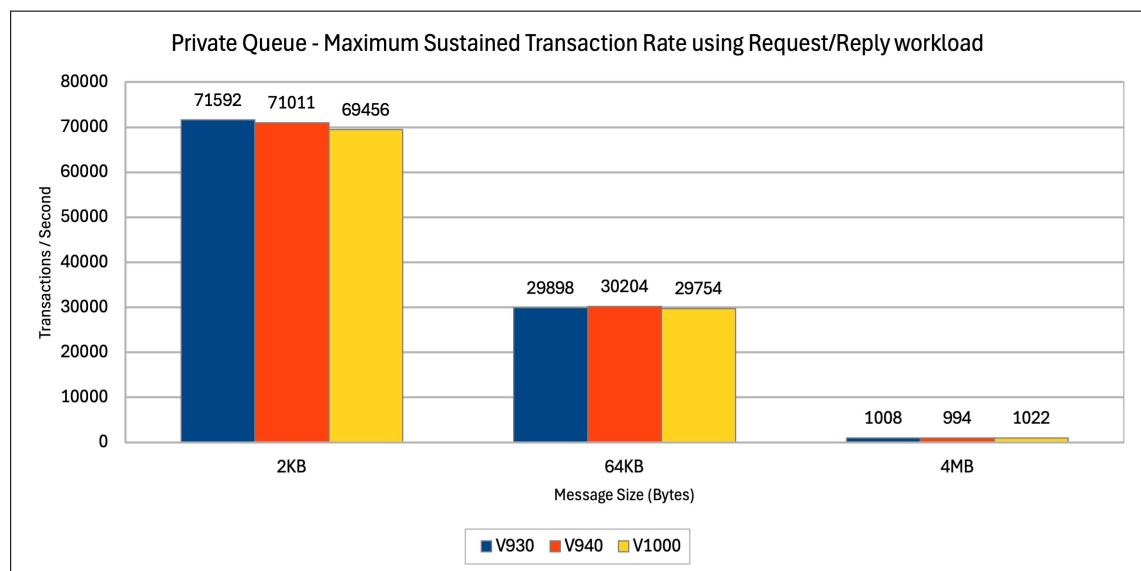
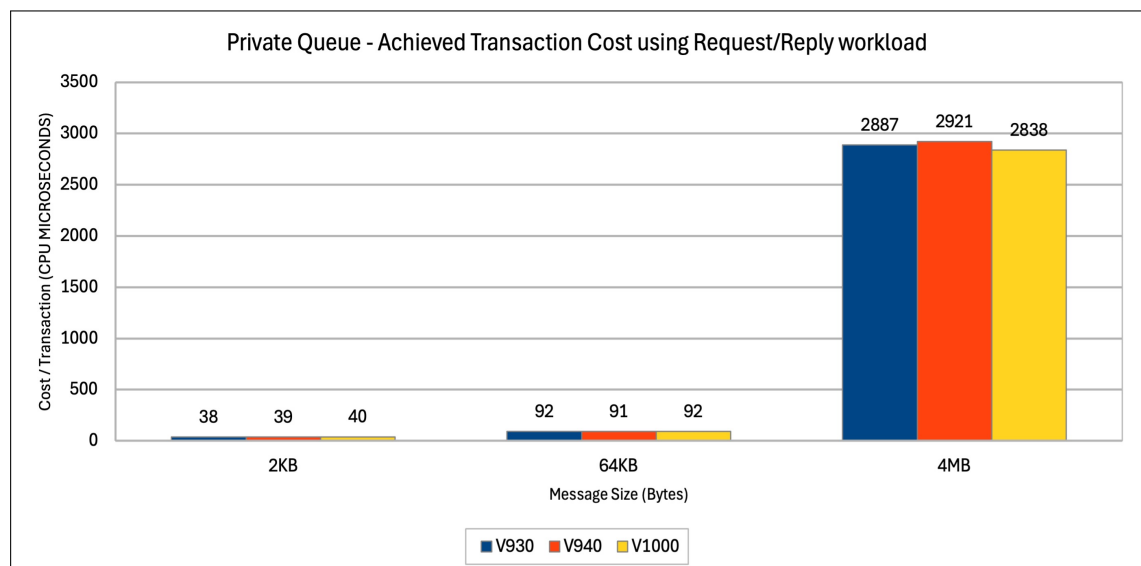


Chart: Transaction cost for non-persistent in syncpoint workload



Scalability of request/reply model across multiple queues

The queue manager is configured with pagesets 0 through 15 and with 1 buffer pool per pageset.

On each of pagesets 1 to 15, a pair of request and reply queues are defined. The test starts up 1 requester and 1 server task accessing the queues on pageset 1 and runs a request/reply workload. At the end, the test starts a second pair of requester and server tasks which access the queues on pageset 2 and so on until there are 15 requester and 15 server tasks using queues on all 15 pagesets with the application queues defined.

The requester tasks specify NO_SYNCPOINT for all messages and the server tasks get and put messages within syncpoint.

The measurements are run on a single LPAR with 16 dedicated processors online on the z17 (9175) used for testing.

The measurements are run using 2KB, 64KB and 4MB messages. The 4MB message measurement uses a maximum of 6 sets of queues and tasks.

Note: Since the performance for 9.3, 9.4 and 10.0 s comparable, the charts show only 10.0 data for the purpose of clarity.

Chart: Transaction rate for non-persistent in syncpoint scalability workload with 2KB messages

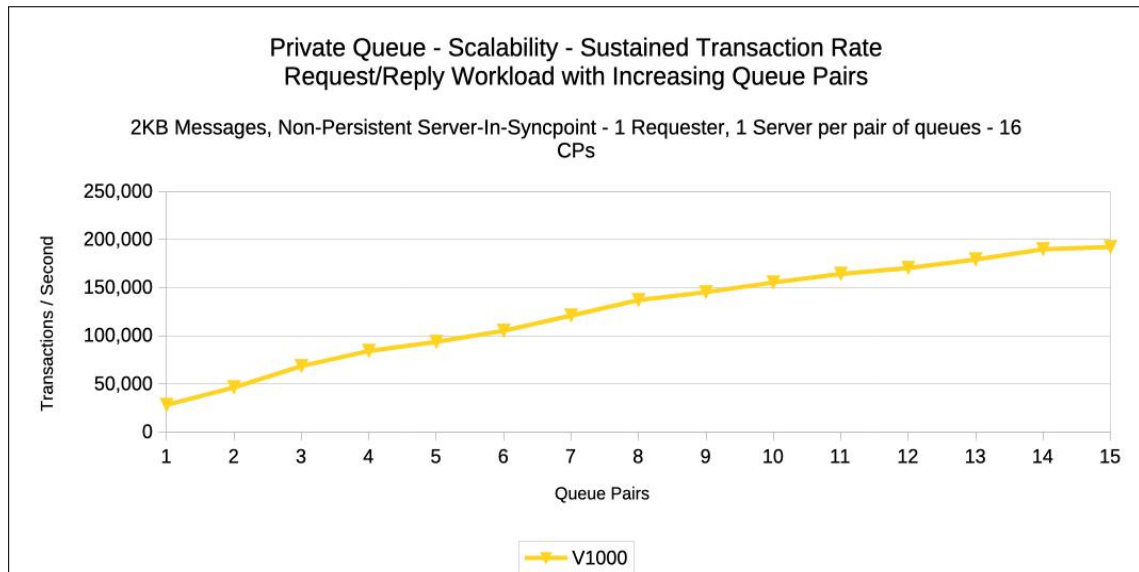


Chart: Transaction cost for non-persistent in syncpoint scalability workload with 2KB messages

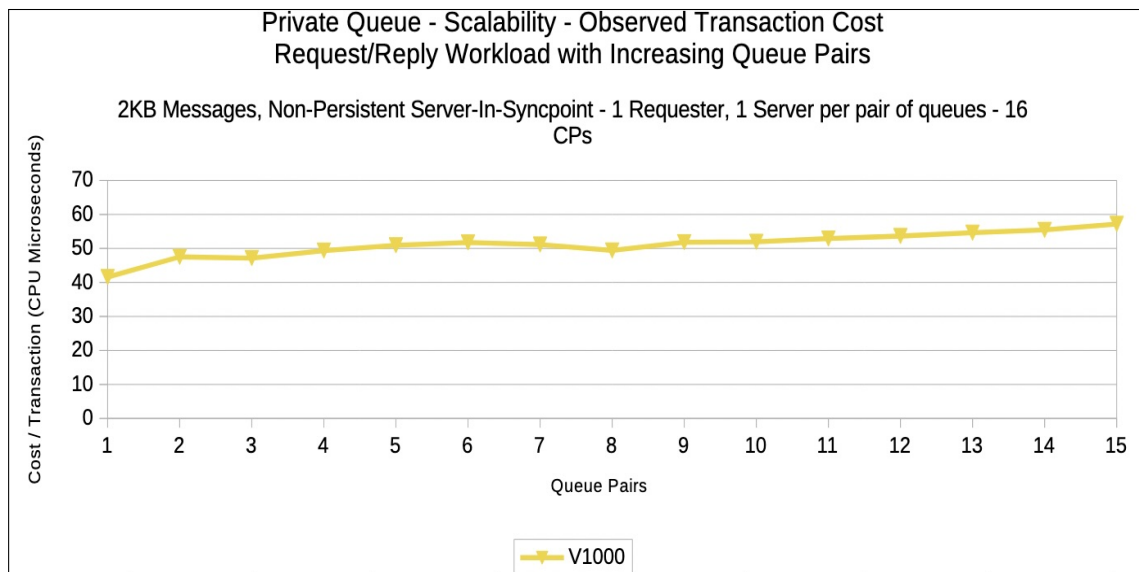


Chart: Transaction rate for non-persistent in syncpoint scalability workload with 64KB messages

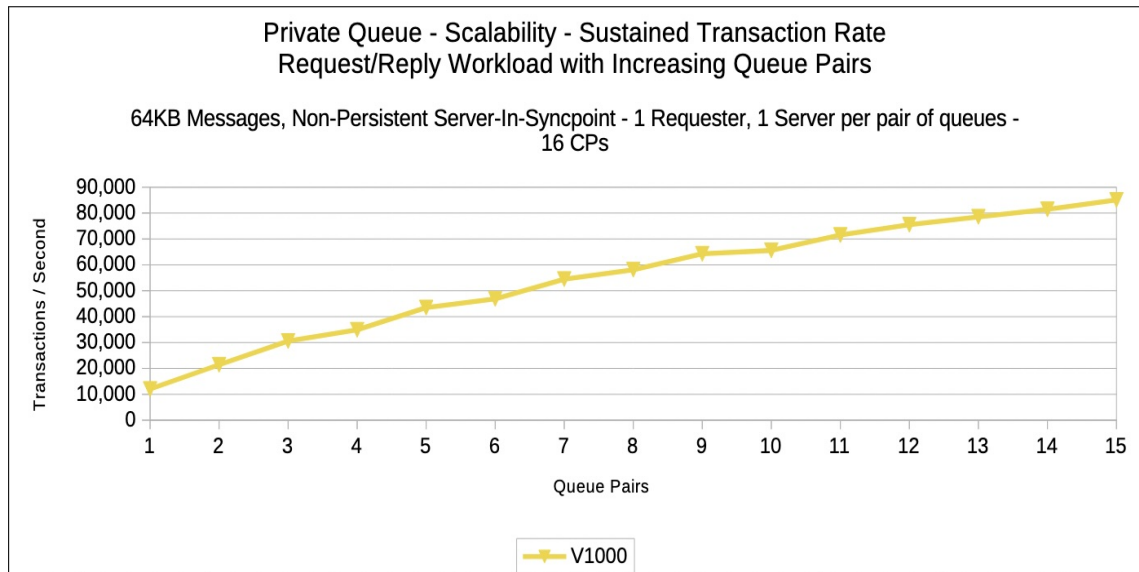


Chart: Transaction cost for non-persistent in syncpoint scalability workload with 64KB messages

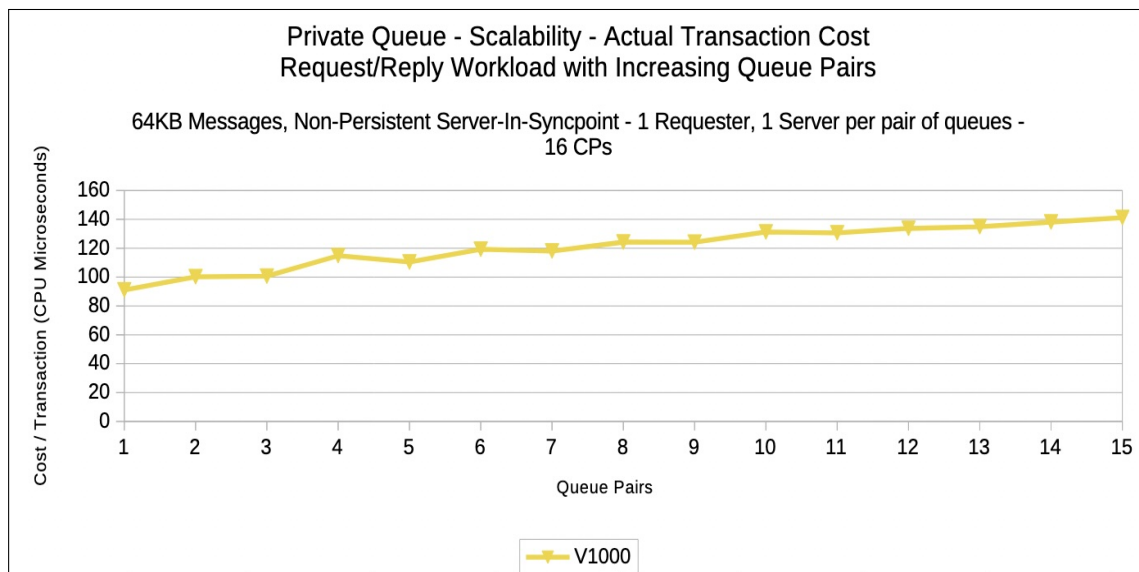


Chart: Transaction rate for non-persistent in syncpoint scalability workload with 4MB messages

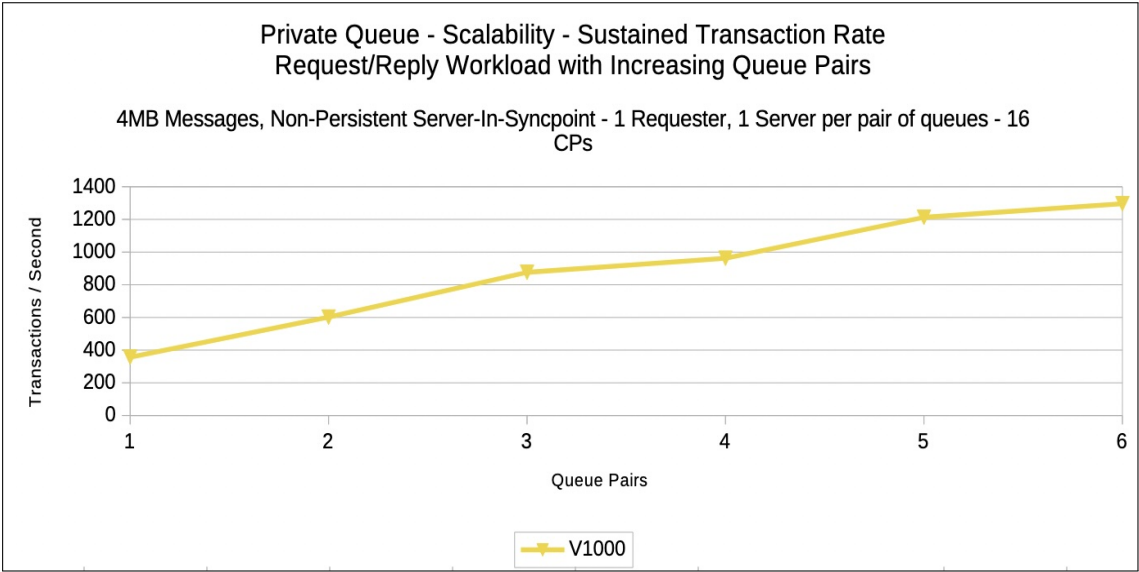
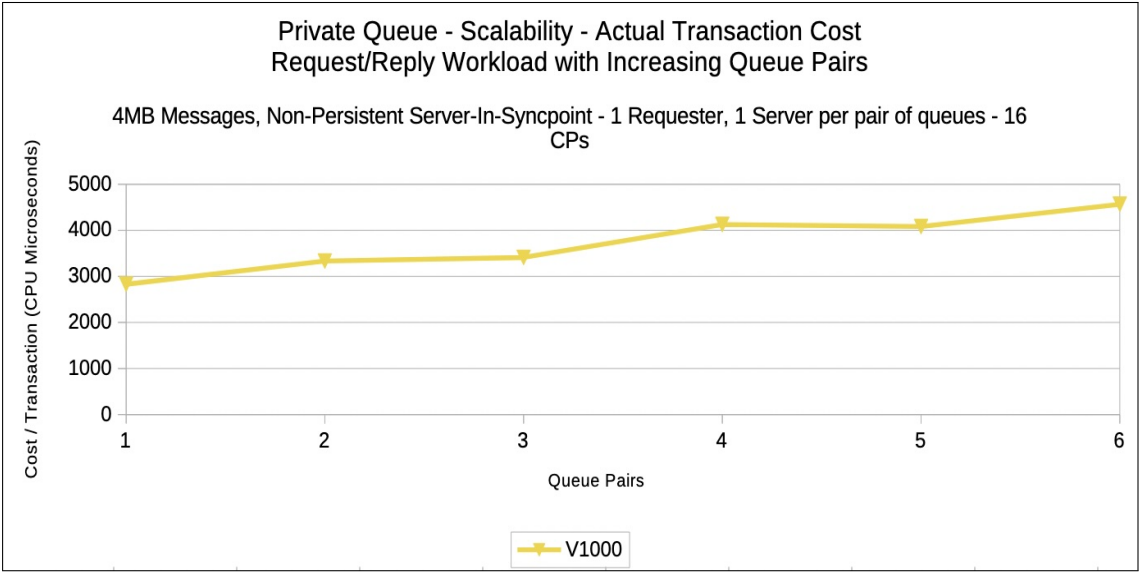


Chart: Transaction cost for non-persistent in syncpoint scalability workload with 4MB messages



Persistent server in-syncpoint workload

Maximum throughput on a single pair of request/reply queues

The test uses 60 batch requester tasks that each put a message to a common request queue and wait for a specific response on the reply queue. Once they have gotten the message, they put another message to the request queue. The messages are put in-syncpoint and got in-syncpoint.

There are 10 batch server tasks that action MQGET-with-wait calls on the request queue, get the message and put a reply to the known (and pre-opened) reply queue and then the application goes back into the MQGET-with-wait call. The messages are got and put in syncpoint with 1 MQGET and 1 MQPUT per commit.

Chart: Transaction rate for persistent in syncpoint workload

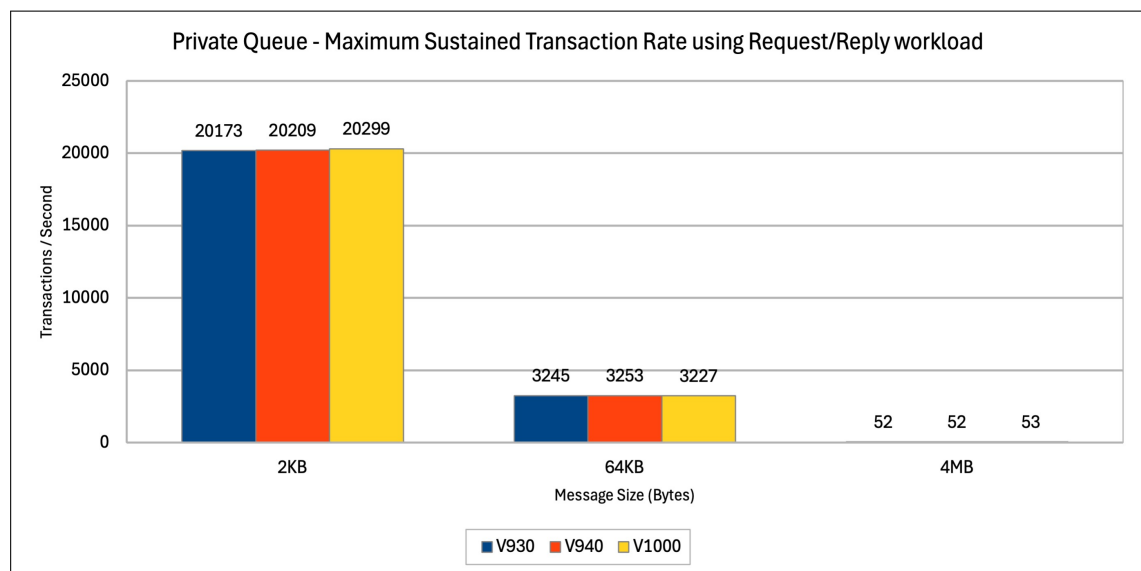
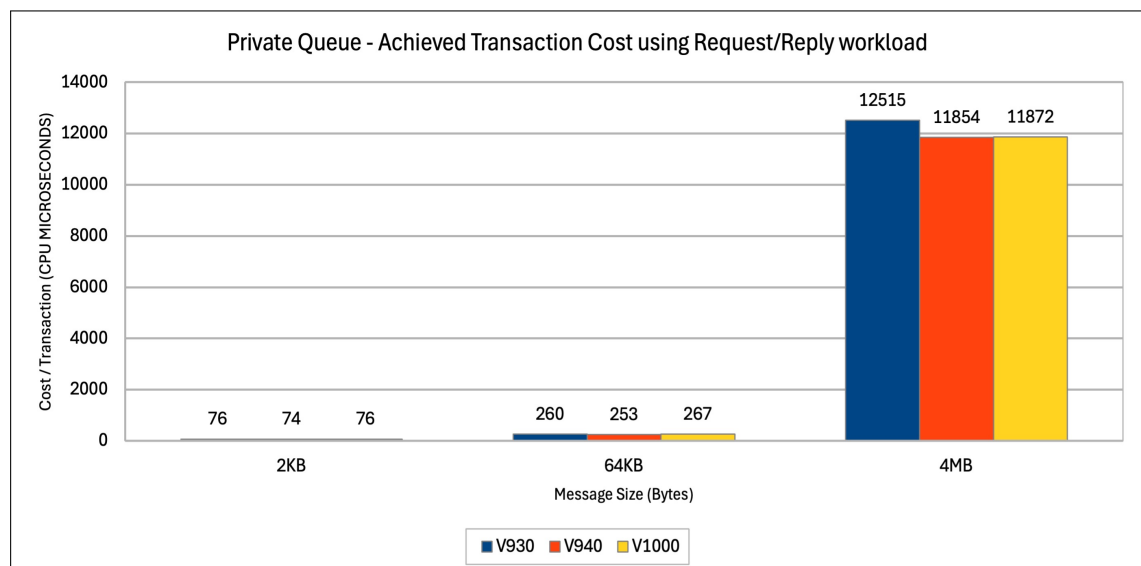


Chart: Transaction cost for persistent in syncpoint workload



Upper bounds of persistent logging rate

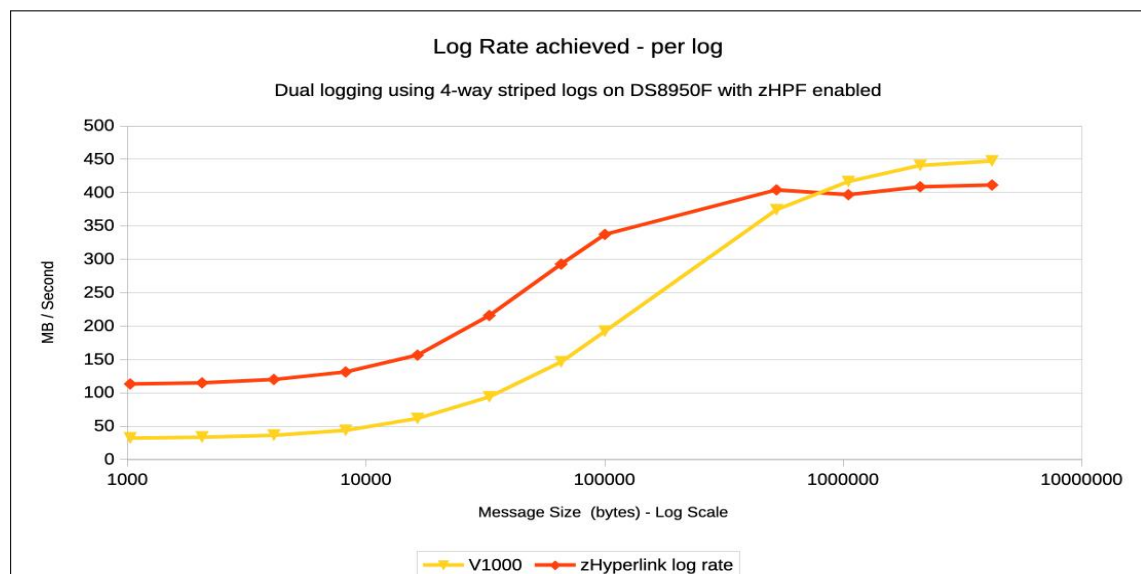
This test uses 3 batch tasks that each put and get messages from a common queue. One of the tasks only uses a 1KB persistent message. The remaining 2 tasks vary the size of their message from 1KB to 4MB.

The following chart shows the achieved log rate on a 3-way LPAR of the z17 (9175).

Since the performance for 9.3, 9.4 and 10.0 is comparable, the chart shows only 10.0 data for the purpose of clarity.

Also included in the data is the performance of the MQ logger when zHyperLink is enabled. Support for zHyperLink on MQ's active log data sets was introduced in [IBM MQ for z/OS 9.4](#).

Chart: Peak log rate achieved during persistent message workload



Notes:

The peak log rate in excess of 450MB per second is for each copy of the dual logs, i.e. there is more than 900 MB of data being written to IBM MQ log data sets per second.

These logs are configured with 4 stripes.

CICS Workload

When a CICS transaction makes a call using an MQ API, the resulting processing performed at end of task can have a significant effect on the transaction cost. This cost is most noticeable when adding the first MQ call e.g. MQPUT1 to the application.

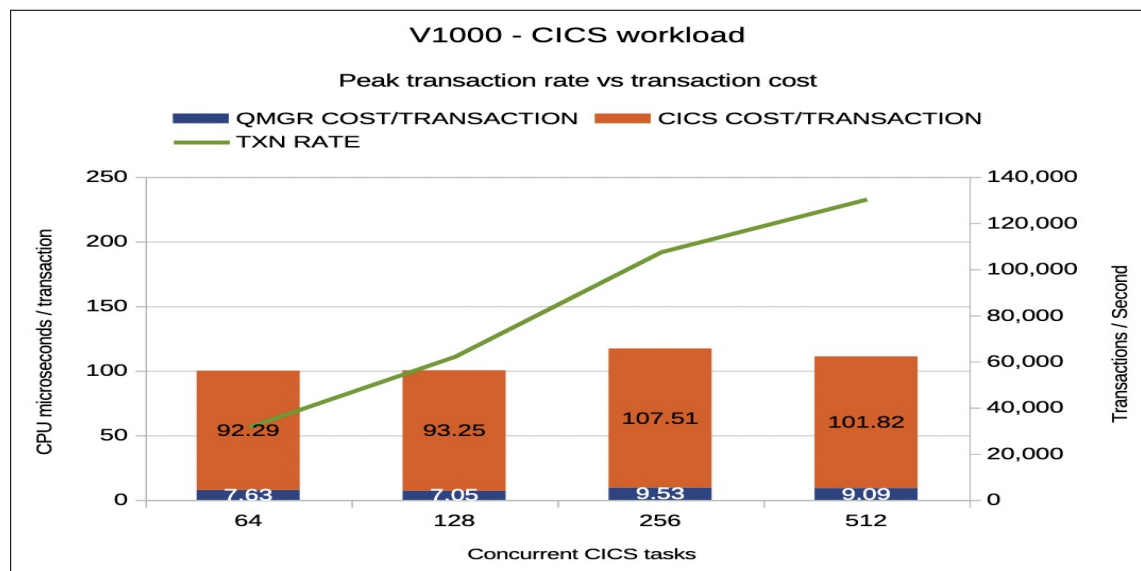
The following chart uses a set of simple CICS transactions that open queues, put a 2KB non-persistent message, get a 2KB non-persistent message, close queues and initiate a new CICS transaction. Running multiple sets of these transactions simulates a number of concurrent transactions across multiple CICS regions.

In these measurements, the system is running with 16 dedicated processors and becomes CPU constrained with 512 concurrent CICS tasks.

Notes on chart:

- As the 9.3, 9.4 and 10.0 data is similar, only 10.0 data is shown.

Chart: CICS workload with MQ 10.0



Shared Queue

Version 7.1.0 introduced CFLEVEL(5) with Shared Message Data Sets (SMDS) as an alternative to Db2 as a way to store large shared messages. This resulted in significant performance benefits with both larger messages (greater than 63KB) as well as smaller messages when the Coupling Facility approached its capacity by using tiered thresholds for offloading data. For more details on the performance of CFLEVEL(5), please refer to performance report [MP16](#) “Capacity Planning and Tuning Guide”.

CFLEVEL(5) with SMDS with the default offload thresholds has been used for the shared queue measurements.

Non-persistent out-of-syncpoint workload

Maximum throughput on a single pair of request/reply queues

The test uses 5 batch requester tasks that each put a message to a common request queue and wait for a specific response on the reply queue. Once they have gotten the message, they put another message to the request queue. The messages are put and got out of syncpoint.

There are 4 batch server tasks that action MQGET-with-wait calls on the request queue, get the message and put a reply to the known (and pre-opened) reply queue and then the application goes back into the MQGET-with-wait call. The messages are put and got out of syncpoint.

An increasing number of queue managers are allocated to process the workload. Each queue manager has 5 requester tasks and 4 server tasks.

Chart: Transaction rate for non-persistent out-of-syncpoint workload - 2KB messages.

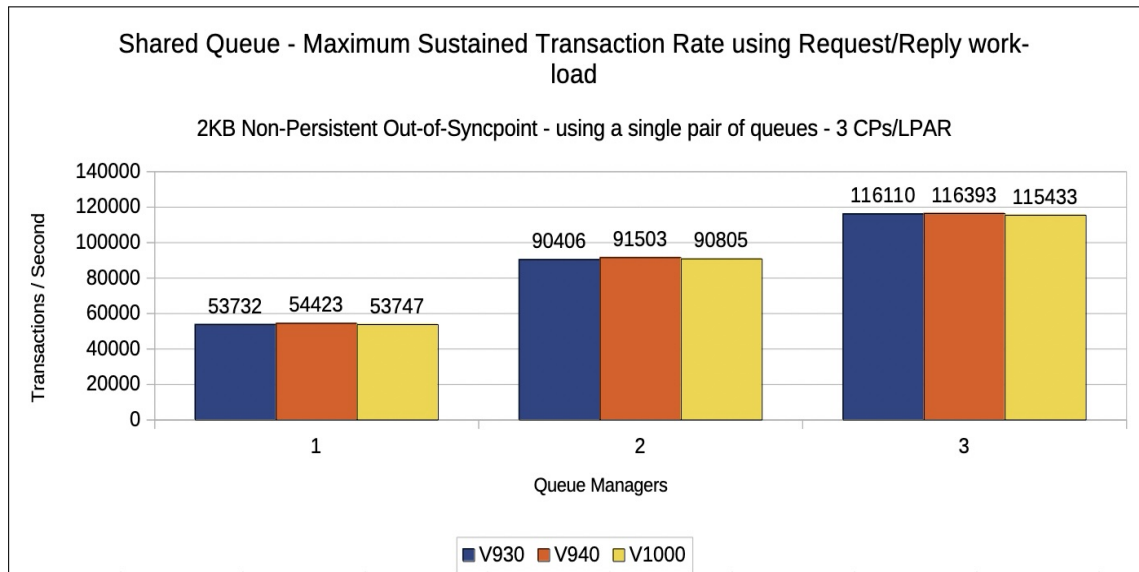


Chart: Transaction cost for non-persistent out-of-syncpoint workload - 2KB messages.

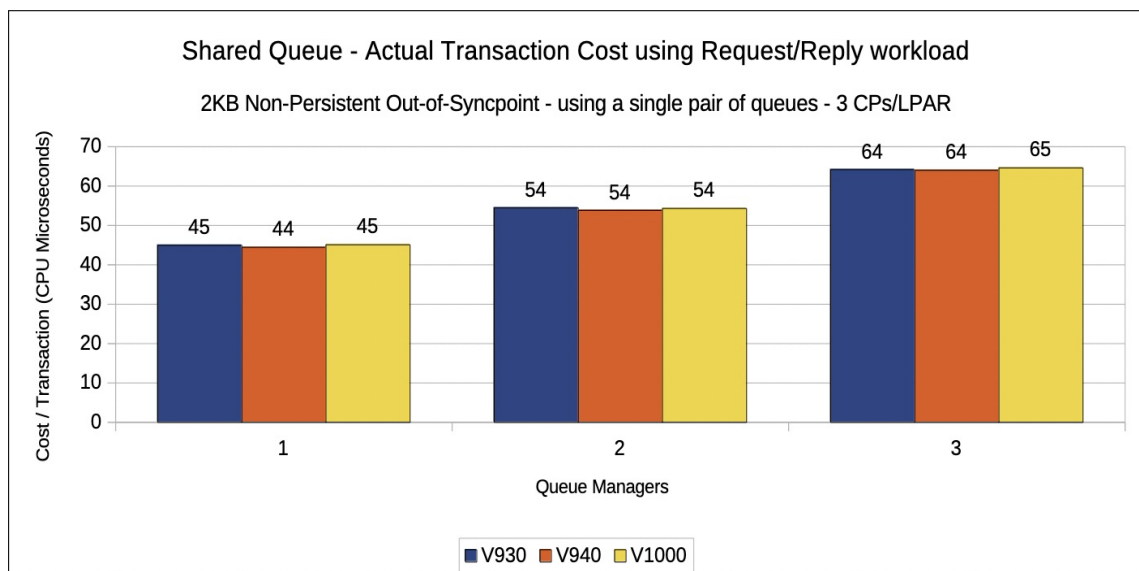


Chart: Transaction rate for non-persistent out-of-syncpoint workload - 64KB messages.

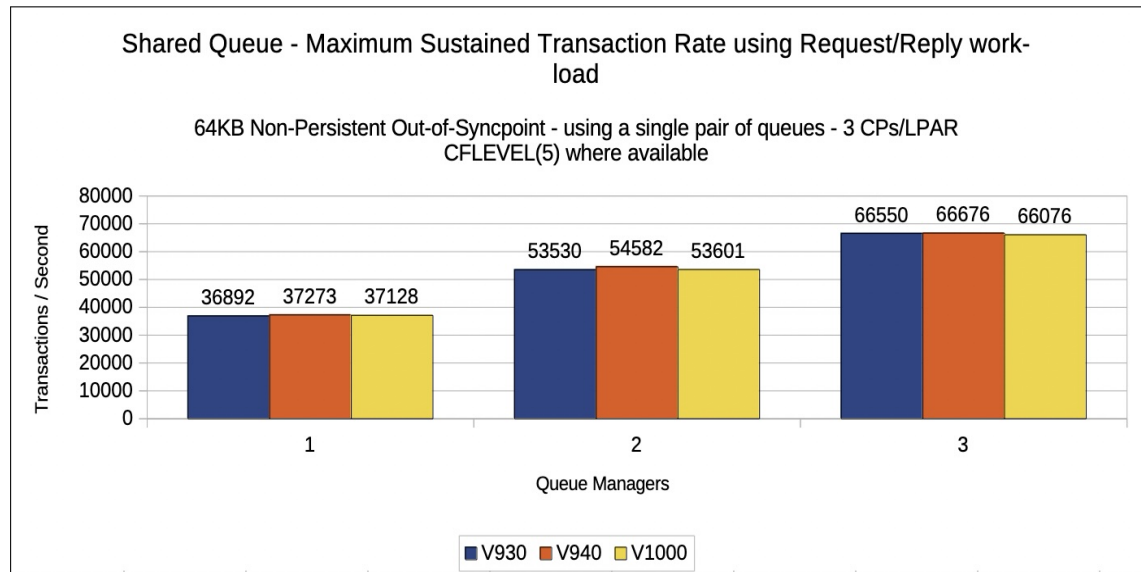


Chart: Transaction cost for non-persistent out-of-syncpoint workload - 64KB messages.

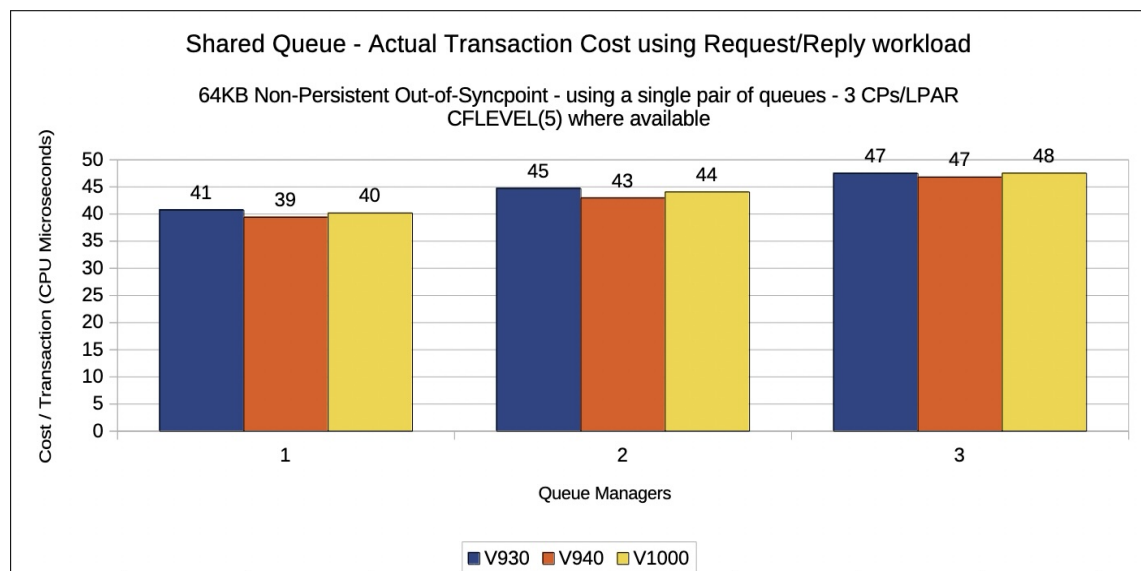


Chart: Transaction rate for non-persistent out-of-syncpoint workload - 4MB messages.

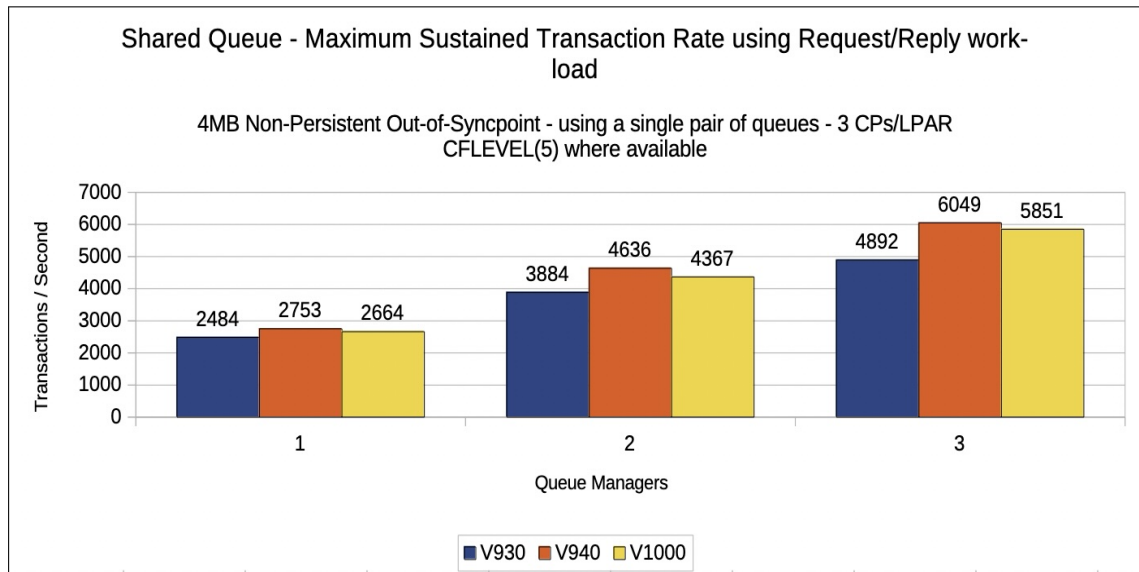
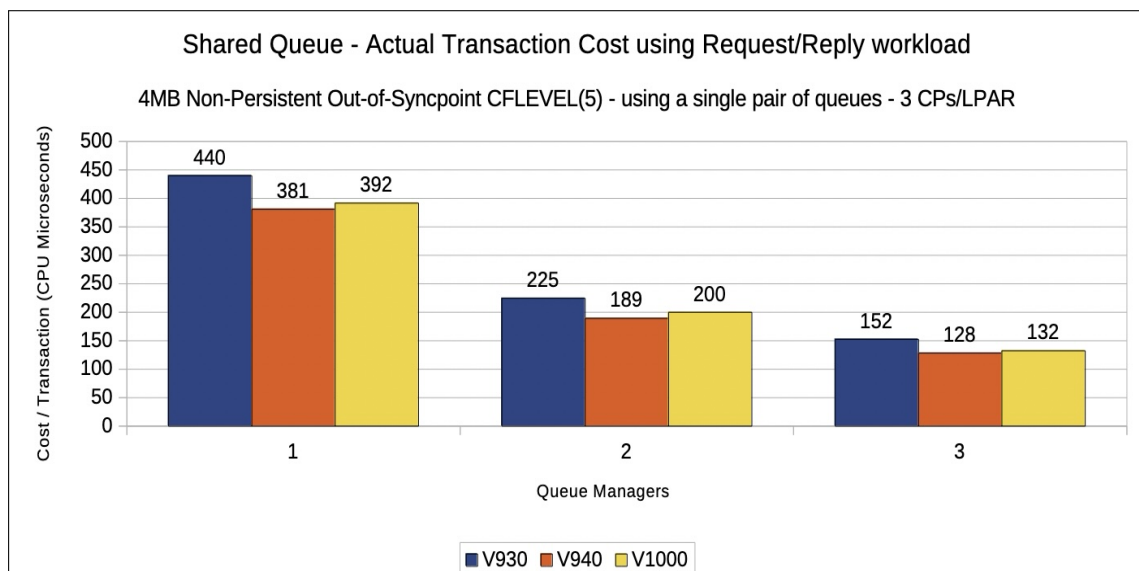


Chart: Transaction cost for non-persistent out-of-syncpoint workload - 4MB messages.



Non-persistent server in-syncpoint workload

Maximum throughput on a single pair of request/reply queues

The test uses 5 batch requester tasks that each put a message to a common request queue and wait for a specific response on the reply queue. Once they have gotten the message, they put another message to the request queue. The messages are put and got out of syncpoint.

There are 4 batch server tasks that action MQGET-with-wait calls on the request queue, get the message and put a reply to the known (and pre-opened) reply queue and then the application goes back into the MQGET-with-wait call. The messages are got and put in syncpoint with 1 MQGET and 1 MQPUT per commit.

An increasing number of queue managers are allocated to process the workload. Each queue manager has 5 requester and 4 server tasks.

Chart: Transaction rate for non-persistent in syncpoint workload - 2KB

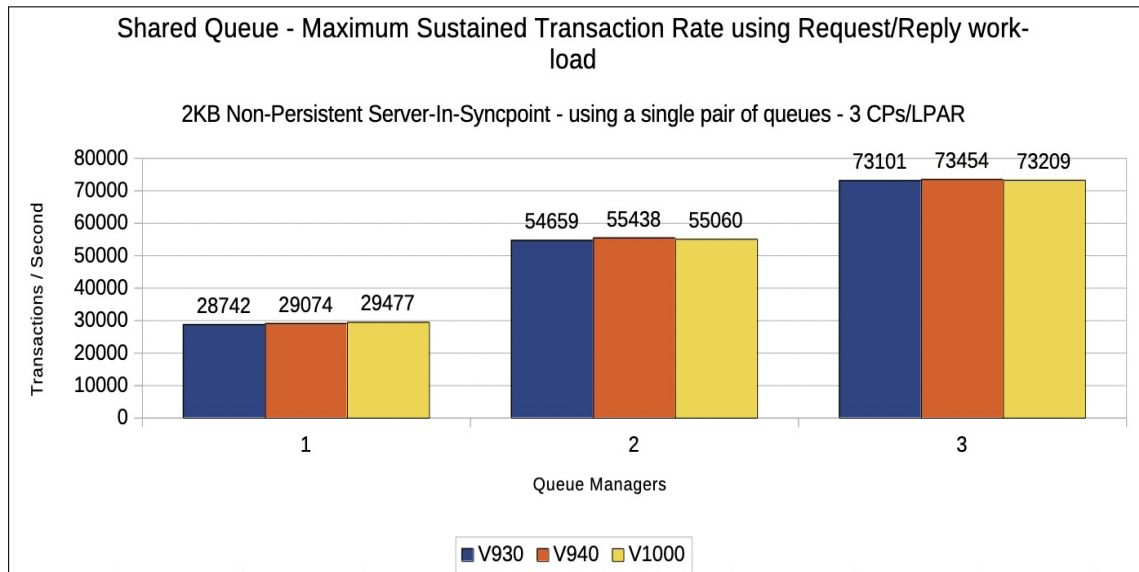


Chart: Transaction cost for non-persistent in syncpoint workload - 2KB

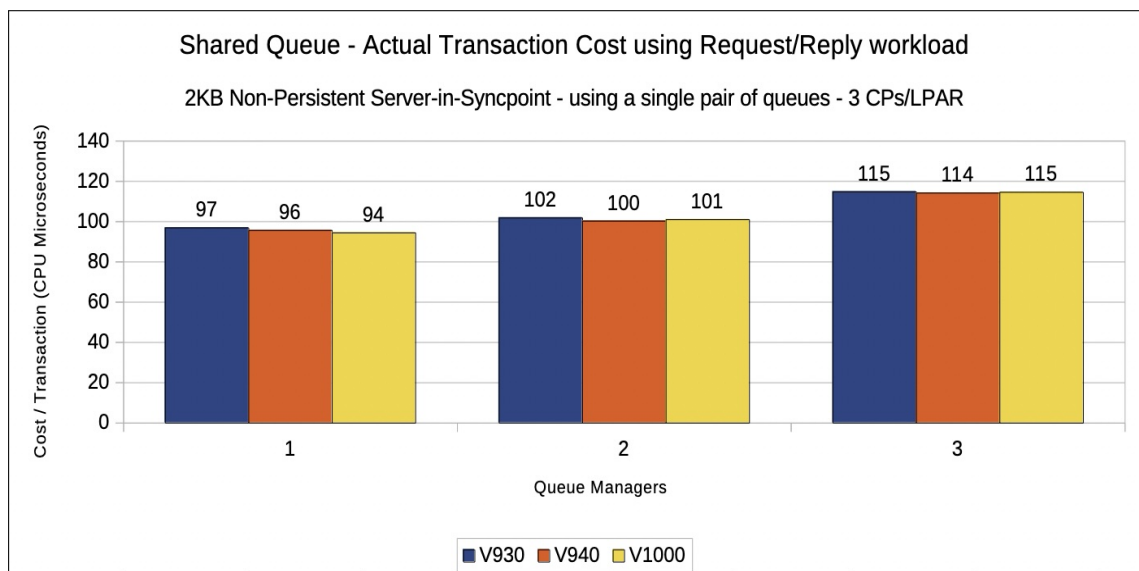


Chart: Transaction rate for non-persistent in syncpoint workload - 64KB

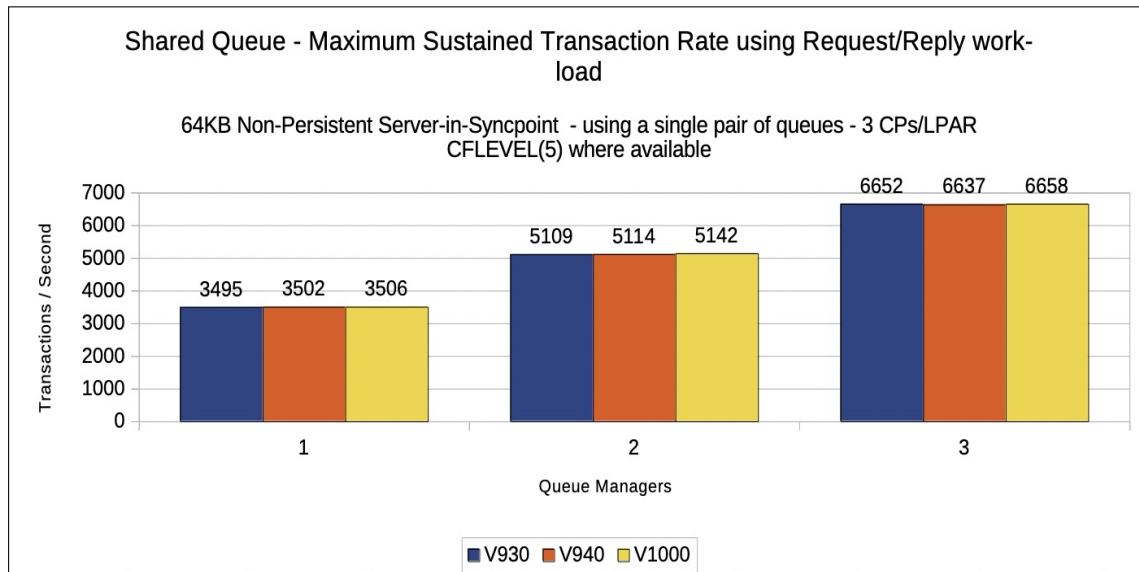


Chart: Transaction cost for non-persistent in syncpoint workload - 64KB

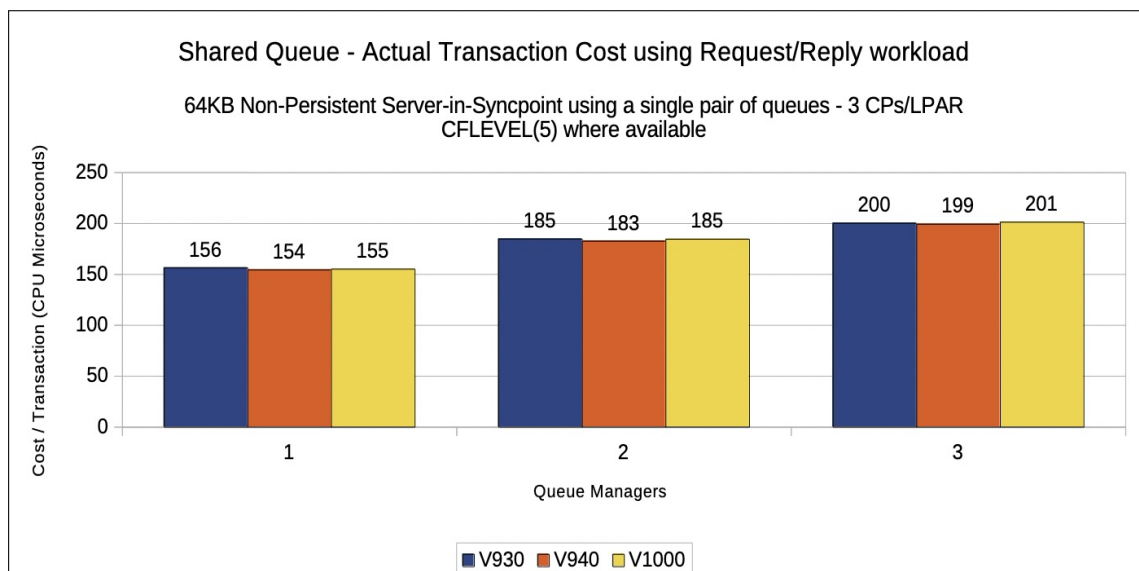


Chart: Transaction rate for non-persistent in syncpoint workload - 4MB

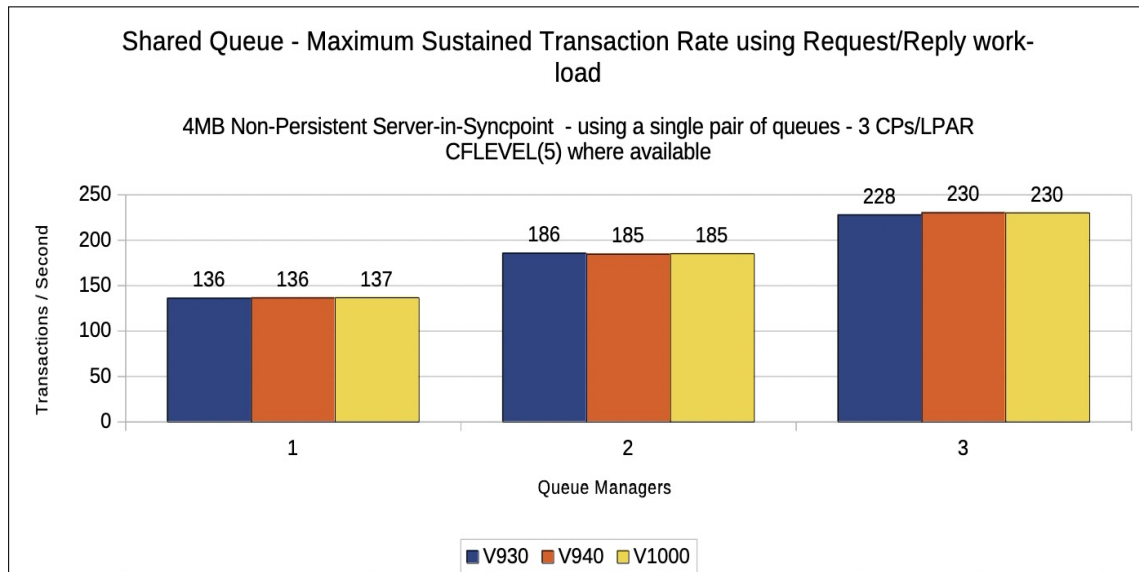
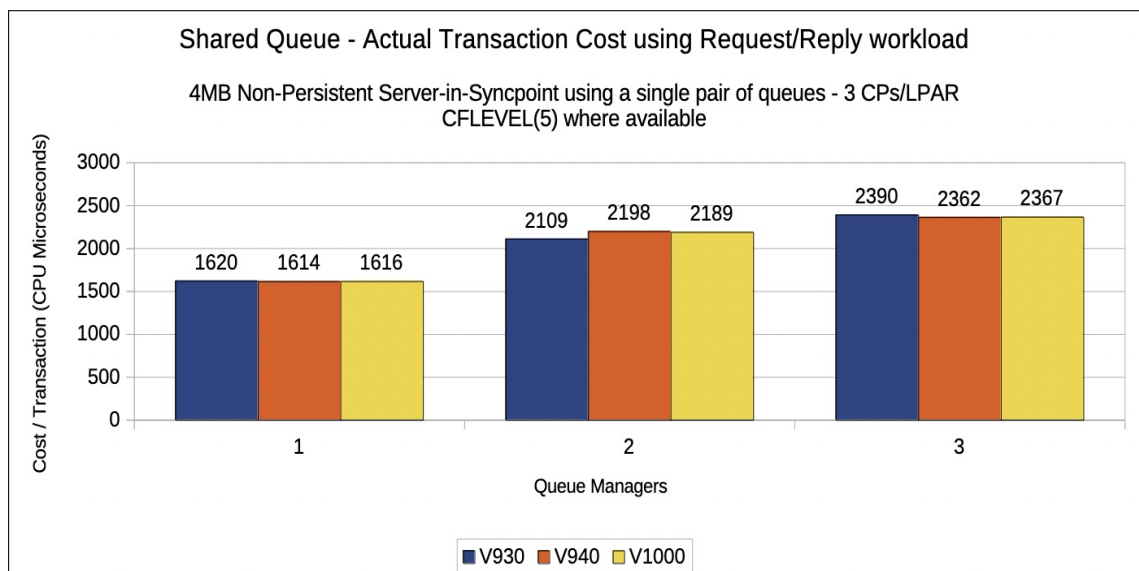


Chart: Transaction cost for non-persistent in syncpoint workload - 4MB



Data sharing non-persistent server in-syncpoint workload

The previous shared queue tests are configured such that any queue manager within the queue sharing group (QSG) can process messages put by any particular requester application. This means that the message may be processed by a server application on any of the available LPARs. Typically the message is processed by a server application on the same LPAR as the requester.

In the following tests, the message can only be processed by a server application on a separate LPAR. This is achieved by the use of multiple pairs of request/reply queues.

The test uses 5 batch requester tasks that each put a message to a common request queue and wait for a specific response on the reply queue. Once they have gotten the message, they put another message to the request queue. The messages are put and got out of syncpoint.

There are 4 batch server tasks that action MQGET-with-wait calls on the request queue, get the message and put a reply to the known (and pre-opened) reply queue and then the application goes back into the MQGET-with-wait call. The messages are got and put in syncpoint with 1 MQGET and 1 MQPUT per commit.

Chart: Transaction rate for data sharing non-persistent in syncpoint workload - 2KB

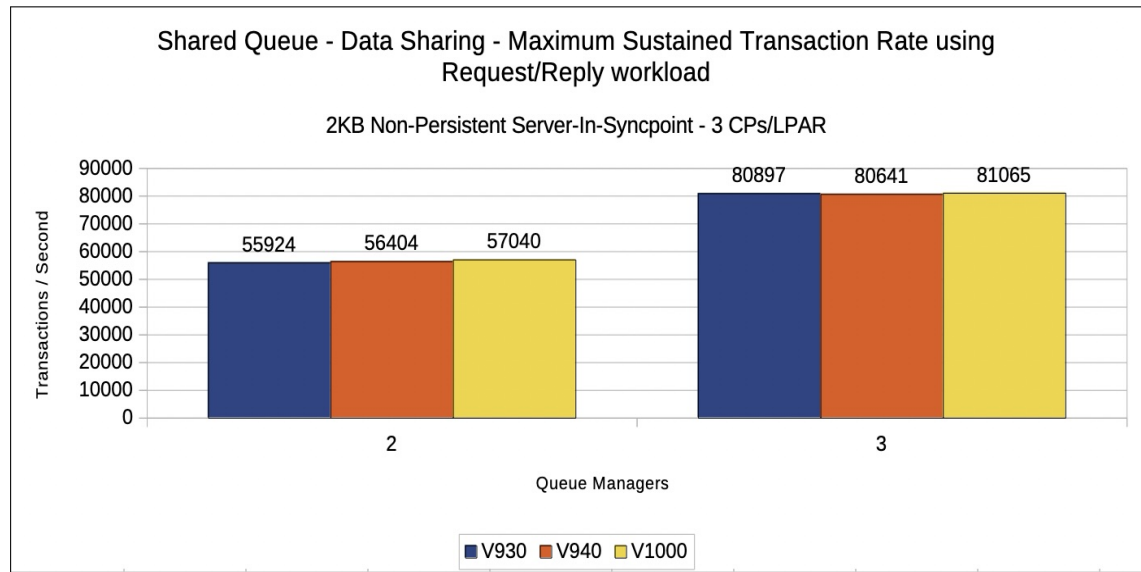


Chart: Transaction cost for data sharing non-persistent in syncpoint workload - 2KB

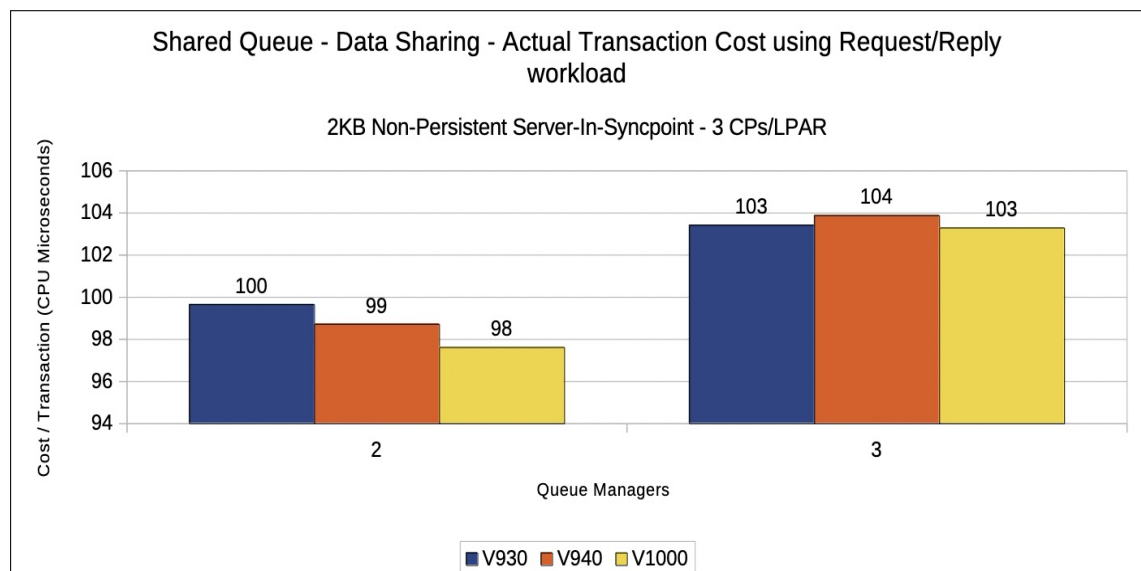


Chart: Transaction rate for data sharing non-persistent in syncpoint workload - 64KB

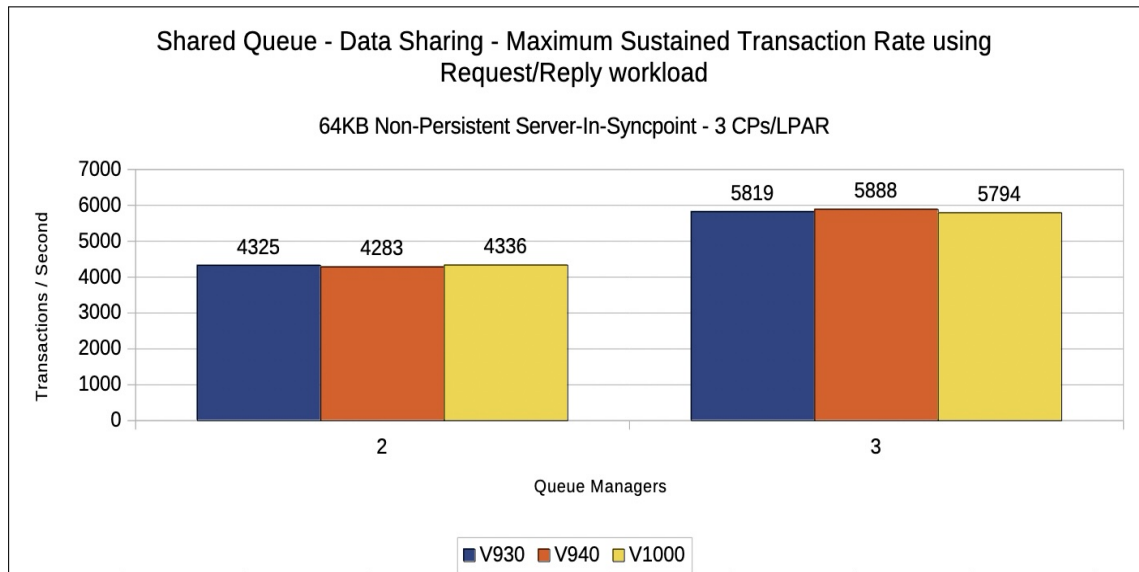
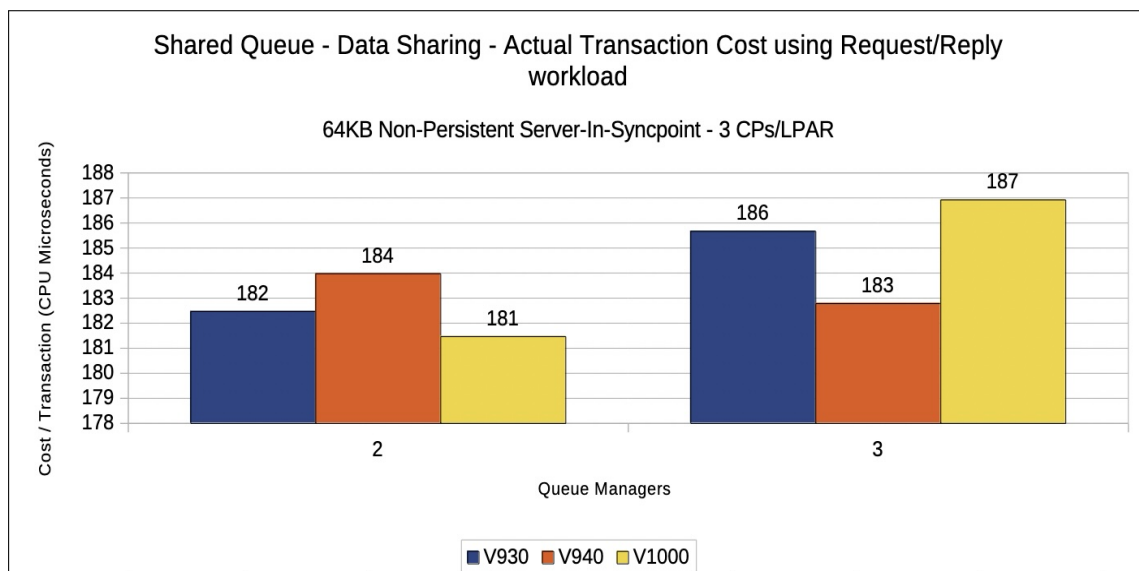


Chart: Transaction cost for data sharing non-persistent in syncpoint workload - 64KB



Moving messages across channels

The regression tests for moving message across channels e.g. sender-receiver channels, are designed to use drive the channel initiator such that system limits rather than MQ are the reason for any constraints, and typically it is CPU that is the constraining factor. Therefore the tests use non-persistent messages in-syncpoint, so that they are not constrained by logging etc.

Within the channel tests there are measurements with small numbers of channels (1 to 5 inbound plus the same number outbound), which was suitable for driving the LPARs to capacity.

Note: Higher throughput can be achieved with additional CPUs but this can increase the transaction cost.

Initially measurements are run both with and without the TLS/SSL encryption enabled.

The TLS 1.3 cipher spec “TLS_AES_256_GCM_SHA384” was used for all TLS/SSL tests.

The SSLRKEYC attribute was set as follows:

- 0 such that the secret key is only negotiated as channel start. This allows us to determine the impact of encryption on the data.
 - TLS 1.3 ciphers have key re-negotiation as part of the protocol, so the processing triggered by reaching the threshold as defined by the SSLRKEYC attribute does not perform key re-negotiation, but does have a non-zero cost. As such, we use SSLRKEYC(0) with our TLS 1.3 protected channels.

The TLS/SSL tests all used a certificate key-size of 2048 bits, which has been the default size for RSA since z/OS 2.2. The impact of certificate key-size is discussed in more detail in blog [Impact of certificate key-size on TLS-protected MQ channels](#).

Channel compression

Since IBM MQ version 8.0.0, the ZLIBFAST compression option is able to exploit zEDC hardware compression, which is discussed in detail in [MQ for z/OS - Channel Compression](#). The compression measurements shown in this document use zEDC hardware compression where possible.

For further guidance on channel tuning and usage, please refer to performance report [MP16](#) “Capacity Planning and Tuning Guide”.

Additionally, the white paper "[MQ for z/OS - Channel Compression](#)" discusses the performance of channel compression on z/OS including where compression may help improve the performance, either by reducing MQ costs or improving channel throughput.

Measurements:

- The measurements using 1 to 5 channels use batch applications to drive the workload at each end of the channel.
- The compression tests use 32KB message of varying compressibility, so there is something to compress, e.g. a message of 32KB that is 80% compressible would reduce to approximately 6.5KB. These tests are run using 1 outbound and 1 inbound channel so include the compression and inflation costs for the request and reply message.

Streaming messages across channels

Many of the performance tests in the regression section use a request/reply model to demonstrate the performance of the MQ channels.

One common use of MQ is to provide the transport mechanism when data is moved between data centres, such as in an IBM InfoSphere Data Replication queue replication (QREP) scenario. This

model sends data in a single direction and can show different characteristics to a typical request/reply model. For example the queue depths may naturally vary significantly, and in some cases may drive MQ page set I/O.

Typically a QREP-scenario would use persistent messages and may have different message sizes for online and batch processing. In this section we use 10KB messages for online processing and 1MB messages for batch processing.

In the streaming configuration, the transmit queue is pre-loaded with messages to ensure the channel is able to send full batches (of 200 messages), thus driving the channel to capacity.

Non-persistent in-syncpoint - 1 to 5 sender receiver channels

Chart: Transaction rate with 32KB messages

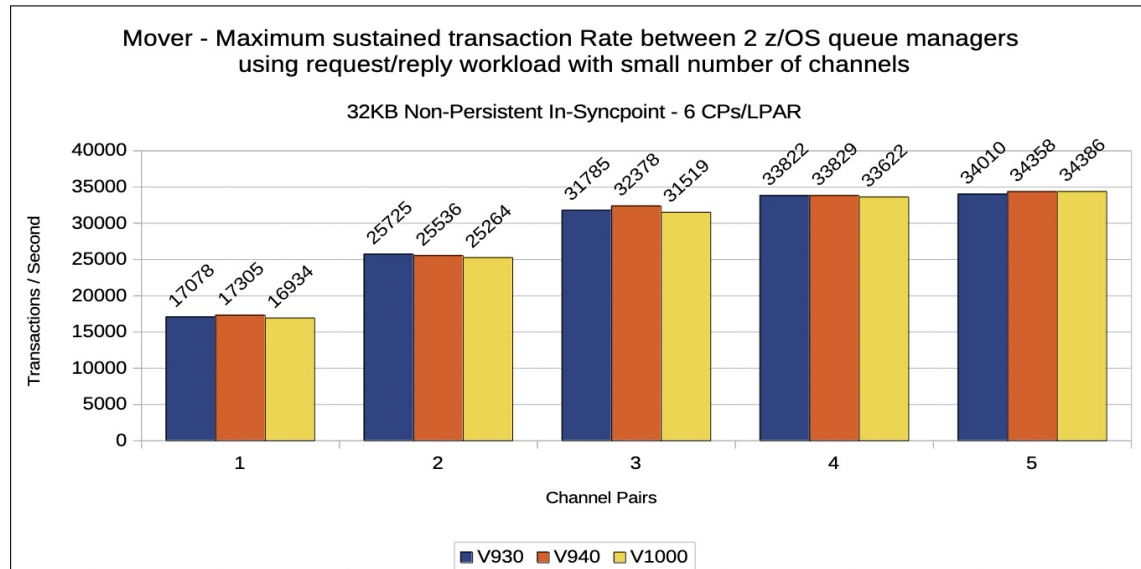


Chart: Transaction cost for 32KB messages

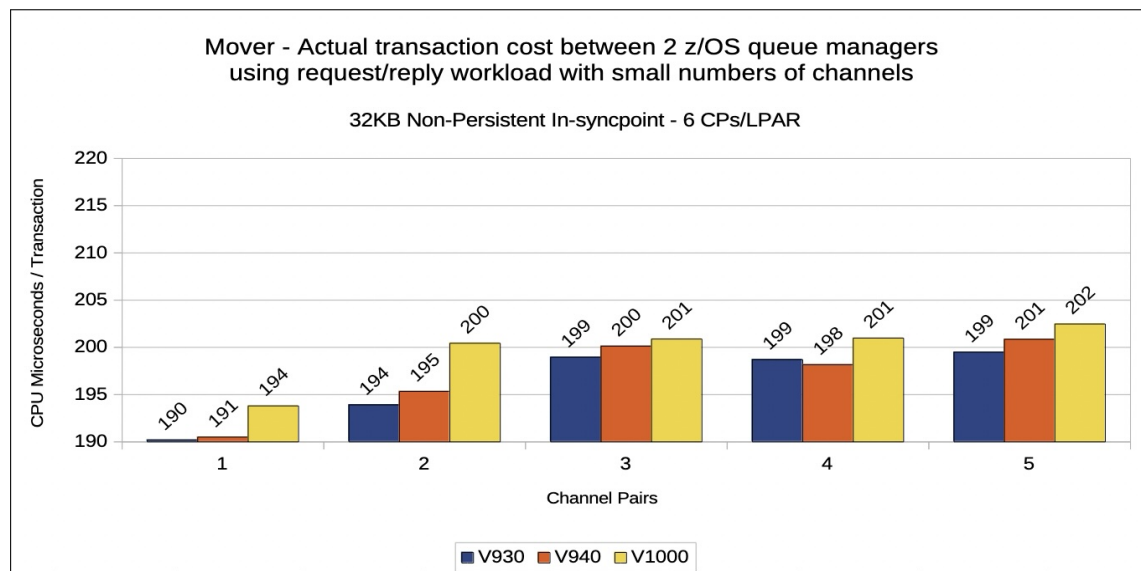


Chart: Transaction rate with 32KB messages over SSL channels with no secret key negotiation

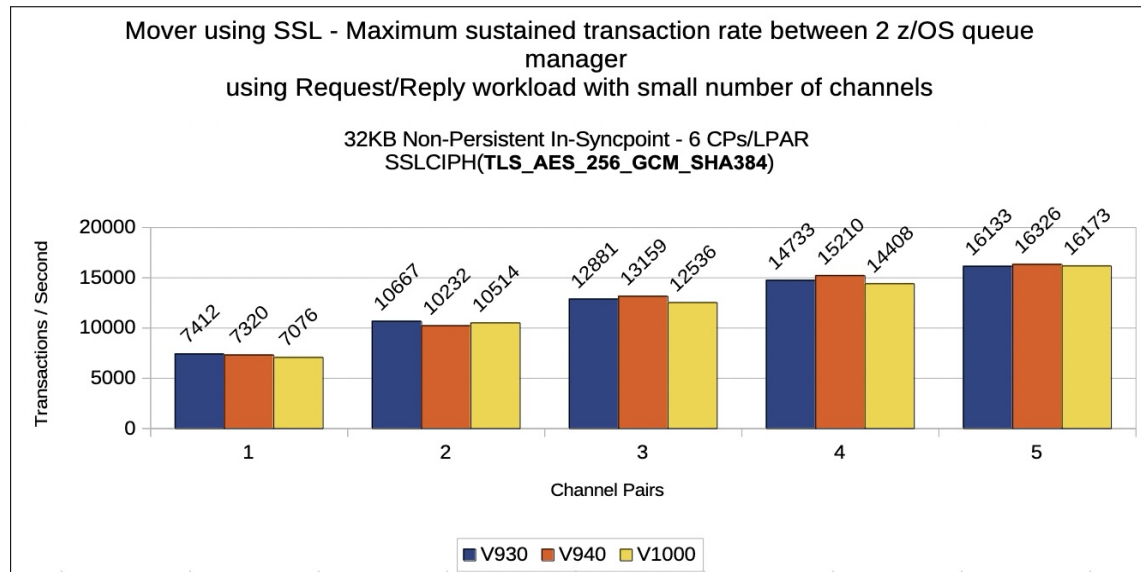
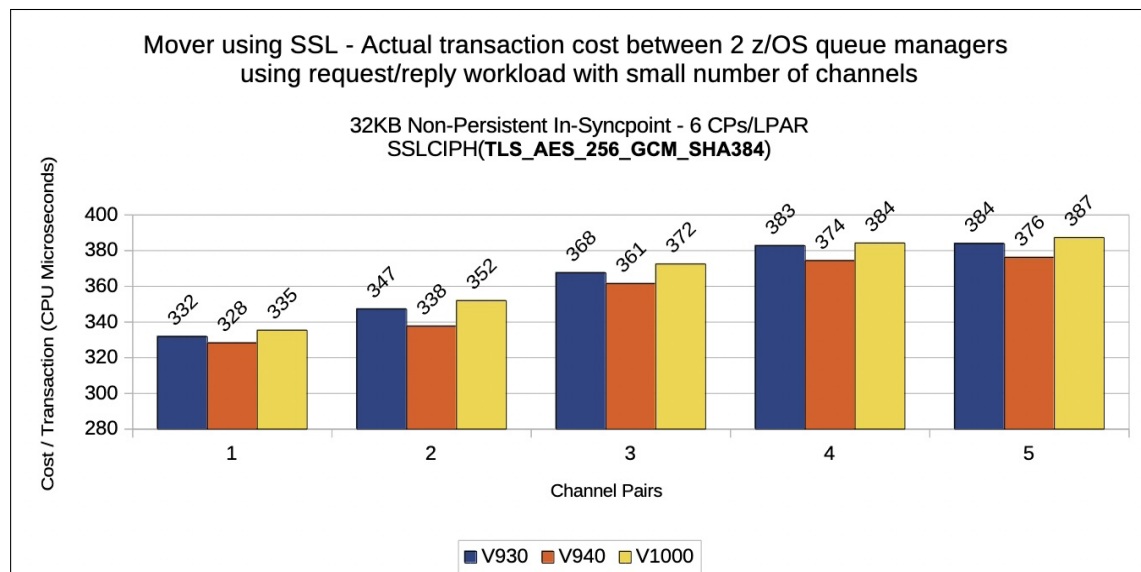


Chart: Transaction cost for 32KB messages over SSL channels with no secret key negotiation



Channel compression using ZLIBFAST

Version 8.0.0 onwards are able to exploit compression using the available zEDC hardware features.

Chart: Transaction rate with ZLIBFAST on 32KB messages

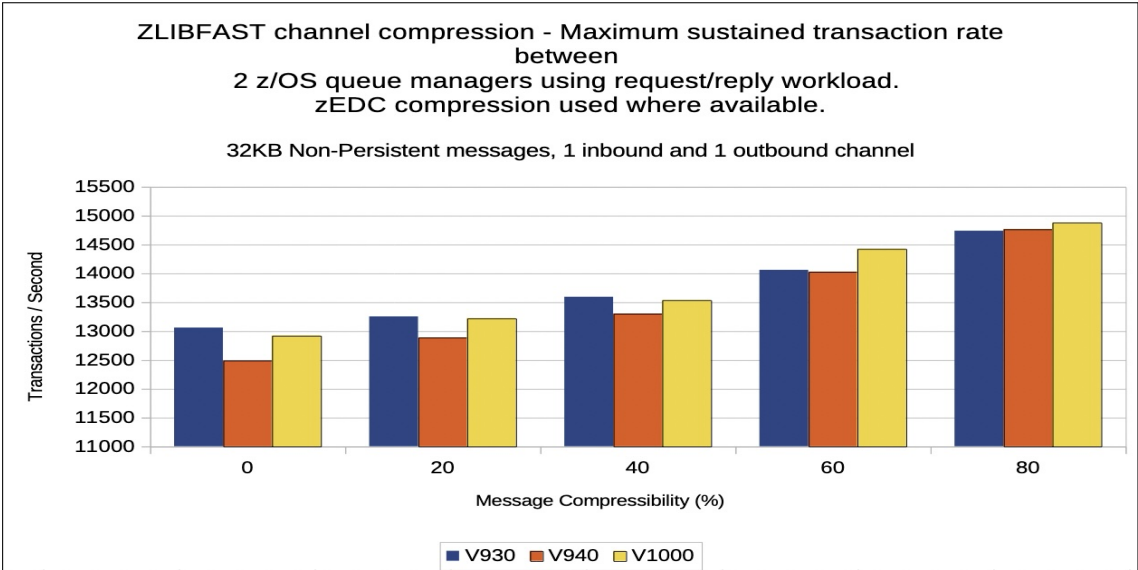
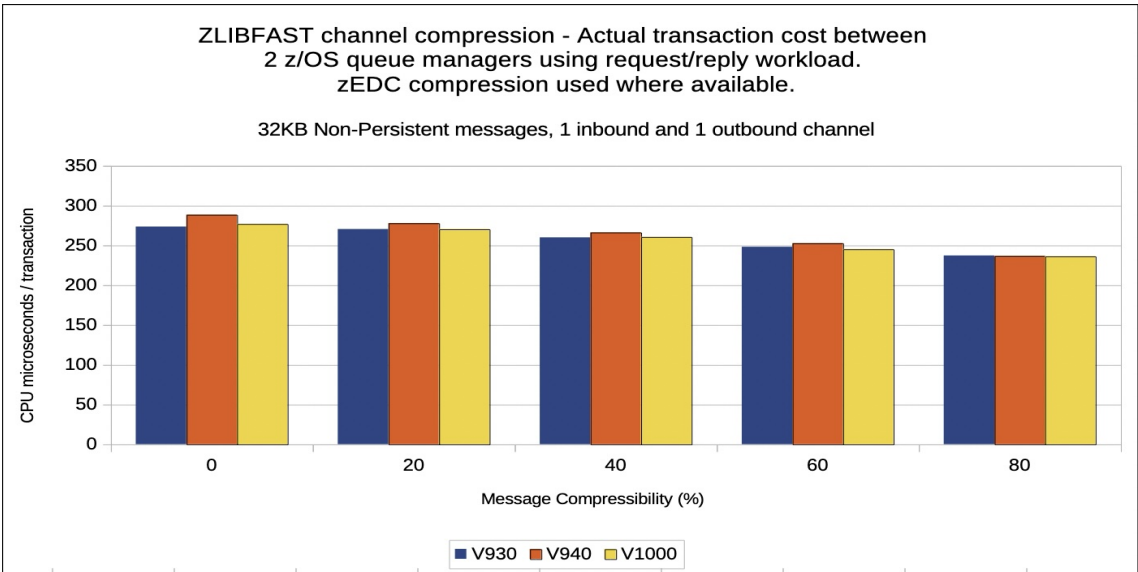


Chart: Transaction cost for ZLIBFAST on 32KB messages



Channel compression using ZLIBHIGH

Chart: Transaction rate with 32KB messages with ZLIBHIGH

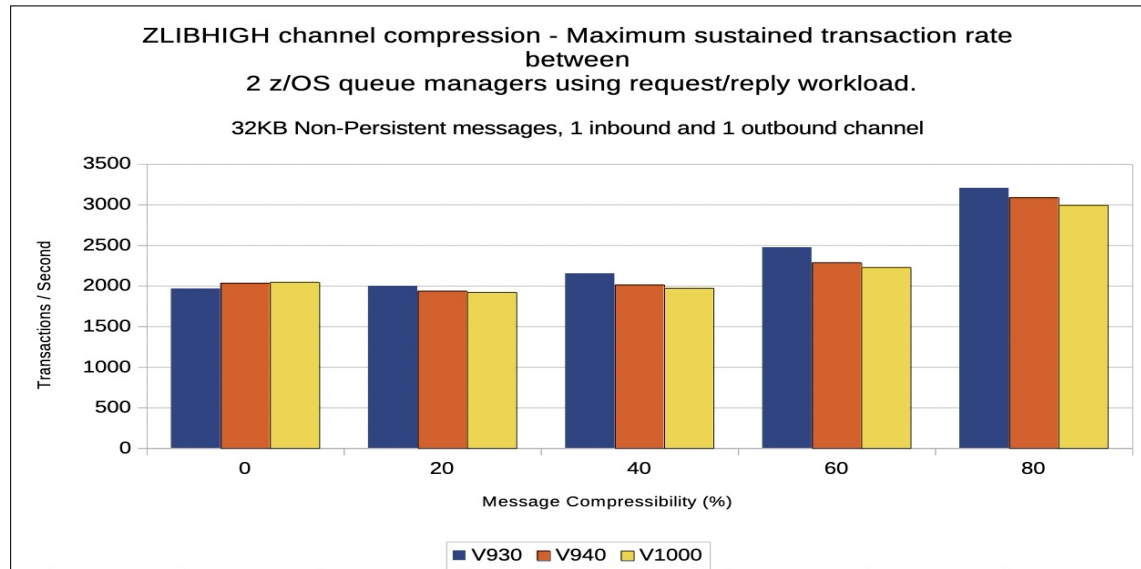
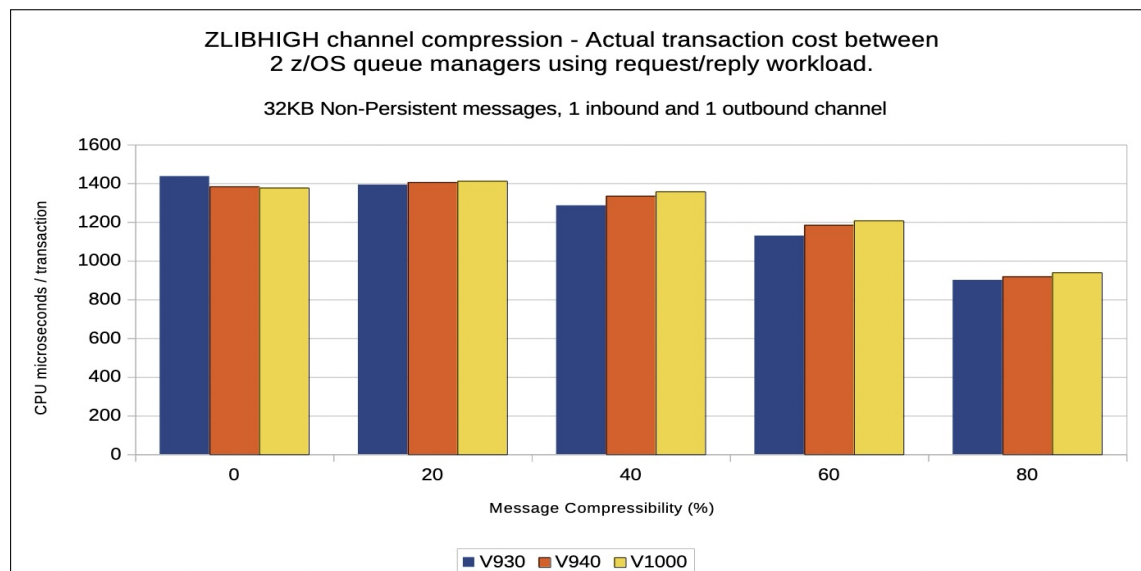


Chart: Transaction cost for 32KB messages with ZLIBHIGH



Channel compression using ZLIBFAST on SSL channels

Version 8.0.0 onwards are able to exploit compression using the available zEDC hardware features.

Chart: Transaction rate with 32KB messages with ZLIBFAST on SSL channels

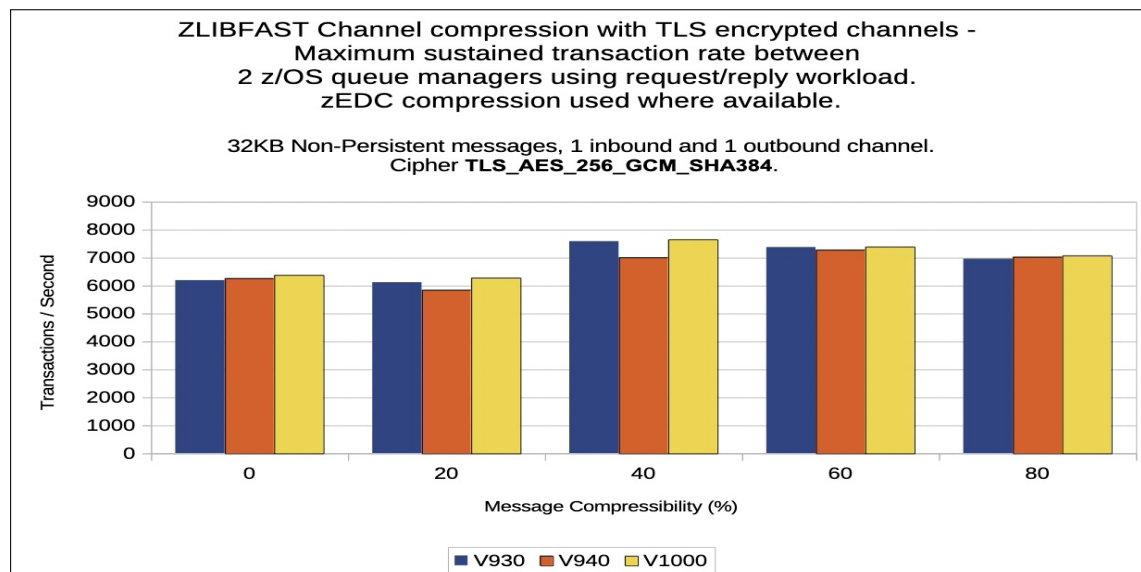
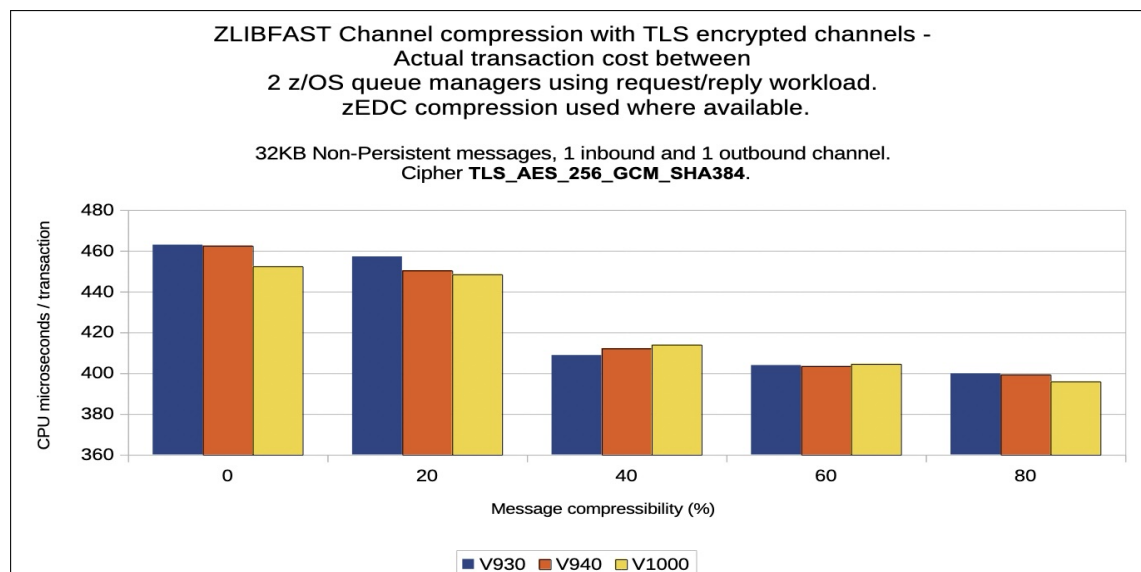


Chart: Transaction cost with 32KB messages with ZLIBFAST on SSL channels



Channel compression using ZLIBHIGH on SSL channels

Chart: Transaction rate with 32KB messages with ZLIBHIGH on SSL channels

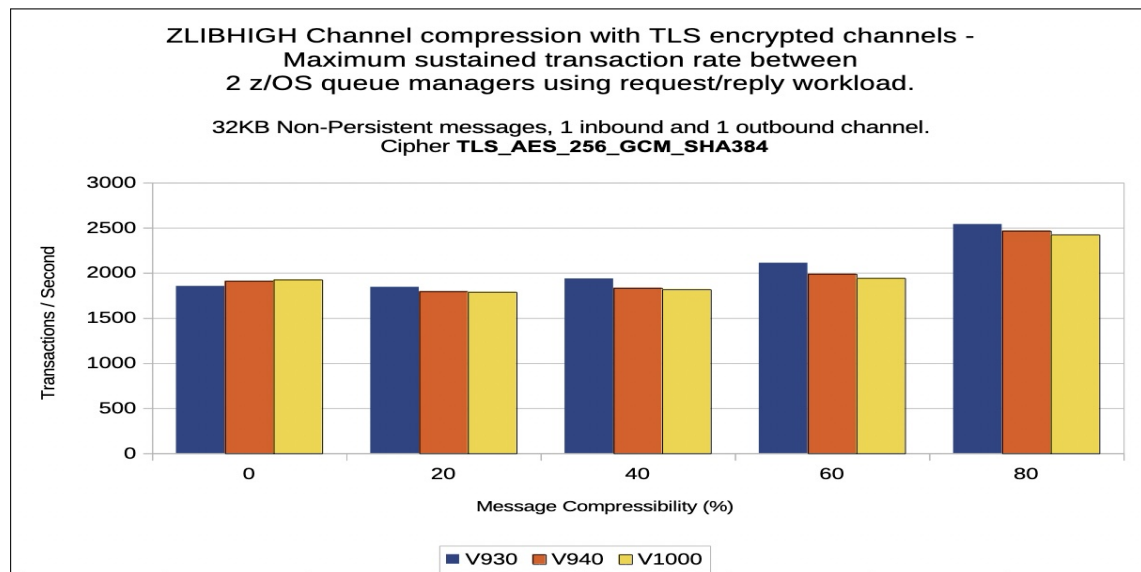
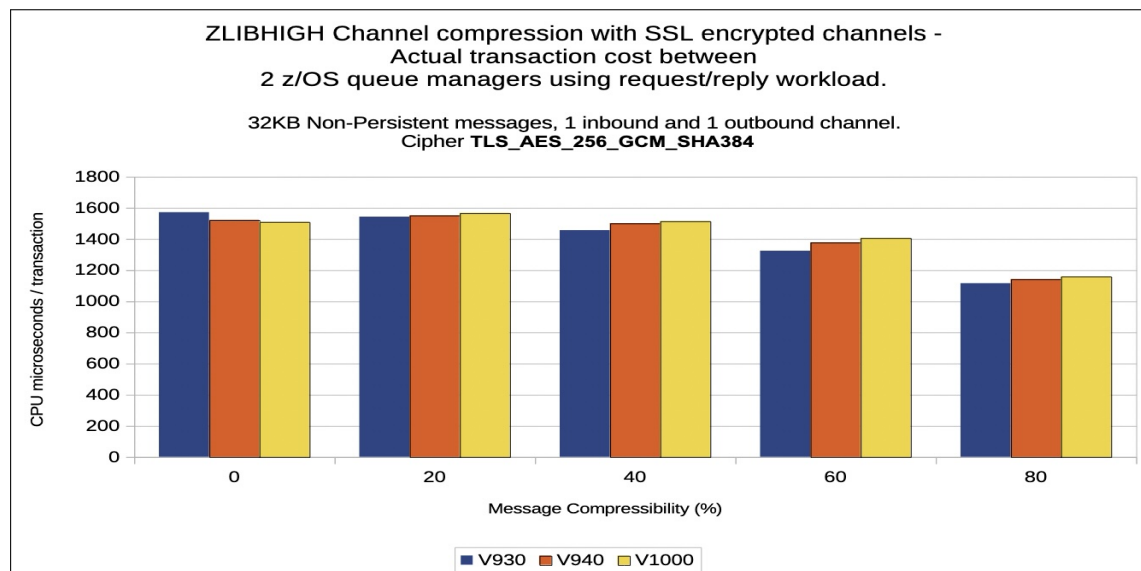


Chart: Transaction cost for 32KB messages with ZLIBHIGH on SSL channels



Streaming workload between 2 z/OS queue managers

Chart: Rate (MB/Second) to stream persistent messages between 2 z/OS queue managers

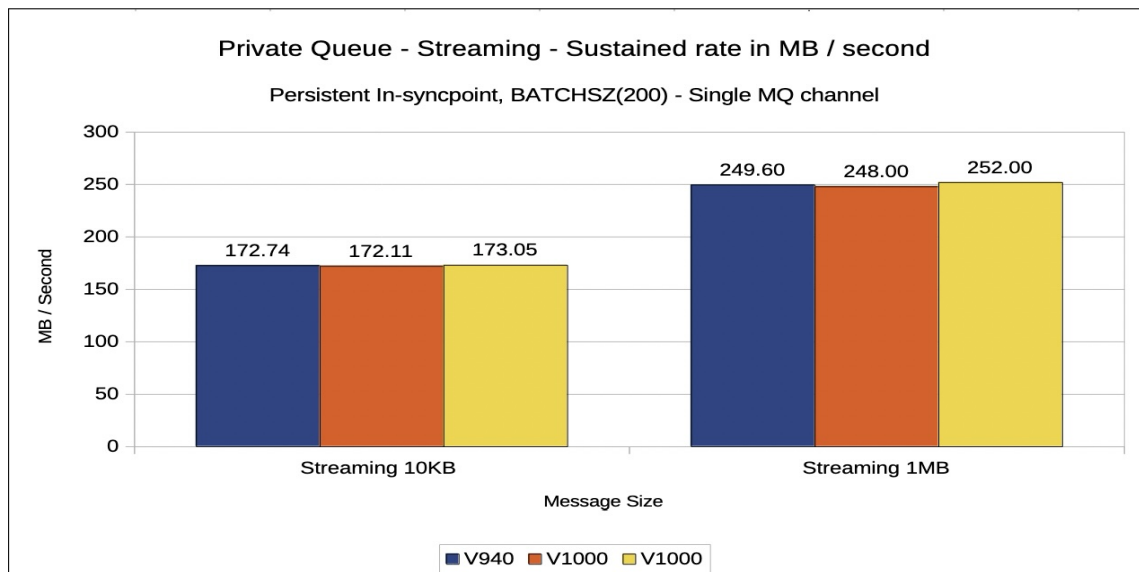
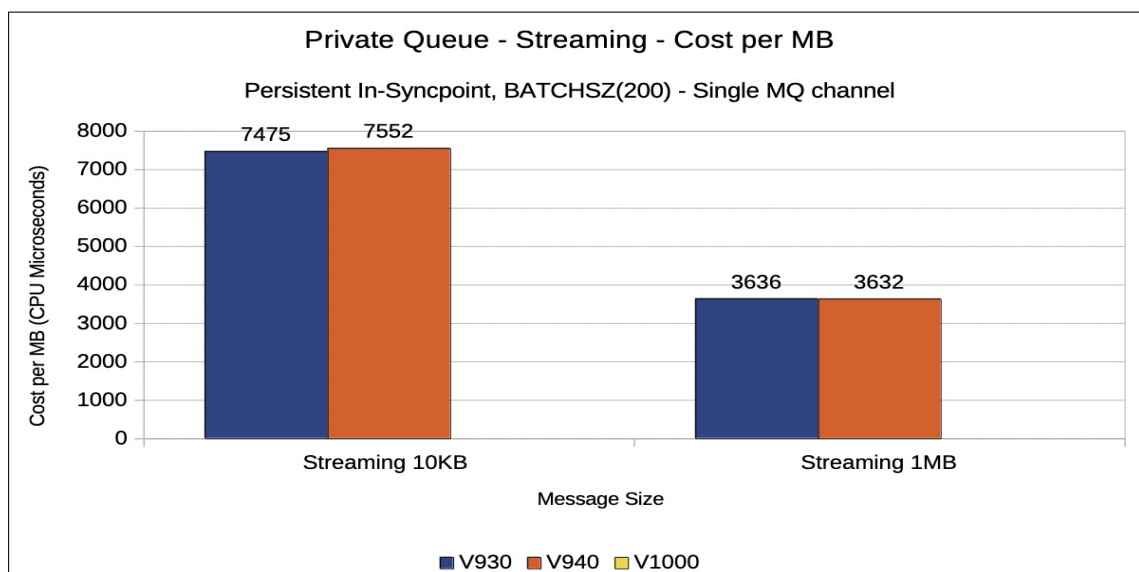


Chart: Cost to stream persistent messages between 2 z/OS queue managers



Moving messages across cluster channels

The regression tests for moving messages across cluster channels are relatively simple, providing an increasing number of destinations for each message put.

The cluster has 8 queue managers - one for the requester workload and the remaining 7 for the server workload. As the test progresses, valid destination queues are defined on an increasing number of the server queue managers. Only the requester queue manager is a full repository - this is not a recommended configuration!

A set of requester tasks is run against one queue manager and the application chooses either bind-on-open or bind-not-fixed. The requester application is written to use an 'MQOPEN, MQPUT, MQCLOSE' model to put messages to the queue. This ensures that the messages are distributed evenly when using the bind-on-open option.

The messages flow across the cluster channels to one of the server queue managers to be processed by the server applications, at which point they are returned to the originating requesting applications.

The measurements are run with 2KB messages and again with 32KB messages. In the 2KB workloads, the influence of the selection of the destination and the general cluster "overhead" is more prevalent.

Bind-on-open

Chart: Bind-on-open transaction rate with 2KB messages

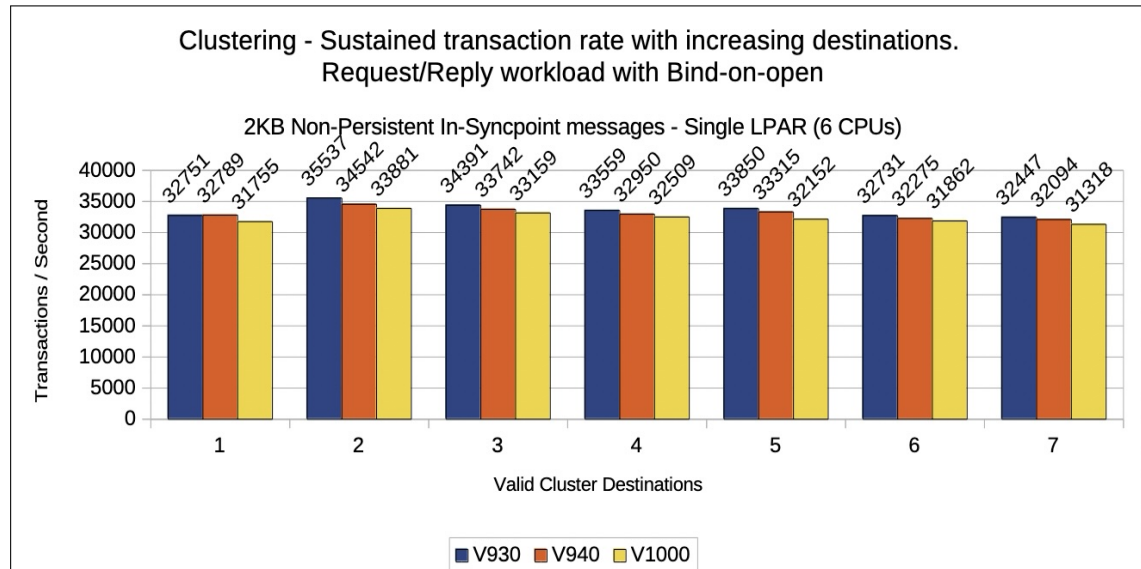


Chart: Bind-on-open transaction cost for 2KB messages

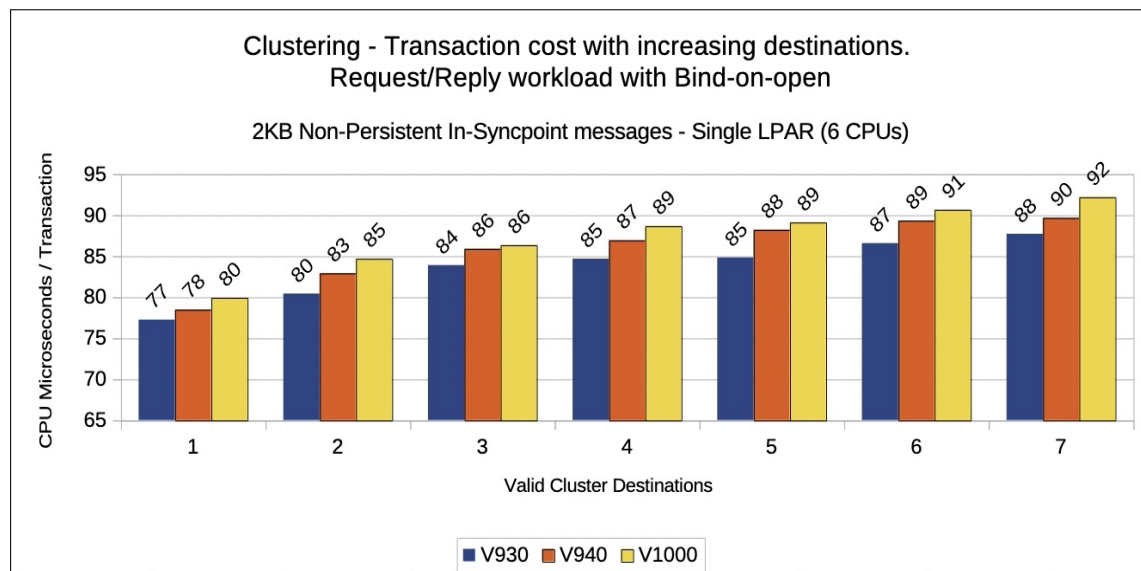


Chart: Bind-on-open transaction rate with 32KB messages

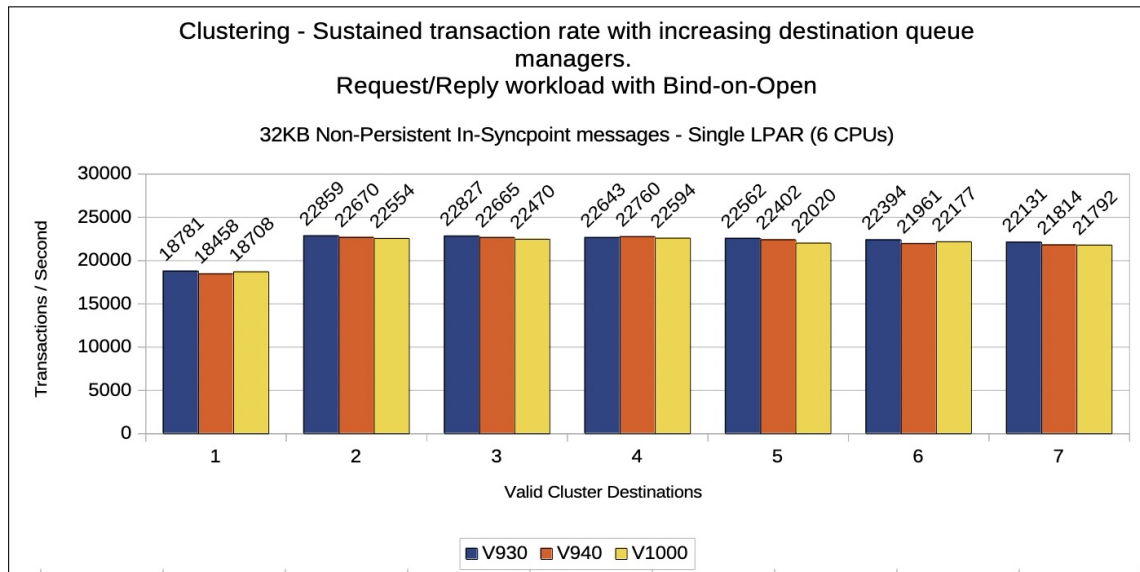
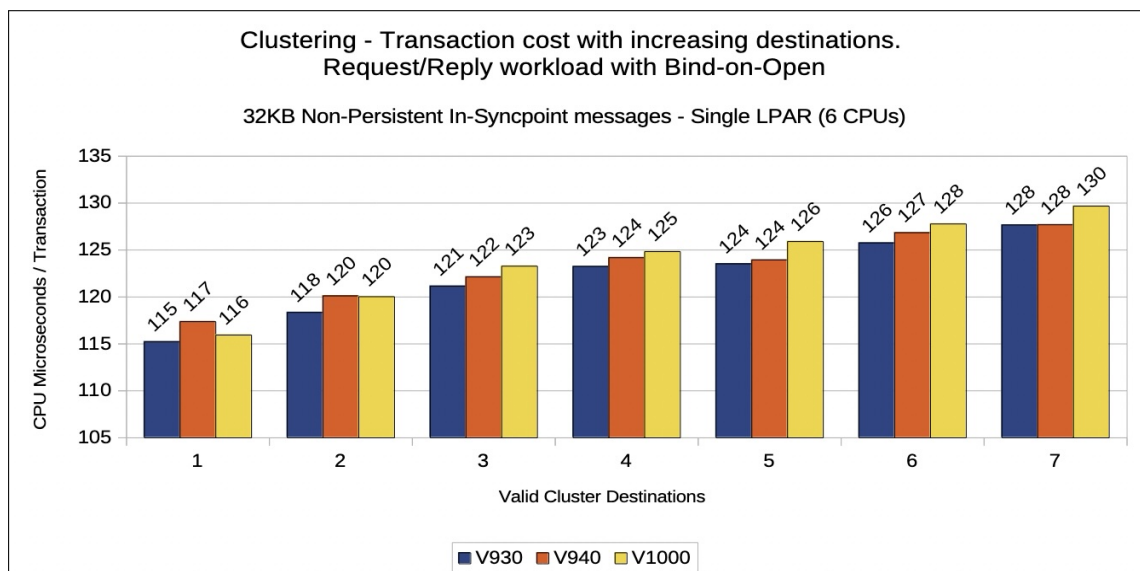


Chart: Bind-on-open transaction cost for 32KB messages



Bind-not-fixed

Chart: Bind-not-fixed transaction rate with 2KB messages

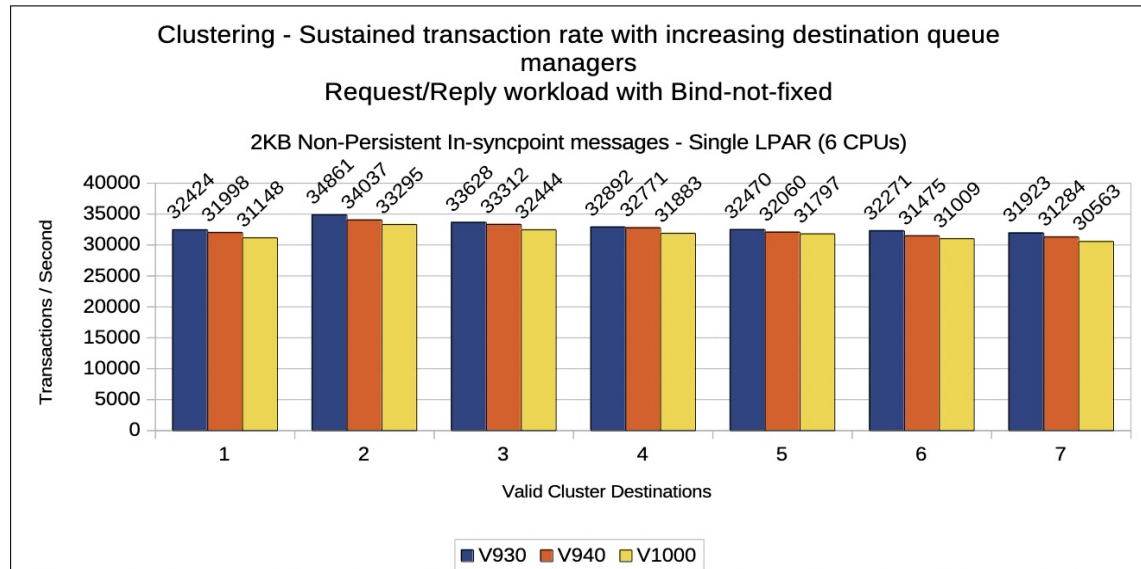


Chart: Bind-not-fixed transaction cost for 2KB messages

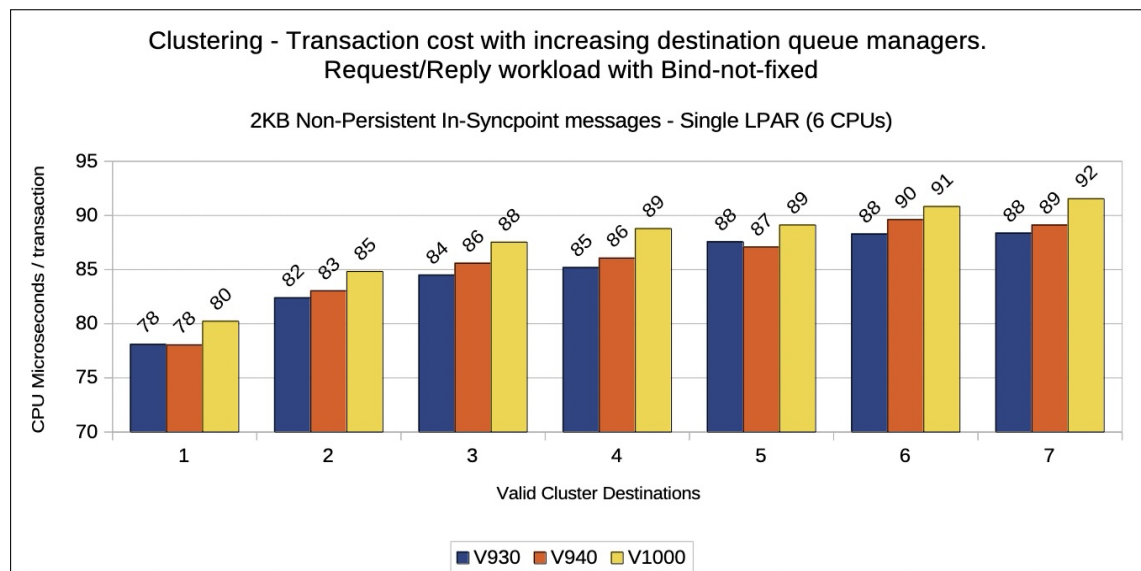


Chart: Bind-not-fixed transaction rate with 32KB messages

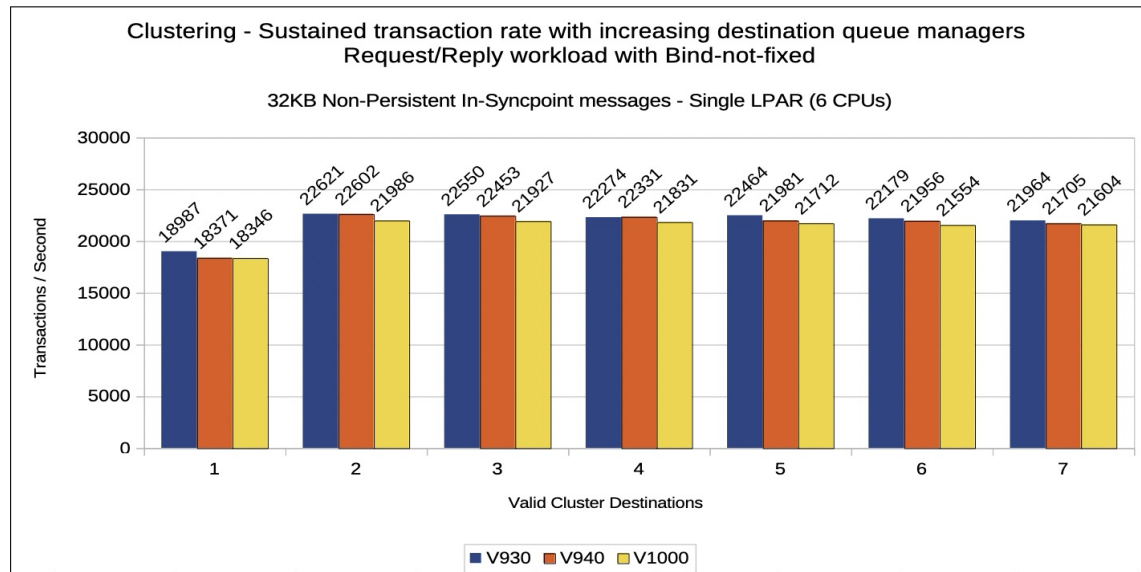
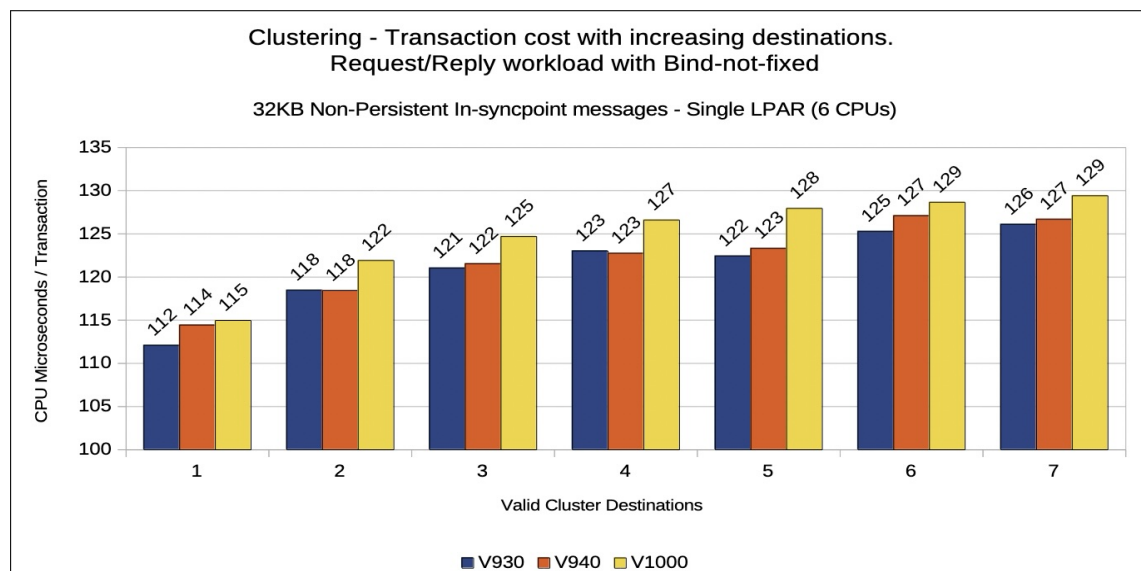


Chart: Bind-not-fixed transaction cost for 32KB messages



Moving messages across SVRCONN channels

The regression tests for moving messages across SVRCONN channels have a pair of client tasks for each queue that is hosted on the z/OS queue manager.

One of the pair of client tasks puts messages to the queue and the other client task gets the messages from the queue. As the test progresses, an increasing number of queues are used, with a corresponding increase in the number of putting and getting client tasks.

Two sets of tests are run, the first uses SHARECNV(0) on the SVRCONN channel to run in a mode comparable to that used pre-version 7.0.0. The second uses SHARECNV(1) so that function such as asynchronous puts and asynchronous gets are used via the DEFPRESP(ASYNC) and DEFREADA(YES) queue options.

Choosing which SHARECNV option is appropriate can make a difference to [capacity](#) and the performance which is discussed in more detail in performance report [MP16](#) “Capacity Planning and Tuning Guide” Channel Initiator section.

Note: The rate and costs are based upon the number of MB of data moved per second rather than the number of messages per second.

Client pass through test using SHARECNV(0)

Chart: Throughput rate for client pass through tests with SHARECNV(0)

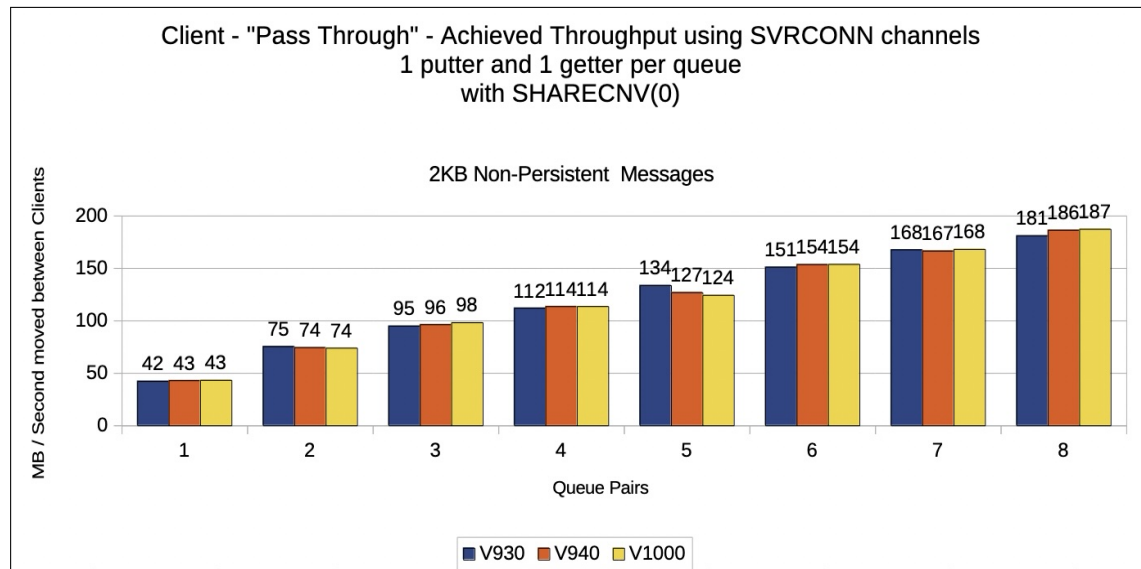
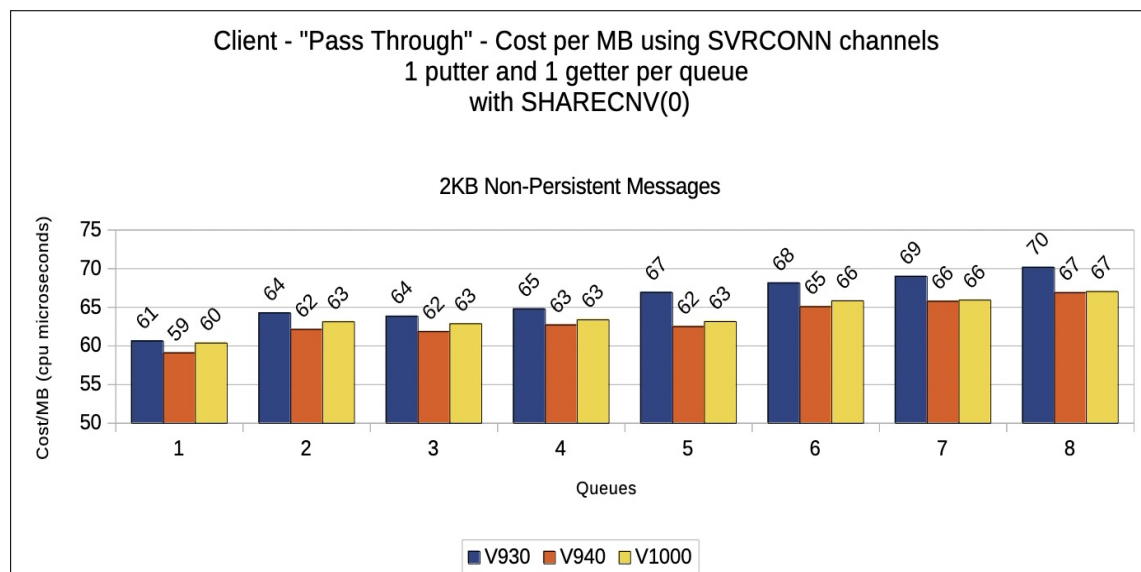


Chart: Cost per MB of client pass through tests with SHARECNV(0)



Client pass through test using SHARECNV(1)

Chart: Throughput rate for client pass through tests with SHARECNV(1)

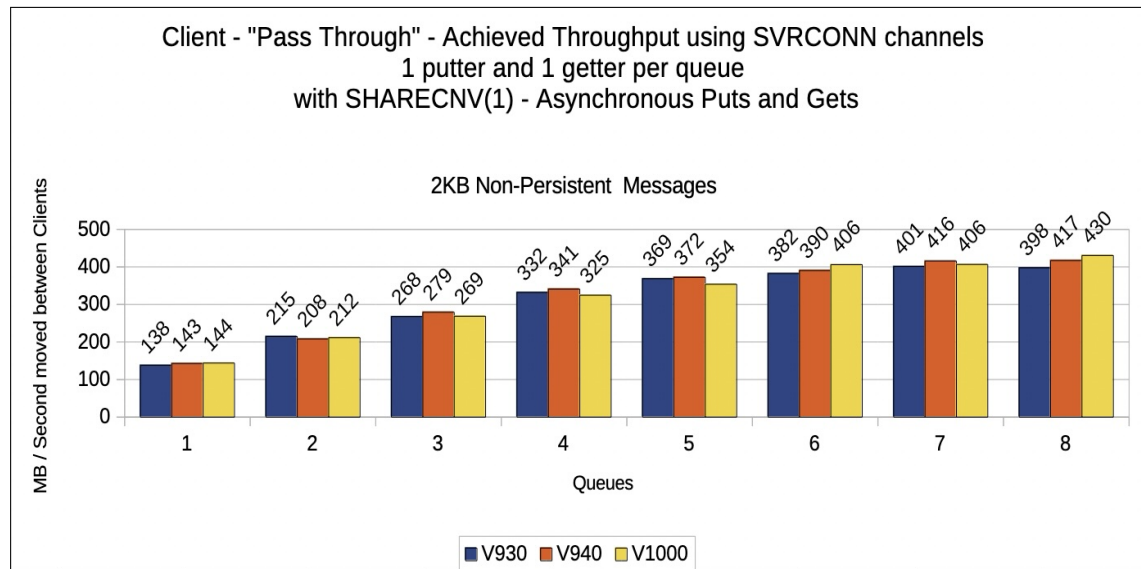
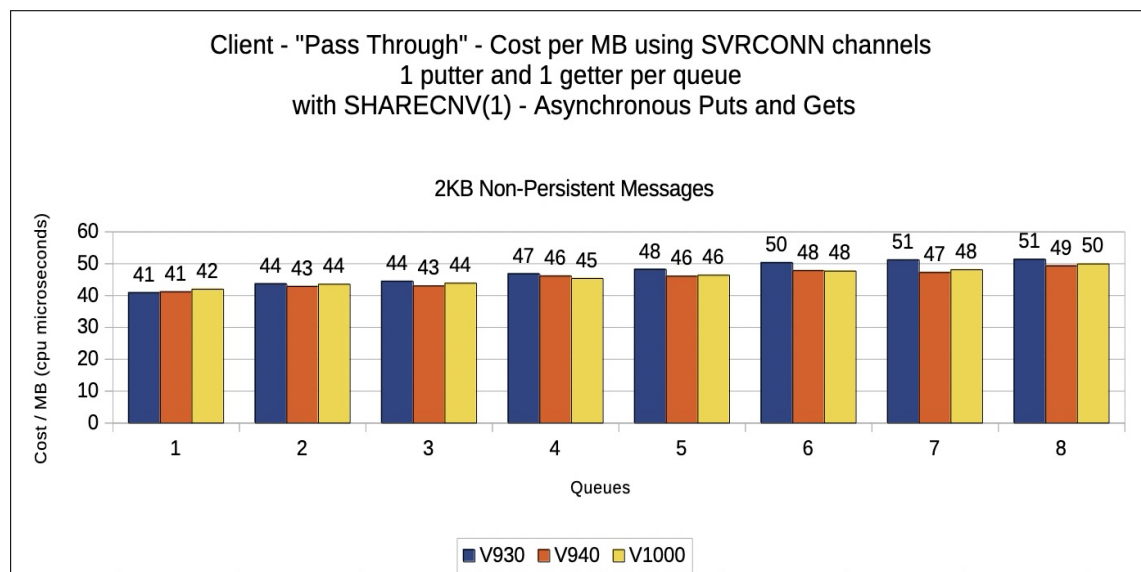


Chart: Cost per MB of client pass through tests with SHARECNV(1)



IMS Bridge

The regression tests used for IMS Bridge use 3 queue managers in a queue sharing group (QSG) each on separate LPARs. A single IMS region is started on 1 LPAR and has 16 Message Processing Regions (MPRs) started to process the MQ workload.

The IMS region has been configured as detailed in performance report [MP16](#) “Capacity Planning and Tuning Guide” using the recommendations in the section “IMS Bridge: Achieving Best Throughput”.

Note: 16 MPRs are more than really required for the 1, 2 and 4 TPIPE tests but they are available for consistent configuration across the test suite.

There are 8 queues defined in the QSG that are configured to be used as IMS Bridge queues.

Each queue manager runs a set of batch requester applications that put a 2KB message to one of the bridge queues and waits for a response on a corresponding shared reply queue.

Tests are run using both Commit Mode 0 (Commit-then-Send) and Commit Mode 1 (Send-then-Commit).

Commit mode 0 (commit-then-send)

Chart: IMS Bridge commit mode 0 - Throughput rate

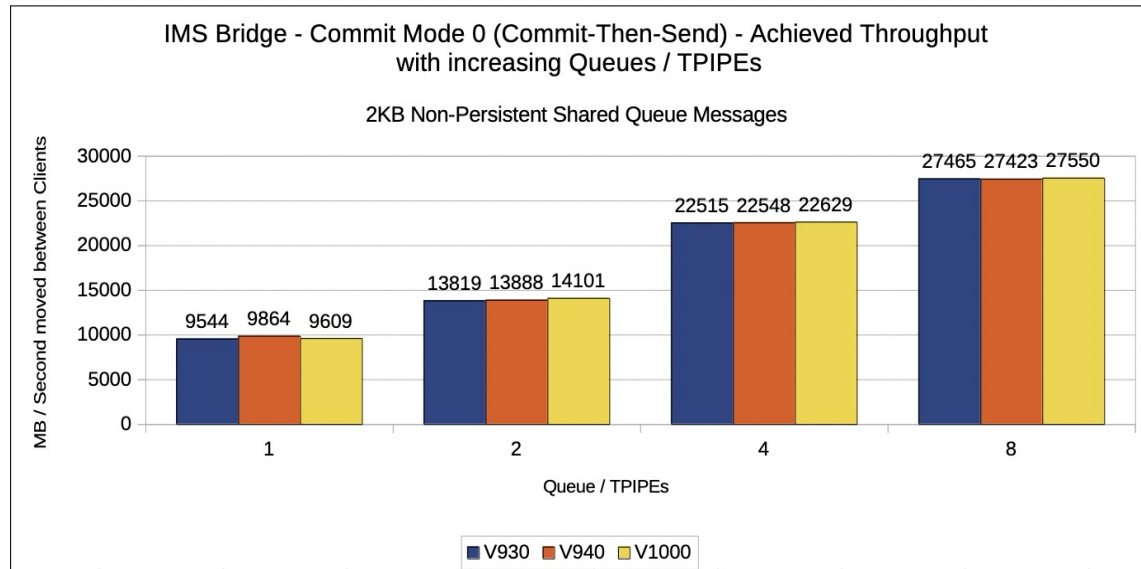
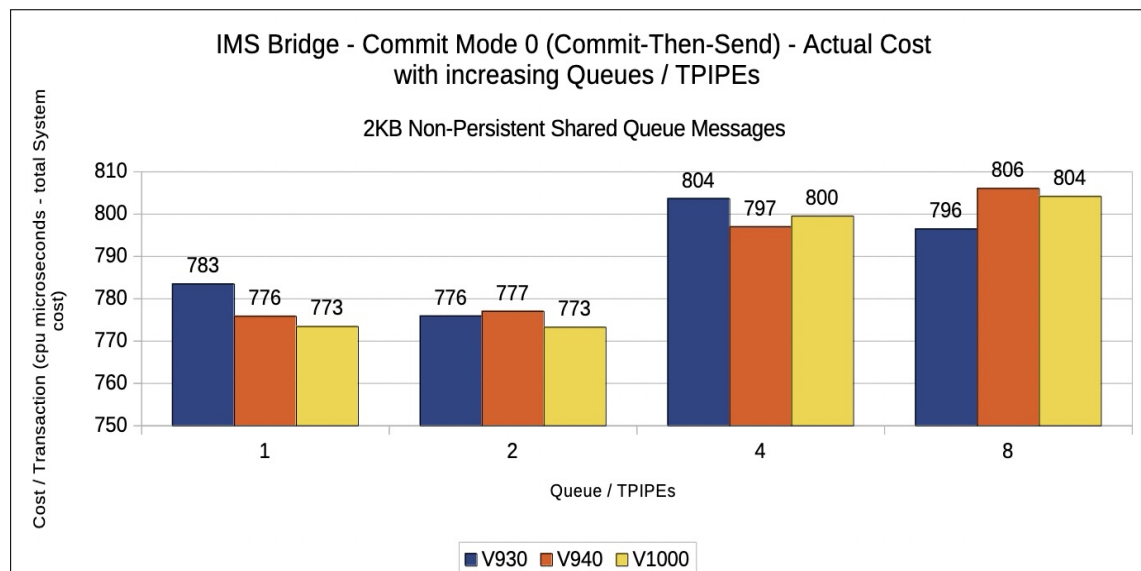


Chart: IMS Bridge commit mode 0 - Transaction cost



Commit mode 1 (send-then-commit)

Chart: IMS Bridge commit mode 1 - Throughput rate

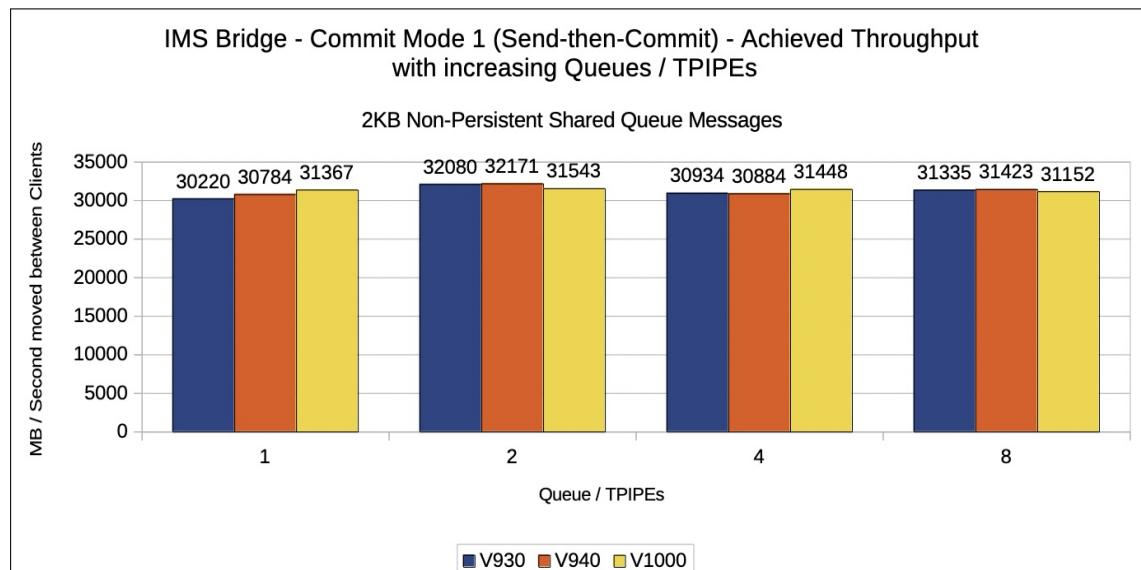
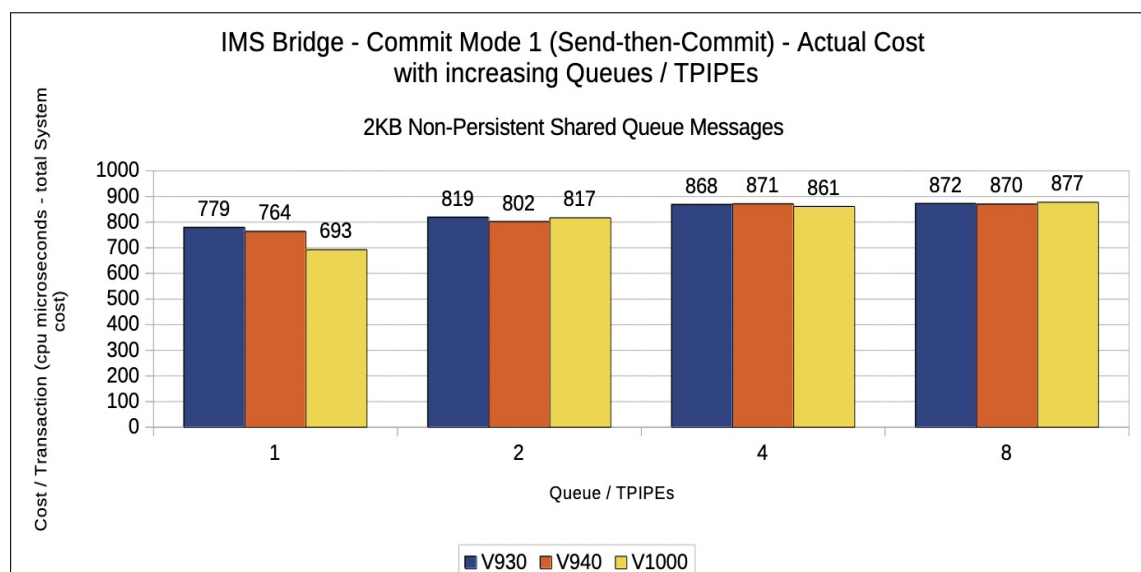


Chart: IMS Bridge commit mode 1 - Transaction cost



Trace

The regression tests for trace cover both queue manager global trace and the channel initiator trace. Since the performance for 9.3, 9.4 and 10.0 is comparable, the charts in this section show only 10.0 data for the purpose of clarity.

Queue manager global trace

The queue manager global trace tests are a variation on the [private queue non-persistent 2KB scalability tests](#) with TRACE(G) DEST(RES) enabled.

To enable a comparison of impact from enabling MQ's global trace, the results from the equivalent measurements without trace enabled are included.

Chart: Queue manager TRACE(G) DEST(RES) - transaction rate

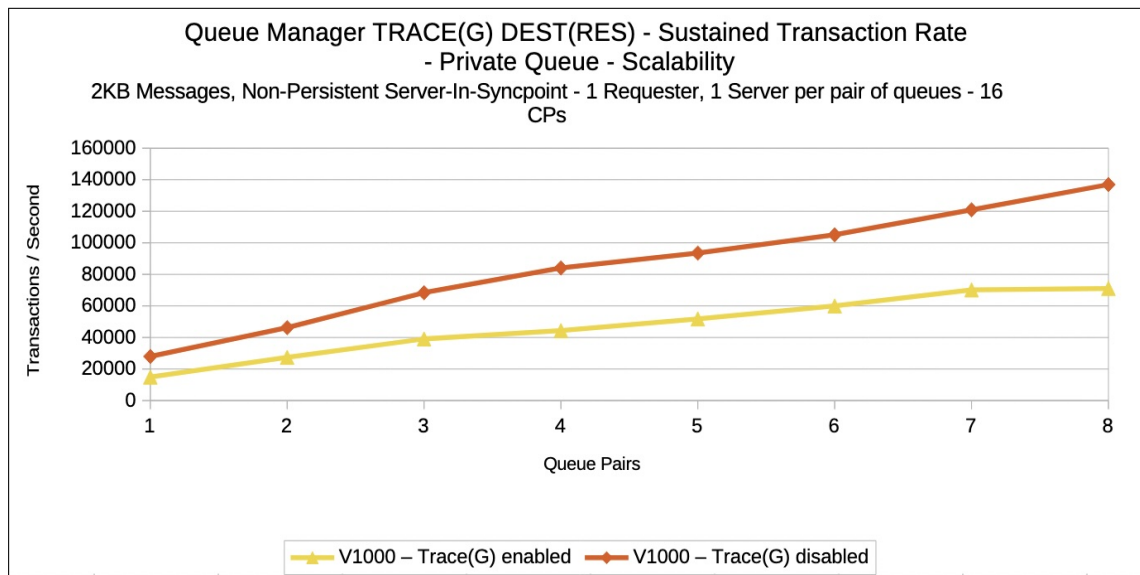
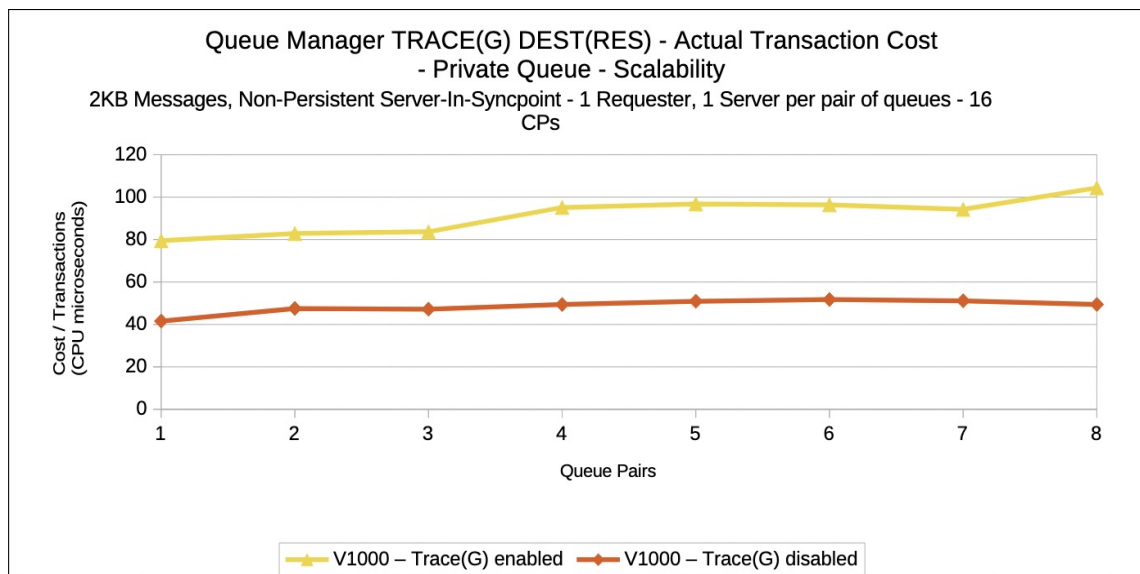


Chart: Queue manager TRACE(G) DEST(RES) - transaction cost



It must be emphasised that whilst the impact from enabling TRACE(G) for these workloads is effectively to double the overall transaction cost, this is primarily due to the high proportion of MQ API work when compared to the overall transaction and the simplistic nature of the application.

Channel initiator trace

The channel initiator trace tests are a variation on the [client pass through tests using SHARECNV\(0\)](#) with TRACE(CHINIT) enabled.

To enable a comparison of impact from enabling MQ's channel initiator trace, the results from the equivalent measurements without trace enabled are included.

Chart: Channel initiator TRACE(CHINIT) - throughput rate

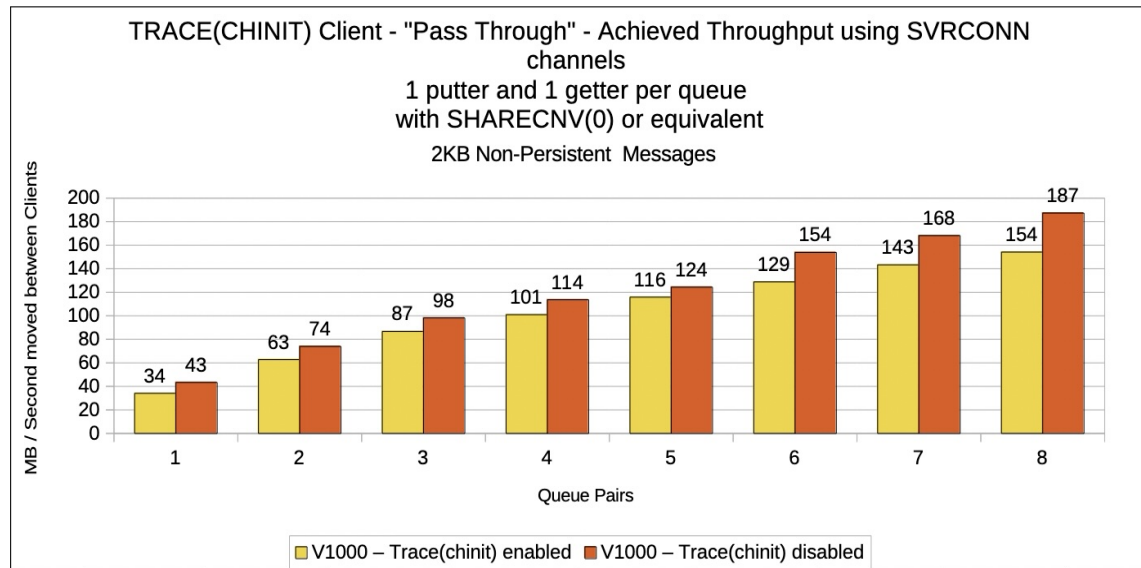
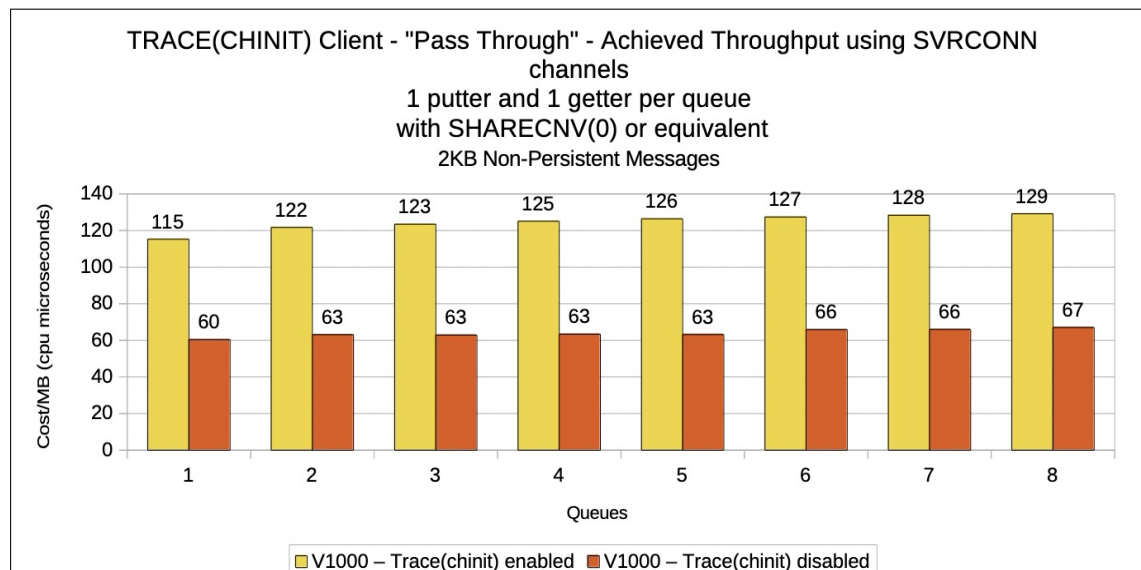


Chart: Channel initiator TRACE(CHINIT) - cost per MB



Note that there was sufficient CPU capacity such that the measurements with TRACE CHINIT enabled were able to achieve similar, or indeed higher, throughput to the measurements where trace was disabled. However, the additional cost would impact the throughput in a CPU constrained system.

Advanced Message Security

IBM MQ Advanced for z/OS 10.0 only

Background

IBM MQ Advanced Message Security (IBM MQ AMS) provides a high level of protection for sensitive data flowing through the MQ network with different levels of protection by using a public key cryptography model.

IBM MQ version 9.0 supplemented the existing two qualities of protection *Integrity* and *Privacy* with a third quality of protection, namely *Confidentiality*.

Integrity protection is provided by digital signing, which provides assurance on who created the message, and that the message has not been altered or tampered with.

Privacy protection is provided by a combination of digital signing and encryption. Encryption ensures that message data is viewable by only the intended recipient, or recipients.

Confidentiality protection is provided by encryption only.

IBM MQ AMS uses a combination of symmetric and asymmetric cryptographic routines to provide digital signing and encryption. Symmetric key operations are very fast in comparison to asymmetric key operations, which are CPU intensive and whilst some of the cost may be offloaded to cryptographic hardware such as Crypto Express8S, this can have a significant impact on the cost of protecting large numbers of messages with IBM MQ AMS.

- **Asymmetric cryptographic routines** as used by Integrity and Privacy

For example, when putting a signed message the message hash is signed using an asymmetric key operation. When getting a signed message, a further asymmetric key operation is used to verify the signed hash. Therefore, a minimum of two asymmetric key operations are required per message to sign and verify the message data. Some of this asymmetric cryptographic work can be offloaded to cryptographic hardware.

- **Asymmetric and symmetric cryptographic routines** as used by Privacy and Confidentiality

When putting an encrypted message, a symmetric key is generated and then encrypted using an asymmetric key operation for each intended recipient of the message. The message data is then encrypted with the symmetric key. When getting the encrypted message the intended recipient needs to use an asymmetric key operation to discover the symmetric key in use for the message. The symmetric key work cannot be offloaded to cryptographic hardware but will be performed in part by CPACF processors.

All three qualities of protection, therefore, contain varying elements of CPU intensive asymmetric key operations, which will significantly impact the maximum achievable messaging rate for applications putting and getting messages.

Confidentiality policies do, however, allow for symmetric key reuse over a sequence of messages.

Key reuse can significantly reduce the costs involved in encrypting a number of messages intended for the same recipient or recipients.

For example, when putting 10 encrypted messages to the same set of recipients, a symmetric key is generated. This key is encrypted for the first message using an asymmetric key operation for each of the intended recipients of the message.

Based upon policy controlled limits, the encrypted symmetric key can then be reused by subsequent messages that are intended for the same recipient(s). An application that is getting encrypted

messages can apply the same optimization, in that the application can detect when a symmetric key has not changed and avoid the expense of retrieving the symmetric key.

In this example 90% of the asymmetric key operations can be avoided by both the putting and getting applications reusing the same key.

IBM MQ for z/OS 9.1 further optimised the performance of AMS qualities of protection, for all 3 levels, with the most optimal offering from a performance perspective being AMS Confidentiality.

As was discussed in blog [Impact of key-size on AMS performance](#), the AMS measurements in the MQ performance reports have historically used 1024-bit key sizes in the certificate. This performance report has moved to using 2048-bit keys as these are the system default.

AMS regression test configuration

A simple request/reply workload is configured using a single z/OS queue manager with a pair of request and reply queues protected by an AMS security policy. In this instance there is 1 requester and 1 server task that run as long-running batch tasks.

The AMS policies defined are:

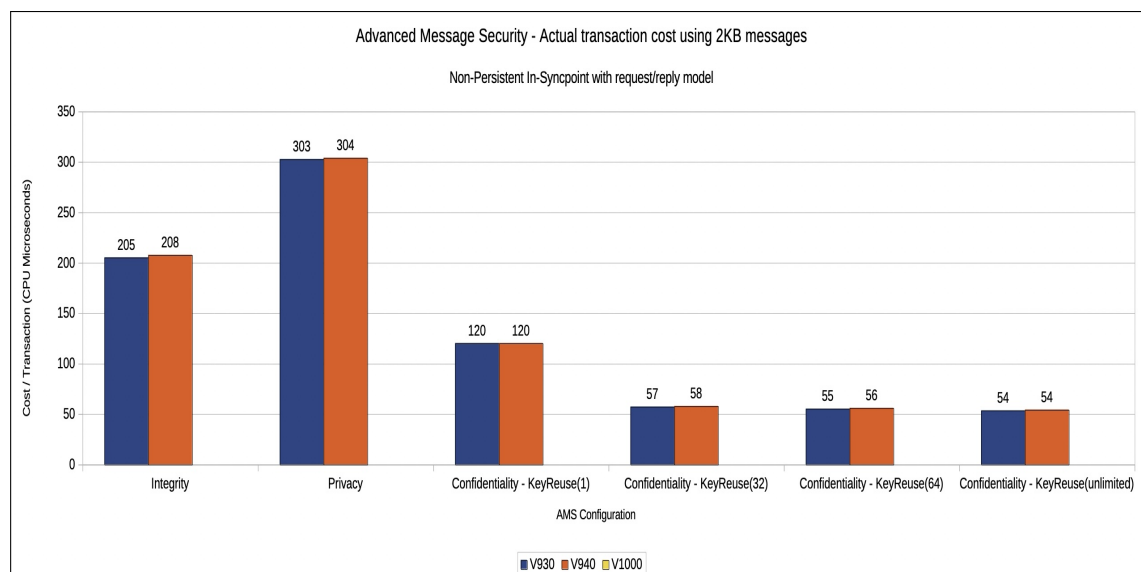
- **Integrity** Messages digitally signed with SHA256.
- **Privacy** Messages digitally signed with SHA256 and encrypted with AES256.
- **Confidentiality** Messages encrypted with AES256 and the key can be reused:
 - For 1 message
 - For 32 messages
 - For 64 messages
 - For an unlimited number of messages whilst the application remains connected.

Impact of AMS policy type on 2KB request/reply workload

Chart: Transaction rate for 2KB non-persistent workload with AMS protection applied to request and reply queues



Chart: Transaction cost for 2KB non-persistent workload with AMS protection applied to request and reply queues



Impact of AMS policy type on 64KB request/reply workload

Chart: Transaction rate for 64KB non-persistent workload with AMS protection applied to request and reply queues

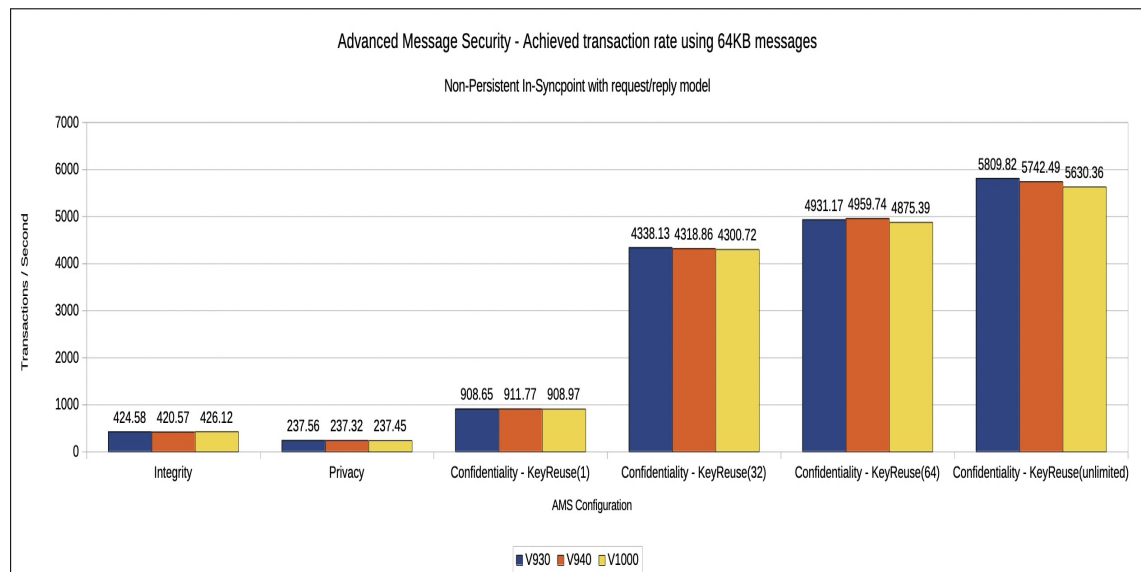
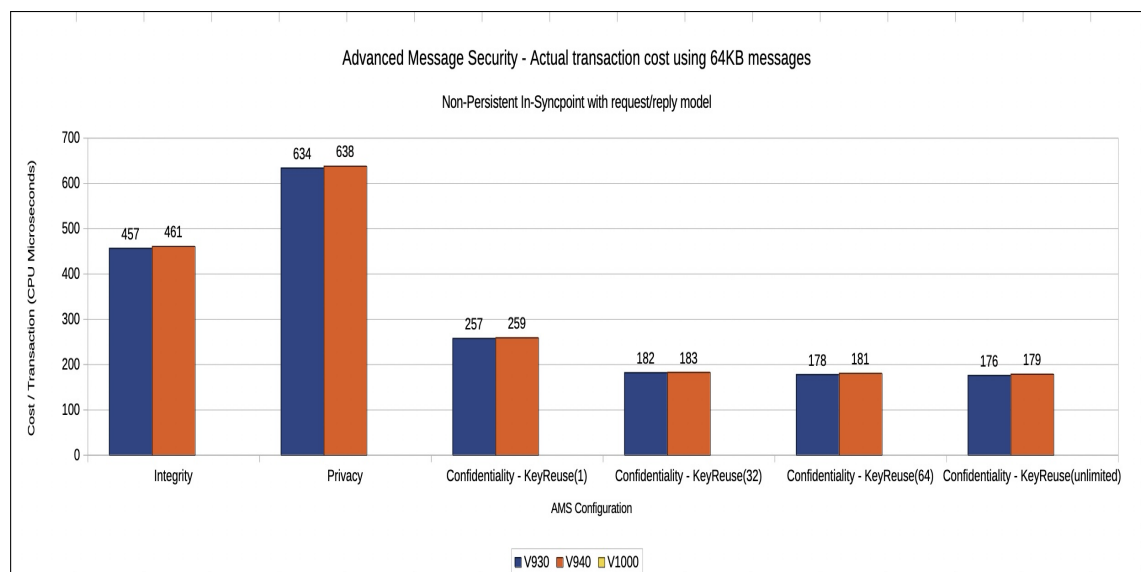


Chart: Transaction cost for 64KB non-persistent workload with AMS protection applied to request and reply queues



Impact of AMS policy type on 4MB request/reply workload

Chart: Transaction rate for 4MB non-persistent workload with AMS protection applied to request and reply queues

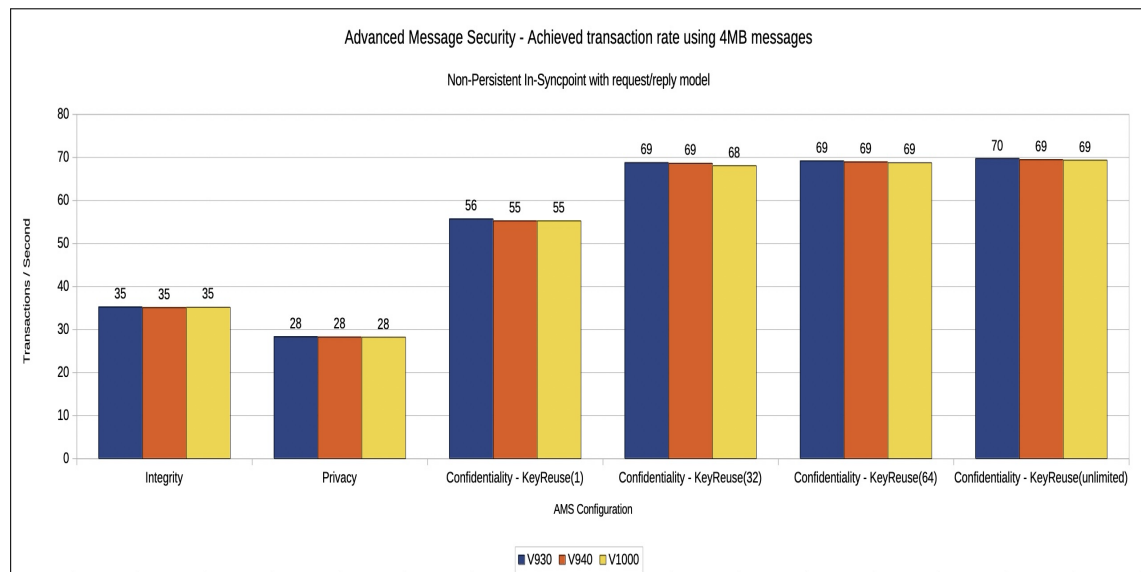
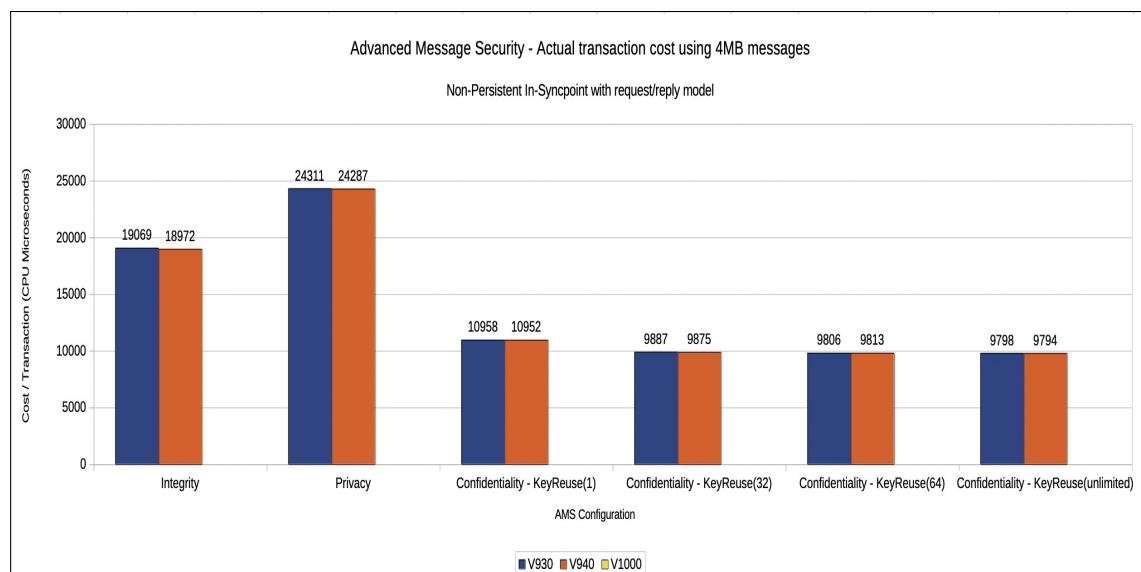


Chart: Transaction cost for 4MB non-persistent workload with AMS protection applied to request and reply queues



Appendix B

System configuration

The majority of the measurements in this report were run on our **IBM MQ Performance Sysplex running on IBM z17 (9175)** configured thus:

LPAR 1: 1-32 dedicated CP processors, plus 2 zIIP, 144GB of real storage.

LPAR 2: 1-3 dedicated CP processors, 48GB of real storage.

LPAR 3: 1-10 dedicated CP processors, plus 2 zIIP, 48GB of real storage.

Default Configuration:

3 dedicated processors on each LPAR, where each LPAR running z/OS 3.2 FMID HBB77F0.

Coupling Facility:

- Internal coupling facility with 4 dedicated processors.
- Coupling Facility running CFCC level 26 service level 00.43.
- Dynamic CF Dispatching off.
- 3 x ICP links between each z/OS LPAR and CF

DASD:

- FICON Express 32S connected DS8950F.
- 4 dedicated channel paths (shared across sysplex)
- HYPERPAV enabled.

System settings:

- zHPF enabled by default.
- HIPERDISPATCH enabled by default.
- LPARs 1 and 3 configured with different subnets such that tests moving messages over channels send data over 10GbE Network Express performance network.
 - SMC-R disabled by default between LPARs 1 and 3.
 - SMC-D disabled by default between LPARs 1 and 3.
- zEDC compression available by default - used with MQ channel attribute COMPMSG(ZLIBFAST).
- Crypto Express8 features configured thus:

- 1 x Accelerator, shared between LPARs 1,2 and 3.
- 2 x Coprocessor on LPAR1.
- 1 x Coprocessor on LPAR2.
- 2 x Coprocessor on LPAR3.

IBM MQ trace status:

- TRACE(GLOBAL) disabled.
- TRACE(CHINIT) disabled.
- TRACE(S) CLASS(1,3,4) enabled.
- TRACE(A) CLASS(3,4) enabled.

General information:

- Client machines:
 - 2 x IBM z 16 (8561) running RHEL 8.10 each with 128GB memory and 6 CPUs configured with 2 threads per core giving 12 CPUs per LPAR.
- Client tests used a 10GbE performance network.
- Other IBM products used:
 - IBM CICS TS 6.3.
 - Db2 for z/OS version 13.
 - IMS 15.5
 - [IBM MQ for z/OS 9.4](#) with latest service applied as of May 2026.
 - [IBM MQ for z/OS 9.3](#) with latest service applied as of May 2026.